BMS COLLEGE OF ENGINEERING, BENGALURU
Department of Electronics and Telecommunication Engineering

Wireless and Cellular Networks Laboratory

Lab Assignment — MATLAB Experiments 1 to 15

Course Code: 23ET7PCWCN · Semester VII · Academic Year 2026–27

USN YOUR_USN_HERE

Execution environment GNU Octave 11.3.0

Contents

1 Frequency Reuse, Co-channel Interference and Trunked Capacity 1
1.1 Aim . . . 1
1.2 Code . . . 1
1.3 Output . . . 3

2 Free Space, Log-Distance and Two-Ray Ground Path Loss Models 5
2.1 Aim . . . 5
2.2 Code . . . 5
2.3 Output . . . 7

3 Okumura–Hata Model and Coverage Prediction 9
3.1 Aim . . . 9
3.2 Code . . . 9
3.3 Output . . . 11

4 Log-Normal Shadowing and Rayleigh Fading 13
4.1 Aim . . . 13
4.2 Code . . . 13
4.3 Output . . . 16

5 Diversity Techniques — Selection, Equal Gain and Maximal Ratio Combining 19
5.1 Aim . . . 19
5.2 Code . . . 19
5.3 Output . . . 21

6 Adaptive Equalisation of a Multipath Channel using the LMS Algorithm 23
6.1 Aim . . . 23
6.2 Code . . . 23
6.3 Output . . . 26

7 GSM TDMA Frame Structure and Logical Channel Mapping 29
7.1 Aim . . . 29
7.2 Code . . . 29
7.3 Output . . . 32

8 GMSK Modulation and Spectral Analysis 34
8.1 Aim . . . 34
8.2 Code . . . 34
8.3 Output . . . 36

9 OFDM Transceiver — Orthogonality, Cyclic Prefix and Peak to Average Power Ratio 39
9.1 Aim . . . 39
9.2 Code . . . 39
9.3 Output . . . 42

10 MIMO — SISO, Alamouti Transmit Diversity and Spatial Multiplexing Capacity **45**
10.1 Aim . . . 45
10.2 Code . . . 45
10.3 Output . . . 48

11 LTE Resource Grid, Pilot Insertion and Channel Estimation **50**
11.1 Aim . . . 50
11.2 Code . . . 50
11.3 Output . . . 52

12 5G NR Numerology and Waveform Comparison **55**
12.1 Aim . . . 55
12.2 Code . . . 55
12.3 Output . . . 57

13 Wireless LAN Basic Service Set — Contention and Scheduled Access **60**
13.1 Aim . . . 60
13.2 Code . . . 60
13.3 Output . . . 63

14 Mobile Ad Hoc Network with an Omnidirectional Antenna Model **65**
14.1 Aim . . . 65
14.2 Code . . . 65
14.3 Output . . . 69

15 Optical Fibre Link — Attenuation, Power Budget, Digital and Analog Performance **72**
15.1 Aim . . . 72
15.2 Code . . . 72
15.3 Output . . . 75

Experiment 1

Frequency Reuse, Co-channel Interference and Trunked Capacity

Aim

To construct a hexagonal cellular layout, evaluate the co-channel reuse ratio and the signal to interference ratio for admissible cluster sizes, and determine the traffic capacity per cell using the Erlang B model.

Check value (lab manual): *For $N = 7$ the co-channel reuse ratio is $\sqrt{21} = 4.583$ and the signal to interference ratio is 18.66 dB at a path loss exponent of four.*

Code

```
% Experiment 1 : Frequency Reuse, Co-channel Interference and Trunked Capacity
clc; clear; close all;

%% Parameters
BW_total = 25e6;          % total duplex spectrum (Hz)
BW_ch    = 200e3;         % channel spacing (Hz)
n        = 4;             % path loss exponent
i0       = 6;             % first-tier co-channel interferers
SI_req   = 18;            % required S/I (dB)
GoS      = 0.02;          % grade of service
E_sub    = 0.025;         % traffic per subscriber (erlang)
Area     = 400;           % service area (km^2)
Density  = 800;           % subscribers per km^2

Ctot = floor(BW_total/BW_ch);
Nset = [3 4 7 9 12];

%% Reuse ratio, S/I, channels per cell, capacity
R = zeros(numel(Nset),7);
for k = 1:numel(Nset)
    N  = Nset(k);
    DR = sqrt(3*N);                       % co-channel reuse ratio
    SI = 10*log10(DR^n / i0);             % signal to interference ratio (dB)
    C  = floor(Ctot/N);                   % channels per cell
    A  = erlangb_traffic(C,GoS);          % offered traffic at 2 % blocking
    Ac = A*(1-GoS);                       % carried traffic
    S  = A/E_sub;                         % subscribers per cell
    R(k,:) = [N DR SI C A Ac S];
end

fprintf('Total duplex channels = %d\n\n',Ctot);
fprintf('  N     D/R    S/I(dB)   Ch/cell   Offered(E)   Carried(E)   Subs/cell\n');
fprintf(' %2d  %6.3f  %7.2f  %7d  %10.3f  %10.3f  %9.0f\n',R.');

feas = R(R(:,3)>=SI_req,:);
Nmin = feas(1,1);  Cmin = feas(1,4);  Smin = feas(1,7);
fprintf('\nMinimum feasible cluster size N = %d\n',Nmin);
fprintf('Channels per cell               = %d\n',Cmin);
fprintf('Subscribers per cell            = %.0f\n',Smin);

%% Trunking efficiency
i7 = find(R(:,1)==7); i12 = find(R(:,1)==12);
```

```
fprintf('\nN = 7 to N = 12 : channels fall %.1f %%, offered traffic falls %.1f %%\n' ,...
    100*(1-R(i12,4)/R(i7,4)), 100*(1-R(i12,5)/R(i7,5)));

%% Cell count for the given subscriber density
Ncell = ceil(Density*Area/Smin);
Acell = Area/Ncell;
Rcell = sqrt(2*Acell/(3*sqrt(3)));
fprintf('\nSubscribers to be served = %d\n',Density*Area);
fprintf('Cells required           = %d\n',Ncell);
fprintf('Cell radius              = %.3f km\n',Rcell);

%% Sectoring
fprintf('\nSectoring:\n');
for ii = [6 2 1]
    for N = [1 3 4 7 9 12 13 16 19]
        if 10*log10((3*N)^(n/2)/ii) >= SI_req, break; end
    end
    fprintf('  i0 = %d : N = %2d, channels/cell = %d\n',ii,N,floor(Ctot/N));
end

%% Figure 1 : frequency group assignment for N = 7
Rc = 1; q = []; r = [];
for a = -3:3
    for b = -3:3
        if abs(a+b) <= 3, q(end+1,1) = a; r(end+1,1) = b; end
    end
end
xc = sqrt(3)*Rc*(q + r/2);  yc = 1.5*Rc*r;
grp = mod(q + 3*r,7) + 1;
th  = (0:5)'*pi/3 + pi/6;
col = [0.85 0.33 0.31;0.33 0.55 0.78;0.47 0.72 0.42;0.95 0.75 0.30;...
       0.62 0.45 0.75;0.45 0.75 0.78;0.80 0.55 0.40];
figure; hold on;
for k = 1:numel(xc)
    patch(xc(k)+Rc*cos(th), yc(k)+Rc*sin(th), col(grp(k),:));
    text(xc(k),yc(k),sprintf('%d',grp(k)),'HorizontalAlignment','center');
end
axis equal off; title('Frequency Group Assignment, N = 7');

%% Figure 2 : S/I and capacity against cluster size
figure;
subplot(1,2,1);
plot(R(:,1),R(:,3),'o-','LineWidth',2); hold on;
plot([min(Nset) max(Nset)],[SI_req SI_req],'k--','LineWidth',1.5);
grid on; xlabel('Cluster size N'); ylabel('S/I (dB)');
legend('S/I','Requirement','Location','southeast');
title('Interference Performance');
subplot(1,2,2);
plot(R(:,1),R(:,7),'s-','LineWidth',2);
grid on; xlabel('Cluster size N'); ylabel('Subscribers per cell');
title('Traffic Capacity');

function Pb = erlangb(C,A)
% Blocking probability of a lost calls cleared system, stable recursion.
Pb = 1;
for k = 1:C
    Pb = (A*Pb)/(k + A*Pb);
```

```
end
end

function A = erlangb_traffic(C,Pt)
% Offered traffic carried by C channels at blocking probability Pt.
if C == 0, A = 0; return; end
lo = 0; hi = 2*C;
while erlangb(C,hi) < Pt, hi = 2*hi; end
for k = 1:200
    mid = 0.5*(lo+hi);
    if erlangb(C,mid) < Pt, lo = mid; else hi = mid; end
end
A = 0.5*(lo+hi);
end
```

Output

```
Total duplex channels = 125

   N     D/R    S/I(dB)   Ch/cell   Offered(E)   Carried(E)   Subs/cell
   3   3.000    11.30       41      31.916      31.277       1277
   4   3.464    13.80       31      22.827      22.370        913
   7   4.583    18.66       17      10.656      10.443        426
   9   5.196    20.85       13       7.402       7.253        296
  12   6.000    23.34       10       5.084       4.982        203

Minimum feasible cluster size N = 7
Channels per cell               = 17
Subscribers per cell            = 426

N = 7 to N = 12 : channels fall 41.2 %, offered traffic falls 52.3 %

Subscribers to be served = 320000
Cells required           = 751
Cell radius              = 0.453 km

Sectoring:
  i0 = 6 : N =  7, channels/cell = 17
  i0 = 2 : N =  4, channels/cell = 31
  i0 = 1 : N =  3, channels/cell = 41
```

Command Window Output

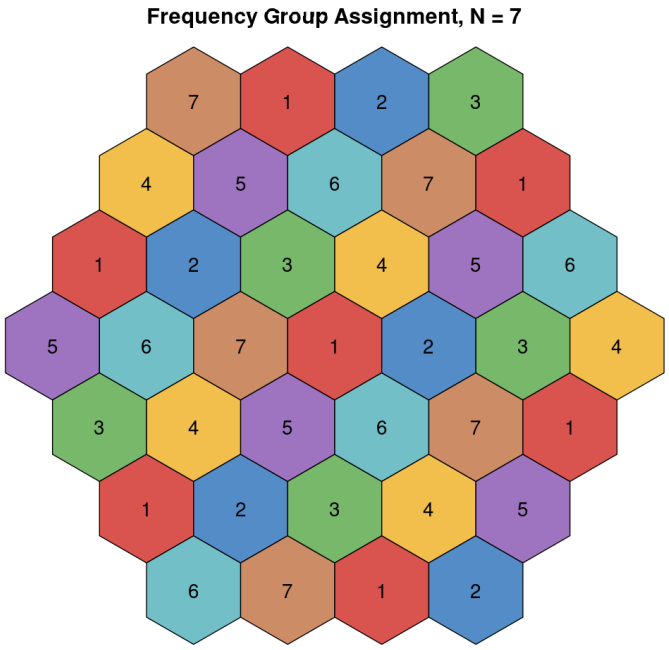


Figure 1.1

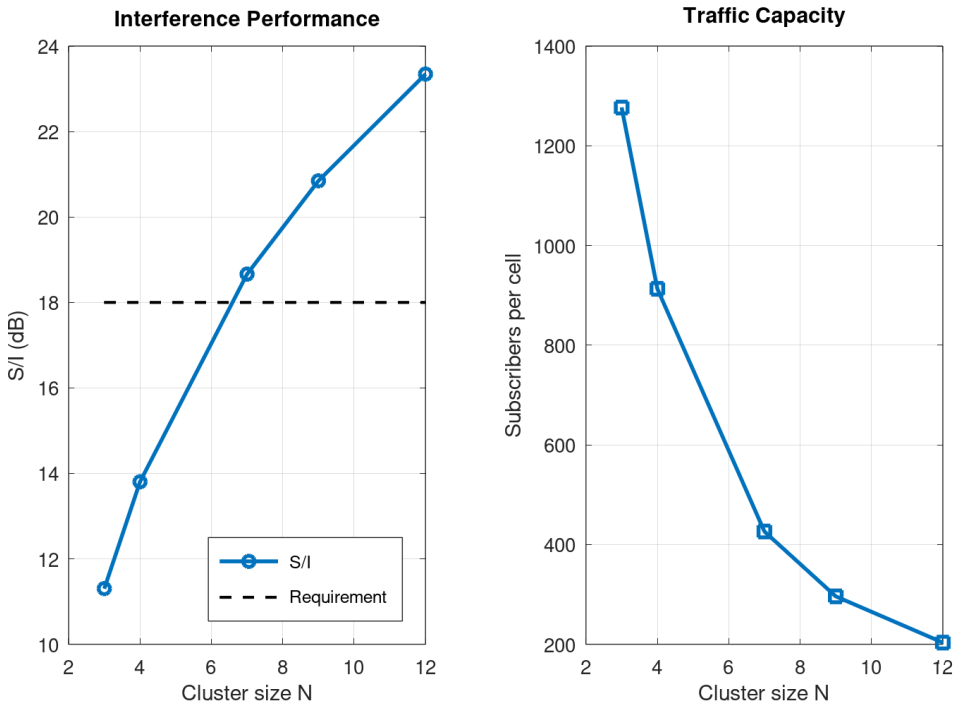


Figure 1.2

Experiment 2

Free Space, Log-Distance and Two-Ray Ground Path Loss Models

Aim

To implement the free space, log-distance and two-ray ground reflection path loss models, determine the two-ray crossover distance, and estimate the maximum cell radius for a specified receiver sensitivity.

Check value (lab manual): *The free space path loss at 1 km and 900 MHz is 91.53 dB.*

Code

```
% Experiment 2 : Free Space, Log-Distance and Two-Ray Ground Path Loss Models
clc; clear; close all;

%% Parameters
Pt   = 43;              % transmit power (dBm)
Gt   = 12;              % base station antenna gain (dBi)
Gr   = 0;               % mobile antenna gain (dBi)
fc   = 900e6;           % carrier frequency (Hz)
ht   = 30;              % base station antenna height (m)
hr   = 1.5;             % mobile antenna height (m)
d0   = 100;             % reference distance (m)
Sens = -104;            % receiver sensitivity (dBm)
c    = 3e8;
lambda = c/fc;

EIRP  = Pt + Gt + Gr;
maxPL = EIRP - Sens;

fprintf('Wavelength                    = %.4f m\n',lambda);
fprintf('Pt + Gt + Gr                  = %.1f dBm\n',EIRP);
fprintf('Maximum tolerable path loss = %.1f dB\n\n',maxPL);

d = logspace(1,5,4000);

%% Free space path loss
PL_free = 20*log10(4*pi*d/lambda);
PL_d0   = 20*log10(4*pi*d0/lambda);
fprintf('Free space path loss at 1 km = %.2f dB\n\n',20*log10(4*pi*1000/lambda));

%% Log-distance path loss
nvec   = [2 2.7 3.5 4];
PL_log = zeros(numel(nvec),numel(d));
for k = 1:numel(nvec)
    PL_log(k,:) = PL_d0 + 10*nvec(k)*log10(d/d0);
end

%% Two-ray ground reflection model
dc     = 4*pi*ht*hr/lambda;
dlos   = sqrt(d.^2 + (ht-hr)^2);
dref   = sqrt(d.^2 + (ht+hr)^2);
dphi   = 2*pi*(dref-dlos)/lambda;
Glin   = 10^((Gt+Gr)/10);
Efield = 1./dlos - (1./dref).*exp(-1j*dphi);
```

```
Pr_2ray = 10*log10(10^(Pt/10)*Glin*(lambda/(4*pi))^2 .* abs(Efield).^2);
Pr_asym = 10*log10(10^(Pt/10)*Glin*(ht^2*hr^2)./(d.^4));

fprintf('Two-ray crossover distance   = %.1f m (%.3f km)\n\n',dc,dc/1000);

%% Range at receiver sensitivity
fprintf('%-26s %10s %14s\n','Model','Exponent','Range (km)');
Rfree = d0*10^((maxPL - PL_d0)/20);
fprintf('%-26s %10.1f %14.2f\n','Free space',2.0,Rfree/1000);
for k = 1:numel(nvec)
    Rk = d0*10^((maxPL - PL_d0)/(10*nvec(k)));
    fprintf('%-26s %10.1f %14.2f\n',sprintf('Log-distance n = %.1f',nvec(k)),nvec(k),
    ↪ Rk/1000);
end
R2ray = 10^((EIRP + 20*log10(ht*hr) - Sens)/40);
fprintf('%-26s %10.1f %14.2f\n','Two-ray asymptotic',4.0,R2ray/1000);

%% Antenna height gain
fprintf('\nAntenna height gain 20log10(ht*hr) = %.2f dB\n',20*log10(ht*hr));
fprintf('Height 30 m to 45 m gains %.2f dB\n',20*log10(45/30));

%% Figure 1 : all models
Pr_free = EIRP - PL_free;
Pr_log  = EIRP - PL_log;
figure;
semilogx(d/1000,Pr_free,'LineWidth',2); hold on;
semilogx(d/1000,Pr_log(2,:),'LineWidth',1.6);
semilogx(d/1000,Pr_log(3,:),'LineWidth',1.6);
semilogx(d/1000,Pr_2ray,'LineWidth',1);
semilogx(d/1000,Pr_asym,'--','LineWidth',1.6);
semilogx([min(d) max(d)]/1000,[Sens Sens],'k:','LineWidth',1.8);
grid on; axis([0.01 100 -180 40]);
xlabel('Distance (km)'); ylabel('Received Power (dBm)');
legend('Free space','Log-distance n = 2.7','Log-distance n = 3.5',...
       'Two-ray exact','Two-ray asymptotic','Sensitivity','Location','southwest');
title('Received Power against Distance, 900 MHz');

%% Figure 2 : interference lobes below crossover
dz = linspace(50,4000,6000);
dl = sqrt(dz.^2+(ht-hr)^2); dr = sqrt(dz.^2+(ht+hr)^2);
Ez = 1./dl - (1./dr).*exp(-1j*2*pi*(dr-dl)/lambda);
Pz = 10*log10(10^(Pt/10)*Glin*(lambda/(4*pi))^2 .* abs(Ez).^2);
Pa = 10*log10(10^(Pt/10)*Glin*(ht^2*hr^2)./(dz.^4));
figure;
plot(dz/1000,Pz,'LineWidth',1.2); hold on;
plot(dz/1000,Pa,'--','LineWidth',1.8);
plot([dc dc]/1000,[min(Pz) max(Pz)],'k-.','LineWidth',1.4);
grid on; xlabel('Distance (km)'); ylabel('Received Power (dBm)');
legend('Two-ray exact','Asymptotic d^{-4}','Crossover distance');
title('Interference Lobes below the Crossover Distance');

%% Figure 3 : sensitivity of range to path loss exponent
nn = 2:0.05:5;
Rn = d0*10.^((maxPL - PL_d0)./(10*nn));
figure;
semilogy(nn,Rn/1000,'LineWidth',2); grid on;
xlabel('Path Loss Exponent n'); ylabel('Range (km)');
title('Range against Assumed Path Loss Exponent');
```

Output

```
Wavelength                    = 0.3333 m
Pt + Gt + Gr                  = 55.0 dBm
Maximum tolerable path loss = 159.0 dB

Free space path loss at 1 km = 91.53 dB

Two-ray crossover distance   = 1696.5 m (1.696 km)

Model                         Exponent     Range (km)
Free space                         2.0        2364.12
Log-distance n = 2.0               2.0        2364.12
Log-distance n = 2.7               2.7         173.68
Log-distance n = 3.5               3.5          31.57
Log-distance n = 4.0               4.0          15.38
Two-ray asymptotic                 4.0          63.33

Antenna height gain 20log10(ht*hr) = 33.06 dB
Height 30 m to 45 m gains 3.52 dB
```

Command Window Output

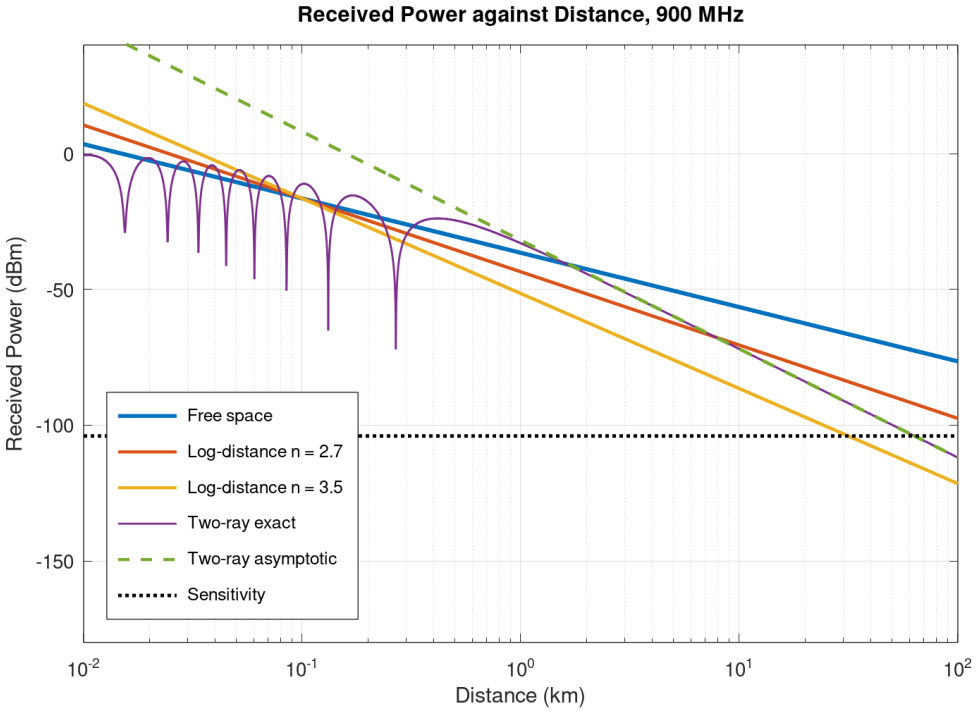


Figure 2.1



Figure 2.2

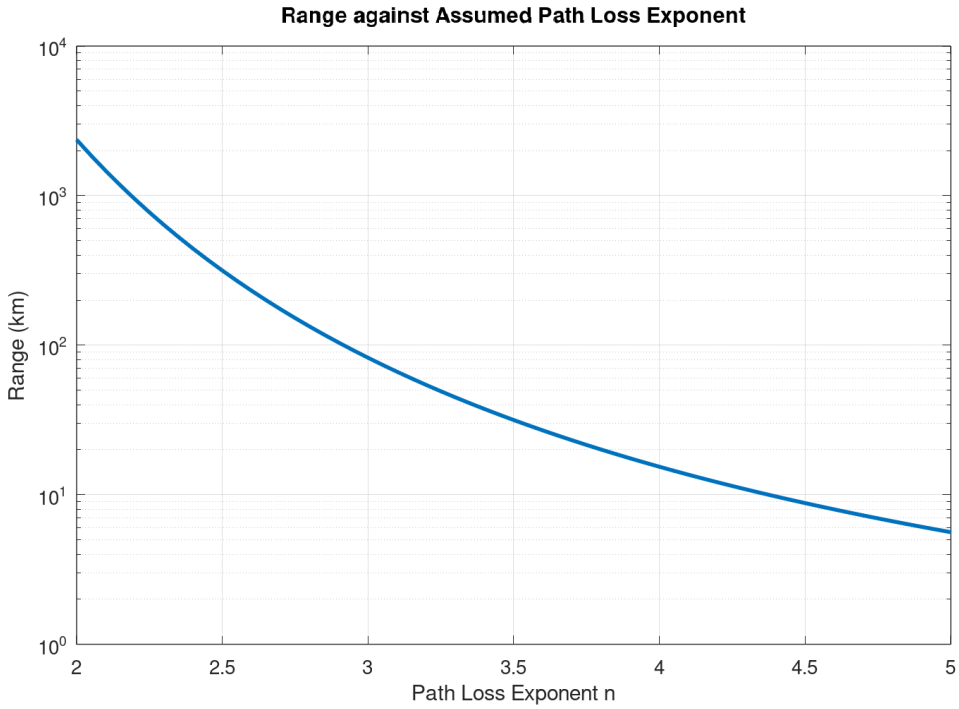


Figure 2.3

Experiment 3

Okumura–Hata Model and Coverage Prediction

Aim

To implement the Okumura–Hata model for urban, suburban and open environments, determine the coverage radius in each, and generate a coverage map for a hexagonal base station layout designed from the computed radius.

Check value (lab manual): *For a medium city at 900 MHz with a 30 m base station and a 1.5 m mobile, the urban median path loss at 1 km is 126.40 dB and the mobile antenna correction is 0.0159 dB.*

Code

```
% Experiment 3 : Okumura-Hata Model and Coverage Prediction
clc; clear; close all;

%% Parameters
f     = 900;            % carrier frequency (MHz)
hb    = 30;             % base station antenna height (m)
hm    = 1.5;            % mobile antenna height (m)
EIRP  = 55;             % Pt + Gt + Gr (dBm)
Sens  = -104;           % receiver sensitivity (dBm)
side  = 20;             % service area side (km)
Area  = side^2;
maxPL = EIRP - Sens;

d = 1:0.05:20;

%% Mobile antenna correction factor, medium city
a_hm = (1.1*log10(f) - 0.7)*hm - (1.56*log10(f) - 0.8);

%% Hata path loss
L_urban = 69.55 + 26.16*log10(f) - 13.82*log10(hb) - a_hm ...
          + (44.9 - 6.55*log10(hb))*log10(d);
L_sub   = L_urban - 2*(log10(f/28))^2 - 5.4;
L_open  = L_urban - 4.78*(log10(f))^2 + 18.33*log10(f) - 40.94;

nEff = (44.9 - 6.55*log10(hb))/10;

fprintf('Mobile antenna correction a(hm) = %.4f dB\n',a_hm);
fprintf('Effective path loss exponent    = %.4f\n',nEff);
fprintf('Maximum tolerable path loss     = %.1f dB\n\n',maxPL);

fprintf('%-12s %12s %12s %14s %14s\n','Environment','L(1 km)','L(10 km)','Radius (km)
    ↪ ','Sites');
L1 = [L_urban(1) L_sub(1) L_open(1)];
env = {'Urban','Suburban','Open'};
Rcov = zeros(1,3); Nsite = zeros(1,3);
for k = 1:3
    Rcov(k)  = 10^((maxPL - L1(k))/(10*nEff));
    Acell    = 1.5*sqrt(3)*Rcov(k)^2;
    Nsite(k) = max(1,ceil(Area/Acell));
    L10 = L1(k) + 10*nEff;
    fprintf('%-12s %12.2f %12.2f %14.2f %14d\n',env{k},L1(k),L10,Rcov(k),Nsite(k));
end
```

```
42
43 fprintf('\nSuburban correction = %.2f dB\n',L_urban(1)-L_sub(1));
44 fprintf('Open area correction = %.2f dB\n',L_urban(1)-L_open(1));
45
46 fprintf('\nValidity of the Hata model : 150-1500 MHz, hb 30-200 m, hm 1-10 m, d 1-20
    ↪ km\n');
47 for k = 1:3
48     if Rcov(k) > 20
49         fprintf('  %-9s radius %.1f km exceeds the 20 km validity limit\n',env{k},
    ↪ Rcov(k));
50     end
51 end
52
53 %% Effect of base station antenna height
54 fprintf('\n%8s %10s %14s %10s\n','hb (m)','n','Radius (km)','Sites');
55 for h = [30 45 60 100]
56     ne  = (44.9 - 6.55*log10(h))/10;
57     Lu1 = 69.55 + 26.16*log10(f) - 13.82*log10(h) ...
58           - ((1.1*log10(f)-0.7)*hm - (1.56*log10(f)-0.8));
59     Ls1 = Lu1 - 2*(log10(f/28))^2 - 5.4;
60     Rh  = 10^((maxPL - Ls1)/(10*ne));
61     fprintf('%8d %10.4f %14.2f %10d\n',h,ne,Rh,max(1,ceil(Area/(1.5*sqrt(3)*Rh^2))));
62 end
63
64 %% Figure 1 : path loss by environment
65 figure;
66 plot(d,L_urban,'LineWidth',2); hold on;
67 plot(d,L_sub,'LineWidth',2);
68 plot(d,L_open,'LineWidth',2);
69 plot([1 20],[maxPL maxPL],'k--','LineWidth',1.8);
70 grid on; xlabel('Distance (km)'); ylabel('Median Path Loss (dB)');
71 legend('Urban','Suburban','Open area','Maximum tolerable','Location','southeast');
72 title('Okumura-Hata Median Path Loss, 900 MHz, h_b = 30 m');
73
74 %% Figure 2 : coverage map built from the calculated coverage radius
75 % The layout is generated for the URBAN environment, whose coverage radius
76 % is the smallest of the three and therefore the demanding case. Base
77 % station spacing is sqrt(3)*R so that the hexagons of circumradius R
78 % tessellate without gaps.
79 Rcell = Rcov(1);                          % urban coverage radius (km)
80 q = []; r = [];
81 nring = ceil(side/(sqrt(3)*Rcell)) + 1;
82 for a = -nring:nring
83     for b = -nring:nring
84         if abs(a+b) <= nring, q(end+1,1) = a; r(end+1,1) = b; end
85     end
86 end
87 xb = sqrt(3)*Rcell*(q + r/2);  yb = 1.5*Rcell*r;
88 keep = abs(xb) <= side/2 + Rcell & abs(yb) <= side/2 + Rcell;
89 xb = xb(keep); yb = yb(keep);
90
91 g  = -side/2:0.1:side/2;
92 [GX,GY] = meshgrid(g,g);
93 Prx = -inf(size(GX));
94 for b = 1:numel(xb)
95     dd  = max(sqrt((GX-xb(b)).^2 + (GY-yb(b)).^2), 1);
96     Lb  = L_urban(1) + 10*nEff*log10(dd);
97     Prx = max(Prx, EIRP - Lb);
```

```
end
covered = 100*mean(Prx(:) >= Sens);

figure;
imagesc(g,g,Prx); set(gca,'YDir','normal'); hold on;
plot(xb,yb,'kv','MarkerFaceColor','k','MarkerSize',7);
colorbar; axis equal; axis([-side/2 side/2 -side/2 side/2]);
xlabel('Distance (km)'); ylabel('Distance (km)');
title(sprintf('Urban Coverage : %d sites, R = %.2f km, %.1f %% above sensitivity',...
      numel(xb),Rcell,covered));

fprintf('\nCoverage map, urban environment\n');
fprintf('  Coverage radius used      = %.2f km\n',Rcell);
fprintf('  Base station spacing      = %.2f km\n',sqrt(3)*Rcell);
fprintf('  Sites placed in the area  = %d\n',numel(xb));
fprintf('  Area above sensitivity    = %.2f %%\n',covered);

%% Estimated against actual site count
Aest = ceil(Area/(1.5*sqrt(3)*Rcell^2));
fprintf('\n  Site count from the area ratio = %d\n',Aest);
fprintf('  Sites actually placed          = %d\n',numel(xb));
fprintf('  The area ratio underestimates the requirement because the hexagons\n');
fprintf('  at the boundary of a square service area are only partly used.\n');
```

Output

```
Mobile antenna correction a(hm) = 0.0159 dB
Effective path loss exponent    = 3.5225
Maximum tolerable path loss     = 159.0 dB

Environment       L(1 km)     L(10 km)    Radius (km)          Sites
Urban              126.40       161.63           8.42              3
Suburban           116.46       151.69          16.13              1
Open                97.90       133.12          54.28              1

Suburban correction = 9.94 dB
Open area correction = 28.51 dB

Validity of the Hata model : 150-1500 MHz, hb 30-200 m, hm 1-10 m, d 1-20 km
  Open      radius 54.3 km exceeds the 20 km validity limit

  hb (m)          n    Radius (km)      Sites
      30     3.5225          16.13          1
      45     3.4071          20.89          1
      60     3.3253          25.37          1
     100     3.1800          36.72          1

Coverage map, urban environment
  Coverage radius used      = 8.42 km
  Base station spacing      = 14.59 km
  Sites placed in the area  = 7
  Area above sensitivity    = 100.00 %

  Site count from the area ratio = 3
  Sites actually placed          = 7
  The area ratio underestimates the requirement because the hexagons
  at the boundary of a square service area are only partly used.
```

Command Window Output

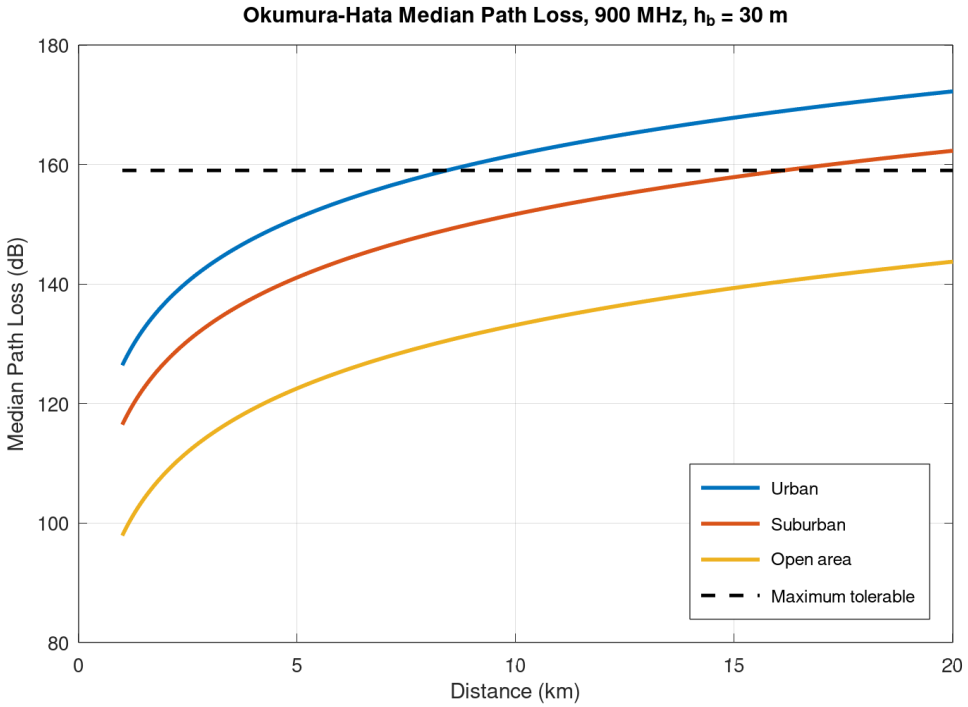


Figure 3.1

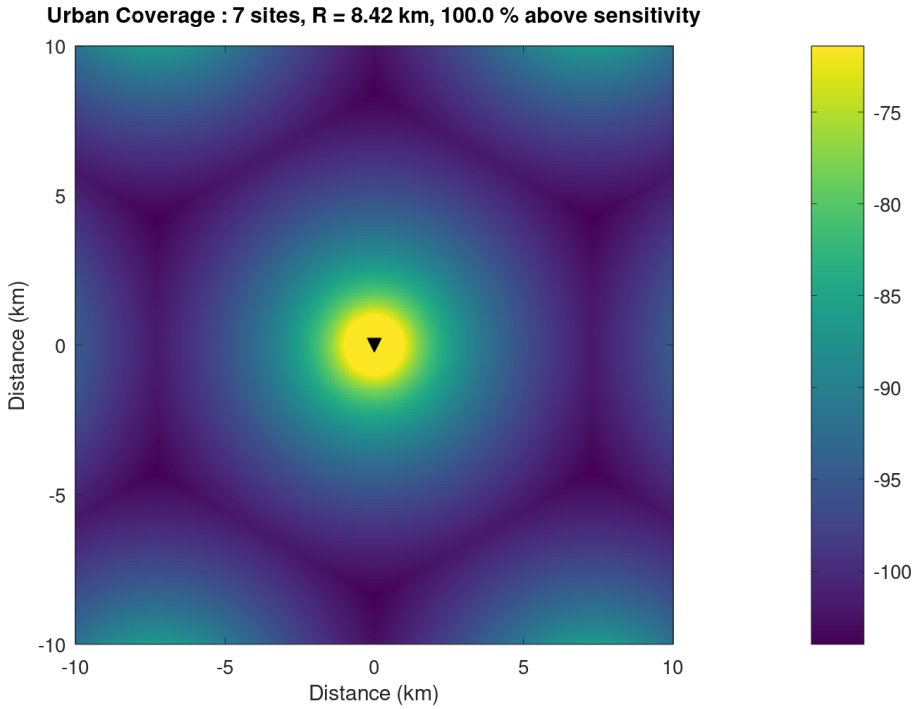


Figure 3.2

Experiment 4

Log-Normal Shadowing and Rayleigh Fading

Aim

To model log-normal shadowing and determine the fade margin required for a specified edge coverage probability, to generate a Rayleigh fading envelope by the sum of sinusoids method, and to measure the level crossing rate and average fade duration against theory.

Check value (lab manual): *The maximum Doppler shift at 60 km/h and 900 MHz is 50 Hz, and the fade margin for 90 per cent edge coverage at $\sigma = 8$ dB is 10.25 dB.*

Code

```
% Experiment 4 : Log-Normal Shadowing and Rayleigh Fading
clc; clear; close all;
randn('state',4); rand('state',4);

%% Parameters
fc     = 900e6;         % carrier frequency (Hz)
v      = 60;            % vehicle speed (km/h)
sigma  = 8;             % shadowing standard deviation (dB)
M      = 32;            % number of oscillators
c      = 3e8;
EIRP   = 55;  Sens = -104;  maxPL = EIRP - Sens;
L1_sub = 116.46;        % suburban Hata path loss at 1 km, from Experiment 3
nEff   = 3.5225;

fm = (v/3.6)*fc/c;
fprintf('Maximum Doppler shift = %.2f Hz\n\n',fm);

%% Part A : log-normal shadowing
X = sigma*randn(1,2e6);
fprintf('Shadowing standard deviation (simulated) = %.4f dB\n',std(X));

target = [0.75 0.90 0.95 0.99];
fprintf('\n%16s %14s %16s\n','Edge coverage','Margin (dB)','Simulated');
for k = 1:numel(target)
    Mk = sigma*sqrt(2)*erfcinv(2*(1-target(k)));
    fprintf('%15.0f%% %14.4f %15.4f\n',100*target(k),Mk,mean(X > -Mk));
end

M90 = sigma*sqrt(2)*erfcinv(2*0.10);
R0  = 10^((maxPL       - L1_sub)/(10*nEff));
R1  = 10^((maxPL - M90 - L1_sub)/(10*nEff));
fprintf('\nCoverage radius on median path loss = %.2f km\n',R0);
fprintf('Coverage radius with %.2f dB margin = %.2f km\n',M90,R1);
fprintf('Radius reduced by a factor of %.3f, sites increased by %.3f\n\n',R0/R1,(R0/
    ↪ R1)^2);

%% Part B : Rayleigh fading by the sum of sinusoids
fs   = 50*fm;                   % sampling rate
Tper = 2000;                    % record length in Doppler periods
N    = round(Tper*fs/fm);
t    = (0:N-1)'/fs;
n    = 1:M;
```

```

theta = 2*pi*rand - pi;
phi   = 2*pi*rand(1,M) - pi;
psi   = 2*pi*rand(1,M) - pi;
alpha = (2*pi*n - pi + theta)/(4*M);

arg = 2*pi*fm*(t*cos(alpha)) + repmat(phi,N,1);
hc  = sqrt(2/M)*(cos(arg)*cos(psi)');
hs  = sqrt(2/M)*(cos(arg)*sin(psi)');
h   = hc + 1j*hs;
r   = abs(h);
rms = sqrt(mean(r.^2));

fprintf('Mean power E|h|^2 = %.4f  (theory 1.0000)\n',mean(r.^2));
fprintf('Envelope mean     = %.4f  (theory %.4f)\n',mean(r),sqrt(pi)/2);
fprintf('E[r^4]/E[r^2]^2   = %.4f  (theory 2.0000)\n\n',mean(r.^4)/mean(r.^2)^2);

%% Autocorrelation against Clarke theory
maxlag = round(3*fs/fm);
hz  = h - mean(h);
Rxx = zeros(maxlag+1,1);
for k = 0:maxlag
    Rxx(k+1) = mean(conj(hz(1:end-k)).*hz(1+k:end));
end
Rxx = real(Rxx)/real(Rxx(1));
tau = (0:maxlag)'/fs;
J0  = besselj(0,2*pi*fm*tau);
fprintf('Maximum |ACF - J0(2*pi*fm*tau)| = %.4f\n\n',max(abs(Rxx-J0)));

%% Level crossing rate and average fade duration
rho_dB = [-20 -15 -10 -5 0 3];
fprintf('%8s %12s %12s %14s %14s\n','rho(dB)','NR sim','NR theory','AFD sim (ms)','
    ↪ AFD thy (ms)');
NRs = zeros(size(rho_dB)); AFDs = NRs; NRt = NRs; AFDt = NRs;
for k = 1:numel(rho_dB)
    rho   = 10^(rho_dB(k)/20);
    level = rho*rms;
    below = r < level;
    up    = sum(~below(2:end) & below(1:end-1));
    NRs(k)  = up/(N/fs);
    AFDs(k) = (sum(below)/fs)/up;
    NRt(k)  = sqrt(2*pi)*fm*rho*exp(-rho^2);
    AFDt(k) = (exp(rho^2)-1)/(rho*fm*sqrt(2*pi));
    fprintf('%8.0f %12.2f %12.2f %14.3f %14.3f\n',...
            rho_dB(k),NRs(k),NRt(k),AFDs(k)*1e3,AFDt(k)*1e3);
end

fprintf('\nGSM frame duration = 4.615 ms\n');
for vp = [5 60]
    fmp = (vp/3.6)*fc/c;
    rho = 10^(-10/20);
    afd = (exp(rho^2)-1)/(rho*fmp*sqrt(2*pi));
    fprintf('At %2d km/h : fm = %5.2f Hz, AFD at -10 dB = %7.3f ms\n',vp,fmp,afd*1e3)
    ↪ ;
end

%% Figure 1 : fading envelope
ns = round(20*fs/fm);
```

```
figure;
plot(t(1:ns)*1000,20*log10(r(1:ns)/rms),'LineWidth',1); hold on;
plot([0 t(ns)*1000],[-10 -10],'r--','LineWidth',1.5);
grid on; xlabel('Time (ms)'); ylabel('Envelope relative to rms (dB)');
legend('Rayleigh envelope','-10 dB threshold','Location','southwest');
title(sprintf('Fading Envelope at %d km/h, f_m = %.0f Hz',v,fm));

%% Figure 2 : envelope distribution
figure;
[cnt,ctr] = hist(r/rms,80);
bar(ctr,cnt/(sum(cnt)*(ctr(2)-ctr(1))),1); hold on;
xr = linspace(0,max(ctr),300);
plot(xr,2*xr.*exp(-xr.^2),'r','LineWidth',2);
grid on; xlabel('Envelope / rms'); ylabel('Probability Density');
legend('Simulated','Rayleigh pdf'); title('Envelope Distribution');

%% Figure 3 : Doppler spectrum
seg = 4096; nseg = floor(N/seg);
w = 0.5 - 0.5*cos(2*pi*(0:seg-1)'/(seg-1));
P = zeros(seg,1);
for k = 1:nseg
    P = P + abs(fft(h((k-1)*seg+1:k*seg).*w)).^2;
end
P = fftshift(P/nseg);
fax = (-seg/2:seg/2-1)'*fs/seg;
figure;
plot(fax,10*log10(P/max(P)),'LineWidth',1.2); hold on;
plot([-fm -fm],[-60 5],'r--',[fm fm],[-60 5],'r--','LineWidth',1.5);
grid on; axis([-2*fm 2*fm -60 5]);
xlabel('Frequency (Hz)'); ylabel('Normalised PSD (dB)');
title('Doppler Spectrum');

%% Figure 4 : level crossing rate and fade duration
figure;
subplot(1,2,1);
semilogy(rho_dB,NRs,'o-','LineWidth',1.6); hold on;
semilogy(rho_dB,NRt,'k--','LineWidth',1.6);
grid on; xlabel('\rho (dB)'); ylabel('Crossings per second');
legend('Simulated','Theory','Location','southeast'); title('Level Crossing Rate');
subplot(1,2,2);
semilogy(rho_dB,AFDs*1e3,'o-','LineWidth',1.6); hold on;
semilogy(rho_dB,AFDt*1e3,'k--','LineWidth',1.6);
grid on; xlabel('\rho (dB)'); ylabel('Duration (ms)');
legend('Simulated','Theory','Location','southeast'); title('Average Fade Duration');
```

Output

```
Maximum Doppler shift = 50.00 Hz
Shadowing standard deviation (simulated) = 8.0031 dB
   Edge coverage    Margin (dB)        Simulated
             75%         5.3959          0.7504
             90%        10.2524          0.9000
             95%        13.1588          0.9500
             99%        18.6108          0.9900
Coverage radius on median path loss = 16.13 km
Coverage radius with 10.25 dB margin = 8.25 km
Radius reduced by a factor of 1.955, sites increased by 3.820
Mean power E|h|^2 = 0.9976  (theory 1.0000)
Envelope mean     = 0.8878  (theory 0.8862)
E[r^4]/E[r^2]^2   = 1.9733  (theory 2.0000)
Maximum |ACF - J0(2*pi*fm*tau)| = 0.0151
 rho(dB)       NR sim    NR theory   AFD sim (ms)   AFD thy (ms)
     -20        12.23        12.41          0.828          0.802
     -15        21.28        21.59          1.469          1.442
     -10        35.10        35.86          2.597          2.654
      -5        52.18        51.37          5.066          5.277
       0        47.03        46.11         13.369         13.710
       3        24.13        24.07         35.952         35.892
GSM frame duration = 4.615 ms
At  5 km/h : fm =  4.17 Hz, AFD at -10 dB =  31.843 ms
At 60 km/h : fm = 50.00 Hz, AFD at -10 dB =   2.654 ms
```

Command Window Output

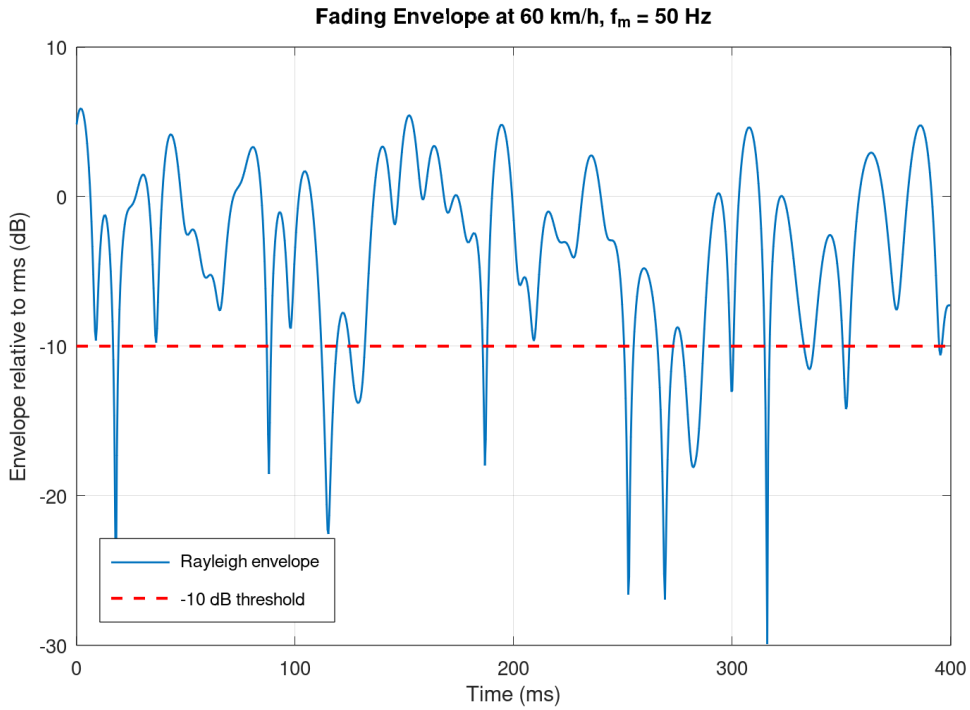


Figure 4.1

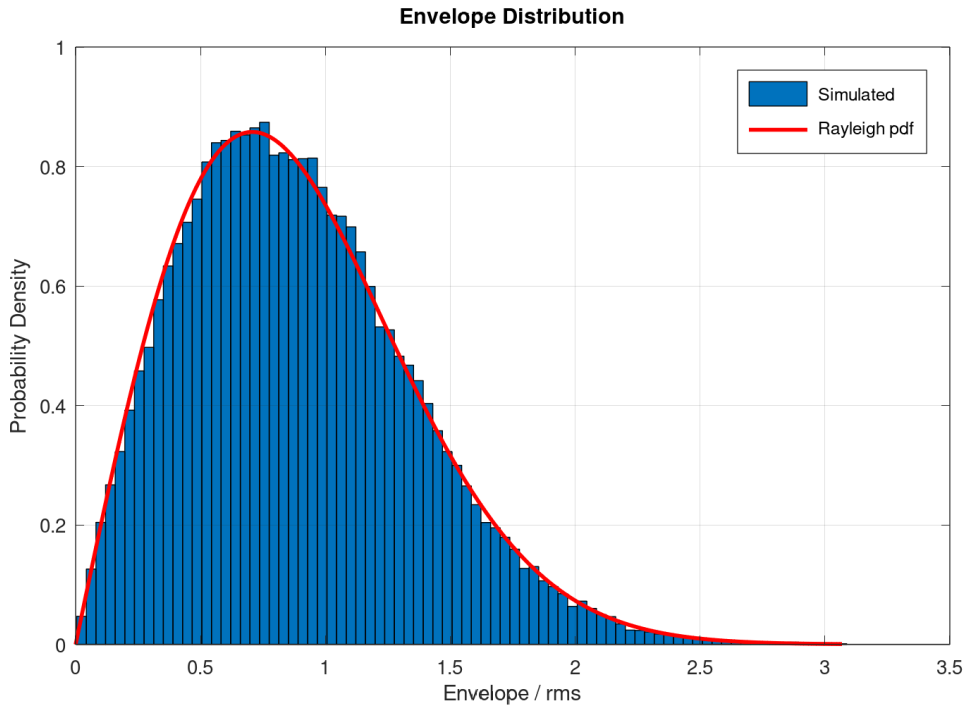


Figure 4.2

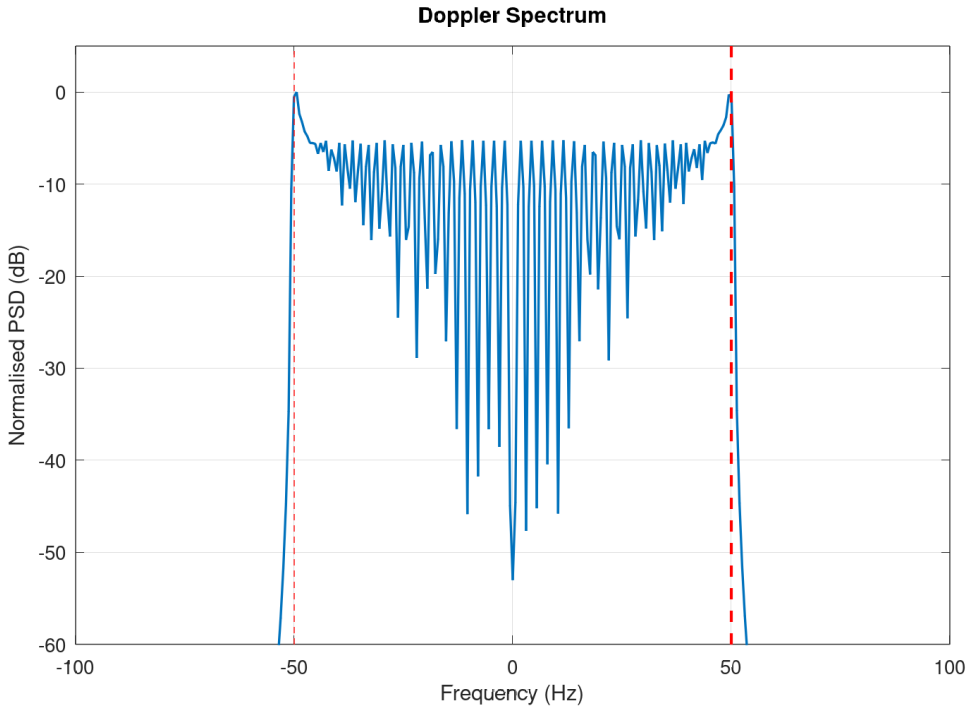


Figure 4.3

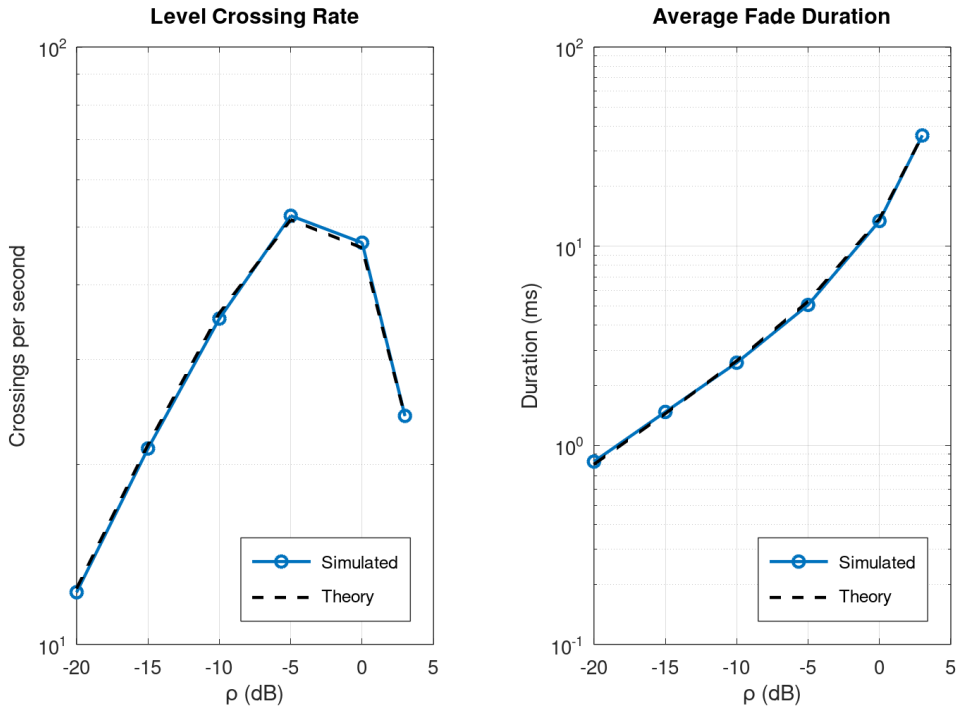

Level Crossing Rate
Crossings per second
ρ (dB)
Simulated
Theory
Average Fade Duration
Duration (ms)
ρ (dB)
Simulated
Theory

Figure 4.4

Experiment 5

Diversity Techniques — Selection, Equal Gain and Maximal Ratio Combining

Aim

To implement selection, equal gain and maximal ratio combining over independent Rayleigh fading branches, and to determine the outage probability, the diversity order and the array gain as functions of the number of branches.

Check value (lab manual): *At four branches the maximal ratio combining array gain is 6.02 dB, which is exactly* $10\log_{10}4$*, and the equal gain value is 5.26 dB.*

Code

```
% Experiment 5 : Diversity Techniques - Selection, Equal Gain and Maximal Ratio
    ↪ Combining
clc; clear; close all;
randn('state',5);

%% Parameters
Lmax    = 4;                    % maximum number of diversity branches
Nsym    = 1e6;                  % channel realisations
gbar_dB = 0:2:30;               % average branch SNR (dB)
gth_dB  = 5;                    % outage threshold (dB)
gth     = 10^(gth_dB/10);

fprintf('Outage threshold = %d dB\n',gth_dB);
fprintf('Branch fading    = Rayleigh, branches independent\n\n');

Pout_sc  = zeros(Lmax,numel(gbar_dB));
Pout_egc = Pout_sc;  Pout_mrc = Pout_sc;
Tout_sc  = Pout_sc;  Tout_mrc = Pout_sc;

for iL = 1:Lmax
    L = iL;
    for ig = 1:numel(gbar_dB)
        gbar = 10^(gbar_dB(ig)/10);

        % Rayleigh branch amplitudes, unit mean power
        hmat = (randn(L,Nsym) + 1j*randn(L,Nsym))/sqrt(2);
        g    = gbar*abs(hmat).^2;                      % branch SNRs

        % Selection combining : strongest branch
        g_sc  = max(g,[],1);

        % Maximal ratio combining : SNRs add
        g_mrc = sum(g,1);

        % Equal gain combining : amplitudes add coherently
        g_egc = (sum(abs(hmat),1).^2)*gbar/L;

        Pout_sc(iL,ig)  = mean(g_sc  < gth);
        Pout_egc(iL,ig) = mean(g_egc < gth);
        Pout_mrc(iL,ig) = mean(g_mrc < gth);

```

```
        % Closed form results
        x = gth/gbar;
        Tout_sc(iL,ig)  = (1 - exp(-x))^L;
        s = 0;
        for k = 0:L-1, s = s + x^k/factorial(k); end
        Tout_mrc(iL,ig) = 1 - exp(-x)*s;
    end
end

%% Monte Carlo resolution
fprintf('Realisations per point = %g, resolution = %.1e\n\n',Nsym,1/Nsym);

%% Outage probability at a representative average SNR
idx = find(gbar_dB == 14);
fprintf('Outage probability at average branch SNR of %d dB\n',gbar_dB(idx));
fprintf('%10s %12s %12s %12s %12s %12s\n','Branches','SC sim','SC theory','EGC sim','
    ↪ MRC sim','MRC theory');
for L = 1:Lmax
    fprintf('%10d %12.3e %12.3e %12.3e %12.3e %12.3e\n',L,...
        Pout_sc(L,idx),Tout_sc(L,idx),Pout_egc(L,idx),Pout_mrc(L,idx),Tout_mrc(L,idx)
    ↪ );
end

%% Diversity order from the slope of the outage curve
% Taken from the closed form results, which remain accurate below the
% Monte Carlo resolution of 1/Nsym.
fprintf('\nDiversity order, magnitude of the outage slope per decade of SNR\n');
fprintf('%10s %12s %12s %12s\n','Branches','SC','EGC','MRC');
i1 = find(gbar_dB == 20); i2 = find(gbar_dB == 30);
j1 = find(gbar_dB == 8);  j2 = find(gbar_dB == 14);
for L = 1:Lmax
    slope_sc  = log10(Tout_sc(L,i1))  - log10(Tout_sc(L,i2));
    slope_mrc = log10(Tout_mrc(L,i1)) - log10(Tout_mrc(L,i2));
    % EGC has no closed form, so its slope is taken from simulated points
    % that lie well above the Monte Carlo resolution.
    slope_egc = (log10(Pout_egc(L,j1)) - log10(Pout_egc(L,j2))) / ...
                ((gbar_dB(j2)-gbar_dB(j1))/10);
    fprintf('%10d %12.2f %12.2f %12.2f\n',L,slope_sc,slope_egc,slope_mrc);
end

%% Array gain : average combined SNR relative to one branch
fprintf('\nArray gain, average output SNR relative to a single branch (dB)\n');
fprintf('%9s %9s %9s %9s %9s %9s %9s\n','Branches',...
        'SC sim','SC thy','EGC sim','EGC thy','MRC sim','MRC thy');
gbar = 1;
for L = 1:Lmax
    hmat  = (randn(L,Nsym) + 1j*randn(L,Nsym))/sqrt(2);
    g     = gbar*abs(hmat).^2;
    a_sc  = 10*log10(mean(max(g,[],1))/gbar);
    a_egc = 10*log10(mean(sum(abs(hmat),1).^2)/(L*gbar));
    a_mrc = 10*log10(mean(sum(g,1))/gbar);
    t_sc  = 10*log10(sum(1./(1:L)));
    t_egc = 10*log10(1 + (L-1)*pi/4);
    t_mrc = 10*log10(L);
    fprintf('%9d %9.2f %9.2f %9.2f %9.2f %9.2f %9.2f\n',...
            L,a_sc,t_sc,a_egc,t_egc,a_mrc,t_mrc);
end

```

```
% Note : at very low outage probabilities a finite Monte Carlo simulation
% may record no outage events. Such points are not zero probability, they
% are simply below the resolution of 1/Nsym, and are excluded from the plot.

%% Replace unresolved zeros so that the logarithmic plots are valid
Pout_sc(Pout_sc==0)   = NaN;
Pout_egc(Pout_egc==0) = NaN;
Pout_mrc(Pout_mrc==0) = NaN;

%% Figure 1 : outage probability
figure;
mk = {'o-','s-','d-','^-'};
for L = 1:Lmax
    semilogy(gbar_dB,Pout_mrc(L,:),mk{L},'LineWidth',1.6); hold on;
end
semilogy(gbar_dB,Tout_mrc(1,:),'k--','LineWidth',1.4);
for L = 2:Lmax
    semilogy(gbar_dB,Tout_mrc(L,:),'k--','LineWidth',1.4);
end
grid on; axis([0 30 1e-6 1]);
xlabel('Average Branch SNR (dB)'); ylabel('Outage Probability');
legend('MRC L = 1','MRC L = 2','MRC L = 3','MRC L = 4','Theory','Location','southwest
    ↪ ');
title('Maximal Ratio Combining : Outage Probability');

%% Figure 2 : comparison of the three combiners at L = 3
figure;
semilogy(gbar_dB,Pout_sc(1,:),'k-','LineWidth',2); hold on;
semilogy(gbar_dB,Pout_sc(3,:),'o-','LineWidth',1.6);
semilogy(gbar_dB,Pout_egc(3,:),'s-','LineWidth',1.6);
semilogy(gbar_dB,Pout_mrc(3,:),'d-','LineWidth',1.6);
grid on; axis([0 30 1e-6 1]);
xlabel('Average Branch SNR (dB)'); ylabel('Outage Probability');
legend('No diversity','Selection L = 3','Equal gain L = 3','Maximal ratio L = 3',...
       'Location','southwest');
title('Comparison of Combining Techniques');
```

Output

```
Outage threshold = 5 dB
Branch fading    = Rayleigh, branches independent

Realisations per point = 1e+06, resolution = 1.0e-06

Outage probability at average branch SNR of 14 dB
  Branches       SC sim    SC theory      EGC sim      MRC sim   MRC theory
         1    1.181e-01    1.183e-01    1.181e-01    1.181e-01    1.183e-01
         2    1.412e-02    1.399e-02    9.705e-03    7.433e-03    7.290e-03
         3    1.681e-03    1.655e-03    5.350e-04    3.220e-04    3.027e-04
         4    1.910e-04    1.958e-04    2.000e-05    8.000e-06    9.465e-06

Diversity order, magnitude of the outage slope per decade of SNR
  Branches           SC          EGC          MRC
         1         0.99         0.87         0.99
         2         1.99         1.78         1.99
         3         2.98         2.75         2.99
         4         3.98         3.78         3.99

Array gain, average output SNR relative to a single branch (dB)
 Branches    SC sim    SC thy   EGC sim   EGC thy   MRC sim   MRC thy
        1      0.01      0.00      0.01      0.00      0.01      0.00
        2      1.76      1.76      2.51      2.52      3.01      3.01
        3      2.63      2.63      4.10      4.10      4.77      4.77
        4      3.18      3.19      5.26      5.26      6.02      6.02
```

Command Window Output

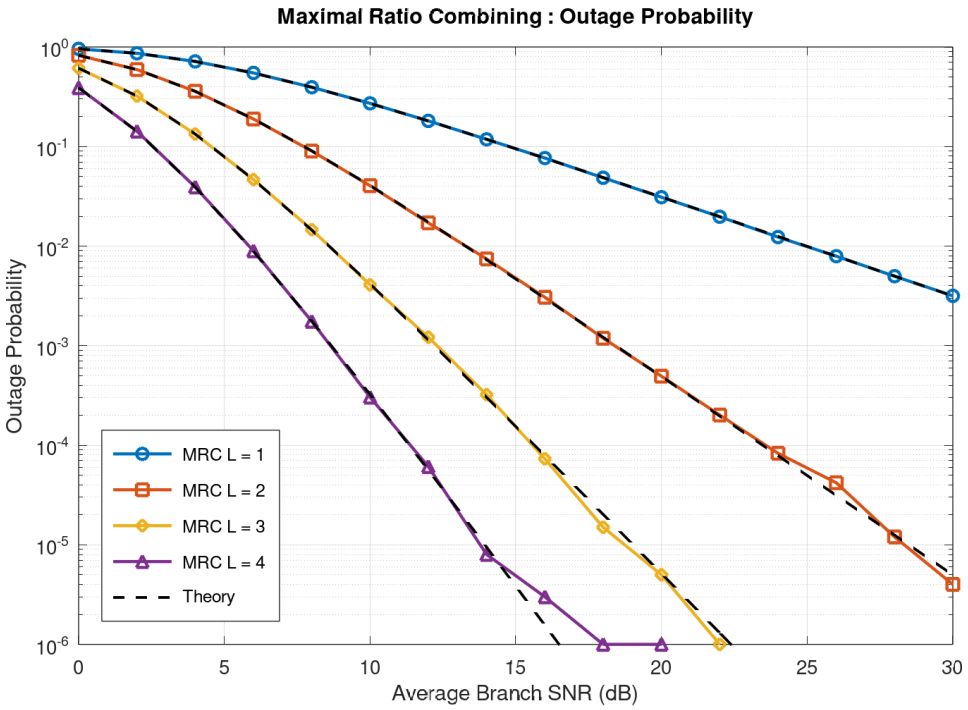


Figure 5.1

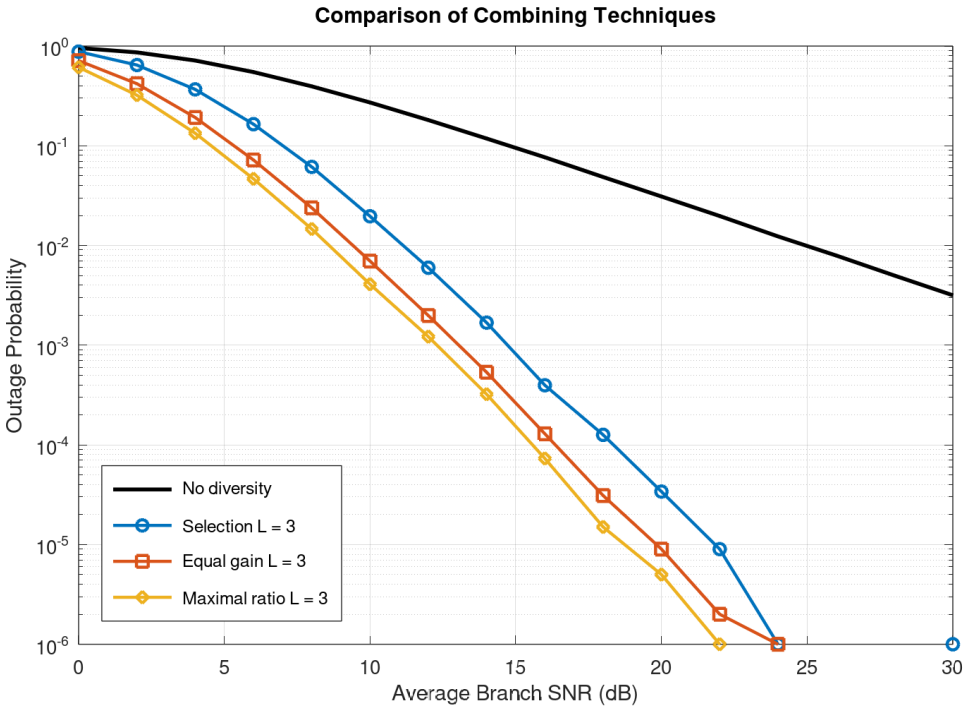


Figure 5.2

Experiment 6

Adaptive Equalisation of a Multipath Channel using the LMS Algorithm

Aim

To implement a transversal equaliser adapted by the least mean square algorithm, to compare the converged tap weights with the Wiener solution, and to determine the reduction in intersymbol interference and in bit error rate.

Check value (lab manual): *For the stated channel the largest eigenvalue of the autocorrelation matrix is 2.2012, giving a step size stability limit of 0.9086, and the Wiener minimum mean square error is 0.0277.*

Code

```
% Experiment 6 : Adaptive Equalisation of a Multipath Channel using the LMS Algorithm
clc; clear; close all;
randn('state',6);

%% Parameters
h      = [0.3 0.9 0.3];      % multipath channel impulse response
Meq    = 11;                 % equaliser length (taps)
D      = 6;                  % decision delay (samples)
mu     = 0.01;               % LMS step size
SNR_dB = 20;                 % signal to noise ratio
Nsym   = 5000;               % symbols per run
Nrun   = 200;                % runs averaged for the learning curve

Lh   = numel(h);
sigv2 = 10^(-SNR_dB/10)*sum(h.^2);

fprintf('Channel impulse response = [%s]\n',num2str(h));
fprintf('Equaliser length         = %d taps\n',Meq);
fprintf('Decision delay           = %d samples\n',D);
fprintf('Step size                = %.3f\n',mu);
fprintf('Noise variance           = %.5f\n\n',sigv2);

%% Wiener solution for comparison
H = zeros(Meq, Meq+Lh-1);
for k = 1:Meq
    H(k,k:k+Lh-1) = h;
end
R = H*H' + sigv2*eye(Meq);
p = H(:,D+1);
w_opt = R\p;
Jmin  = 1 - p'*w_opt;

fprintf('Minimum mean square error (Wiener) = %.6f\n',Jmin);
fprintf('Maximum eigenvalue of R            = %.4f\n',max(eig(R)));
fprintf('Step size stability limit          = %.4f\n\n',2/max(eig(R)));

%% LMS adaptation, averaged learning curve
J = zeros(1,Nsym);
for run = 1:Nrun
    s = 2*randi([0 1],1,Nsym+Meq+Lh) - 1;
    x = filter(h,1,s) + sqrt(sigv2)*randn(size(s));
```

```
    w = zeros(Meq,1);
    for n = Meq:Nsym
        u = x(n:-1:n-Meq+1).';
        y = w'*u;
        e = s(n-D) - y;
        w = w + mu*e*u;
        J(n) = J(n) + e^2;
    end
end
J = J/Nrun;

fprintf('Steady state mean square error (LMS) = %.6f\n',mean(J(end-499:end)));
fprintf('Tap error norm ||w - w_opt||         = %.6f\n\n',norm(w - w_opt));

%% Bit error rate against signal to noise ratio
% The channel peak occurs at tap index 1, so the unequalised detector is
% aligned to that tap. The equalised detector is aligned to the delay D.
[~,ipk]   = max(abs(h));  Dch = ipk - 1;
Ntest     = 2e5;
snr_sweep = 0:2:20;
ber_no    = zeros(size(snr_sweep));
ber_eq    = zeros(size(snr_sweep));
for k = 1:numel(snr_sweep)
    sv2 = 10^(-snr_sweep(k)/10)*sum(h.^2);
    s   = 2*randi([0 1],1,Ntest) - 1;
    x   = filter(h,1,s) + sqrt(sv2)*randn(1,Ntest);

    Rk  = H*H' + sv2*eye(Meq);
    wk  = Rk\p;
    y   = filter(wk,1,x);

    b   = (s+1)/2;
    ber_no(k) = mean(b(1:end-Dch) ~= (x(1+Dch:end) > 0));
    ber_eq(k) = mean(b(1:end-D)   ~= (y(1+D:end)   > 0));
end

fprintf('%10s %16s %16s\n','SNR (dB)','BER no equaliser','BER equalised');
for k = 1:numel(snr_sweep)
    fprintf('%10d %16.5f %16.5f\n',snr_sweep(k),ber_no(k),ber_eq(k));
end
fprintf('\n');

s = 2*randi([0 1],1,Ntest) - 1;
x = filter(h,1,s) + sqrt(sigv2)*randn(1,Ntest);
y = filter(w_opt,1,x);

%% Residual intersymbol interference
cimp = conv(h,w_opt);
[pk,ip] = max(abs(cimp));
isi_before = (sum(abs(h)) - max(abs(h)))/max(abs(h));
isi_after  = (sum(abs(cimp)) - pk)/pk;
fprintf('Intersymbol interference before equalisation = %.4f\n',isi_before);
fprintf('Intersymbol interference after equalisation  = %.4f\n',isi_after);

%% Figure 1 : learning curve
figure;
semilogy(J,'LineWidth',1.2); hold on;
semilogy([1 Nsym],[Jmin Jmin],'r--','LineWidth',1.8);
```

```
grid on; axis([1 Nsym 1e-2 2]);
xlabel('Iteration'); ylabel('Mean Square Error');
legend('LMS, averaged over 200 runs','Wiener minimum');
title('LMS Learning Curve');

%% Figure 2 : bit error rate against signal to noise ratio
figure;
semilogy(snr_sweep,ber_no,'o-','LineWidth',1.8); hold on;
semilogy(snr_sweep,max(ber_eq,1e-6),'s-','LineWidth',1.8);
semilogy(snr_sweep,0.5*erfc(sqrt(10.^(snr_sweep/10))),'k--','LineWidth',1.5);
grid on; axis([0 20 1e-6 1]);
xlabel('SNR (dB)'); ylabel('Bit Error Rate');
legend('Without equaliser','With equaliser','AWGN bound','Location','southwest');
title('Effect of Equalisation on Bit Error Rate');

%% Figure 3 : combined channel and equaliser response
figure;
subplot(1,2,1);
stem(0:Lh-1,h,'filled','LineWidth',1.5); grid on;
xlabel('Tap'); ylabel('Amplitude'); title('Channel Impulse Response');
subplot(1,2,2);
stem(0:numel(cimp)-1,cimp,'filled','LineWidth',1.5); grid on;
xlabel('Tap'); ylabel('Amplitude'); title('Channel and Equaliser Combined');

%% Figure 4 : signal at three points
figure;
subplot(3,1,1); plot(s(1:200),'LineWidth',1.2); grid on;
ylabel('Amplitude'); title('Transmitted BPSK Symbols');
subplot(3,1,2); plot(x(1:200),'LineWidth',1.2); grid on;
ylabel('Amplitude'); title('After the Multipath Channel');
subplot(3,1,3); plot(y(1+D:200+D),'LineWidth',1.2); grid on;
xlabel('Symbol Index'); ylabel('Amplitude'); title('Equaliser Output');

%% Figure 5 : equaliser tap weights
figure;
stem(0:Meq-1,w_opt,'filled','LineWidth',1.5); hold on;
stem(0:Meq-1,w,'r','LineWidth',1);
grid on; xlabel('Tap'); ylabel('Weight');
legend('Wiener solution','LMS after adaptation');
title('Equaliser Tap Weights');
```

Output

```
Channel impulse response = [0.3         0.9         0.3]
Equaliser length         = 11 taps
Decision delay           = 6 samples
Step size                = 0.010
Noise variance           = 0.00990

Minimum mean square error (Wiener) = 0.027724
Maximum eigenvalue of R            = 2.2012
Step size stability limit          = 0.9086

Steady state mean square error (LMS) = 0.030536
Tap error norm ||w - w_opt||         = 0.021974

  SNR (dB) BER no equaliser    BER equalised
         0          0.20539          0.20144
         2          0.15794          0.15625
         4          0.11982          0.11510
         6          0.08700          0.07568
         8          0.06172          0.04211
        10          0.04365          0.01833
        12          0.02832          0.00561
        14          0.01646          0.00097
        16          0.00703          0.00007
        18          0.00203          0.00000
        20          0.00032          0.00000

Intersymbol interference before equalisation = 0.6667
Intersymbol interference after equalisation  = 0.0768
```

Command Window Output

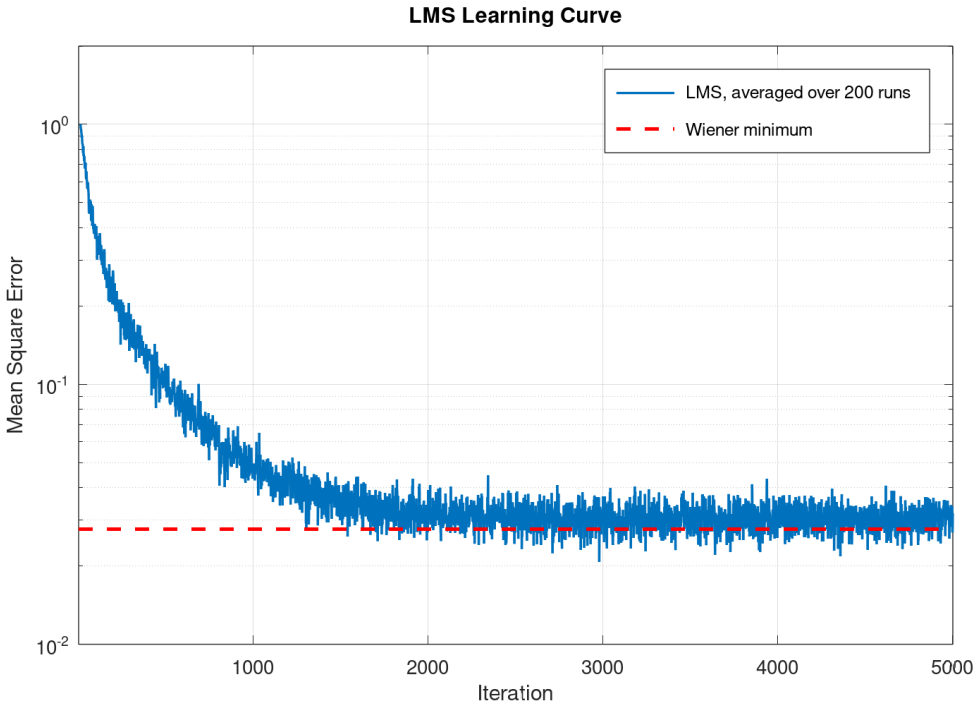


Figure 6.1

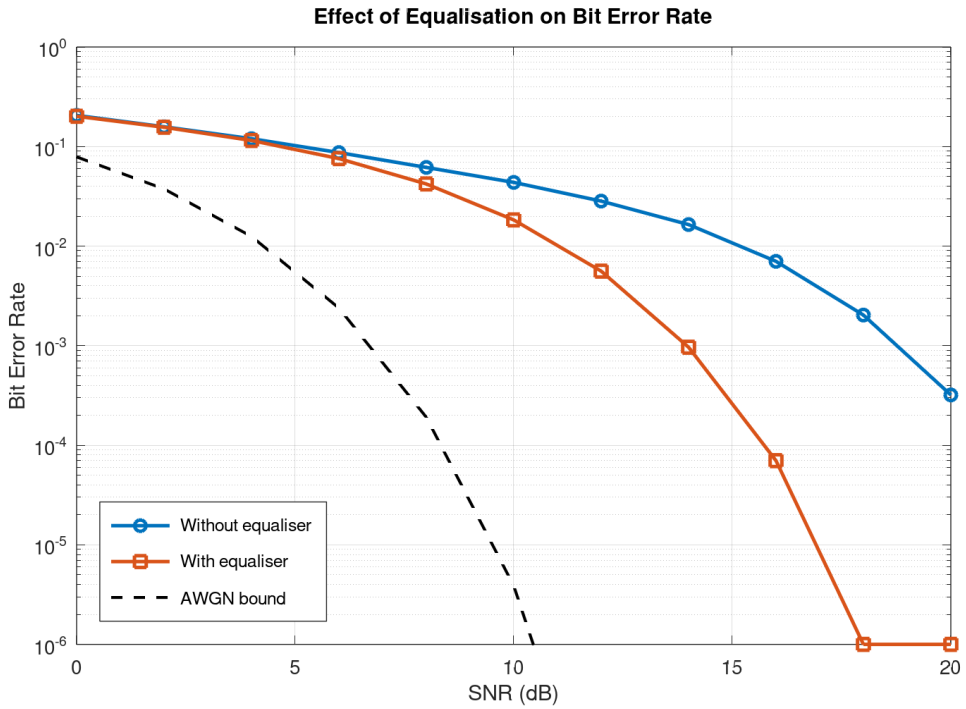


Figure 6.2

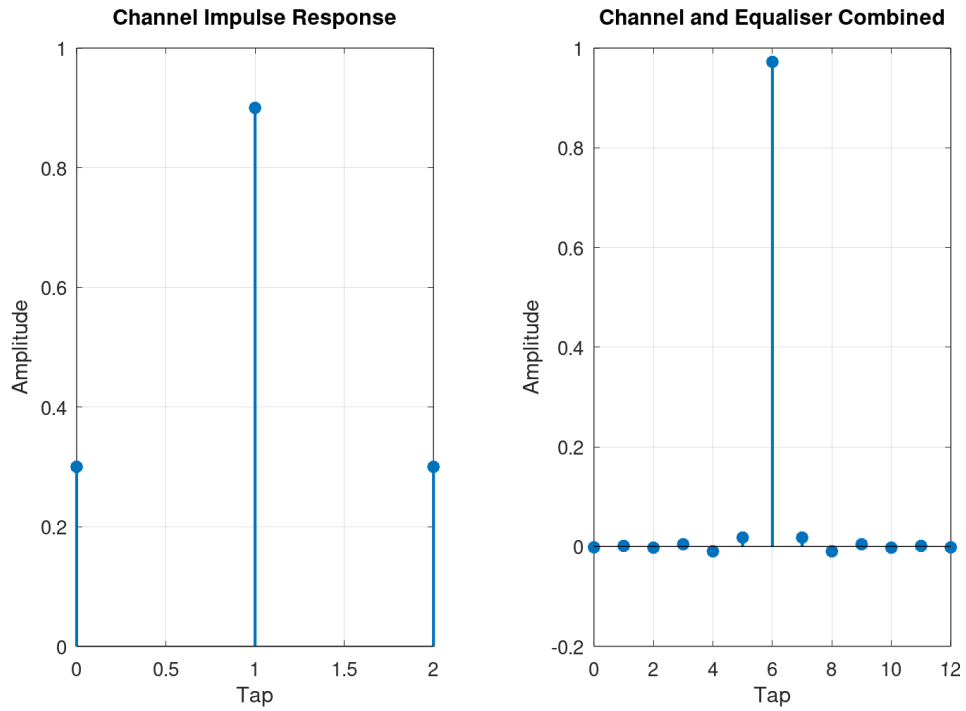


Figure 6.3

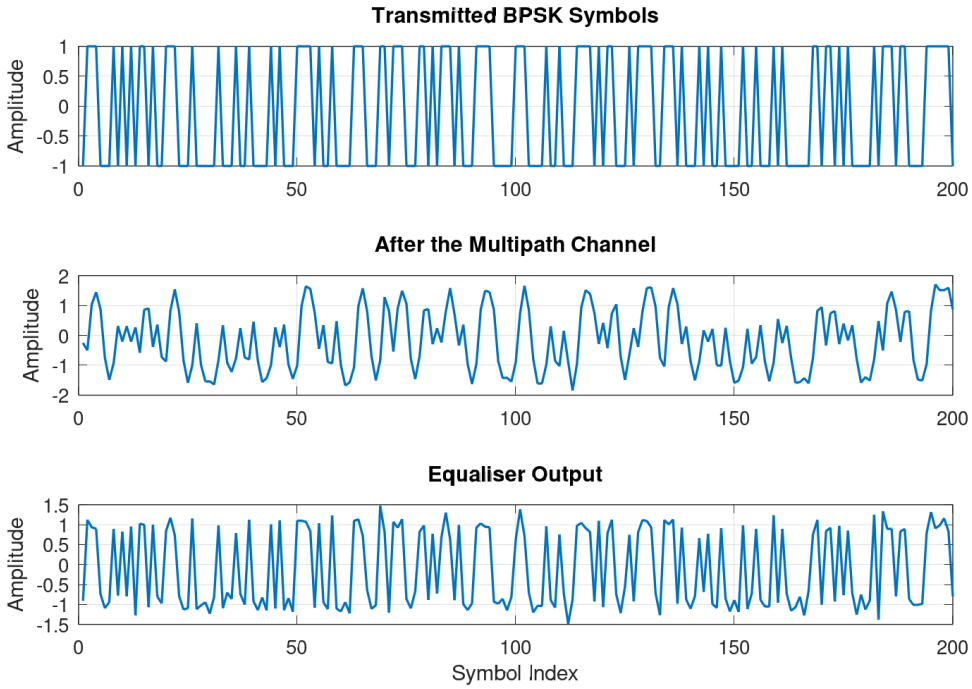


Figure 6.4

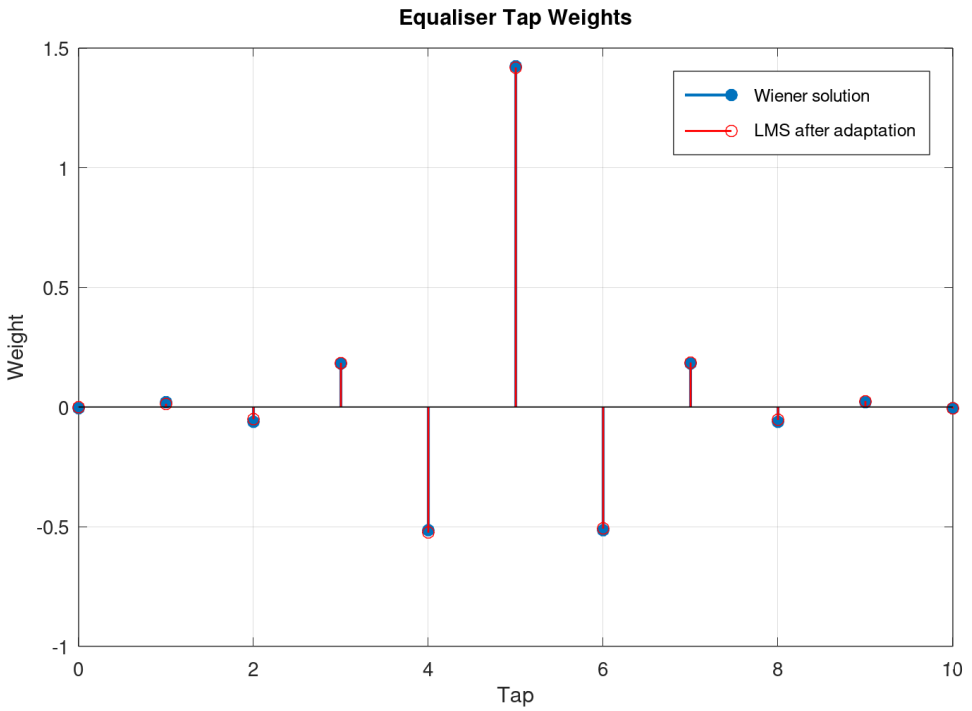


Figure 6.5

Experiment 7

GSM TDMA Frame Structure and Logical Channel Mapping

Aim

To construct the GSM normal burst, compute the complete TDMA timing hierarchy, and map the logical channels onto the traffic and control multiframes.

Check value (lab manual): *The TDMA frame duration is 4.6154 ms and the traffic multiframe is exactly 120 ms.*

Code

```
% Experiment 7 : GSM TDMA Frame Structure and Logical Channel Mapping
clc; clear; close all;

%% Parameters
Rb    = 270833.33;        % gross bit rate (bit/s)
Nslot = 8;                % timeslots per TDMA frame
Nbits = 148;              % bits in a normal burst
Guard = 8.25;             % guard period (bit periods)
Tb    = 1/Rb;

%% Burst composition
tail1 = 3; data1 = 57; steal1 = 1; tsc = 26; steal2 = 1; data2 = 57; tail2 = 3;
total = tail1+data1+steal1+tsc+steal2+data2+tail2;

fprintf('GSM normal burst composition\n');
fprintf('  Tail bits                 : %d + %d\n',tail1,tail2);
fprintf('  Encrypted data bits       : %d + %d\n',data1,data2);
fprintf('  Stealing flags            : %d + %d\n',steal1,steal2);
fprintf('  Training sequence         : %d\n',tsc);
fprintf('  Total                     : %d bits\n',total);
fprintf('  Guard period              : %.2f bit periods\n\n',Guard);

%% Timing hierarchy
Tbit     = Tb;
Tburst   = (Nbits + Guard)*Tb;
Tframe   = Nslot*Tburst;
Tmulti26 = 26*Tframe;
Tmulti51 = 51*Tframe;
Tsuper   = 51*Tmulti26;
Thyper   = 2048*Tsuper;

fprintf('GSM timing hierarchy\n');
fprintf('  Bit period                : %.4f us\n',Tbit*1e6);
fprintf('  Timeslot (burst) duration : %.4f us\n',Tburst*1e6);
fprintf('  TDMA frame (8 slots)      : %.4f ms\n',Tframe*1e3);
fprintf('  Traffic multiframe (26)   : %.2f ms\n',Tmulti26*1e3);
fprintf('  Control multiframe (51)   : %.2f ms\n',Tmulti51*1e3);
fprintf('  Superframe (51x26)        : %.3f s\n',Tsuper);
fprintf('  Hyperframe (2048 super)   : %.2f s\n\n',Thyper);

%% Useful bit rate of one traffic channel
useful = (data1+data2)*Nslot/Nslot;          % bits per burst carrying data
Rtch   = 114*24/(Tmulti26);                  % 24 traffic bursts per 26 frames
```

```
fprintf('Traffic channel gross rate  : %.2f kbit/s\n',114/Tframe/1000);
fprintf('Traffic channel useful rate : %.2f kbit/s\n\n',Rtch/1000);

%% 26-frame traffic multiframe map
tmap = repmat({'T'},1,26);
tmap{13} = 'S';           % SACCH
tmap{26} = 'I';           % idle
fprintf('26-frame traffic multiframe (T traffic, S SACCH, I idle)\n  ');
fprintf('%s ',tmap{:}); fprintf('\n\n');

%% 51-frame control multiframe map, downlink timeslot 0, non-combined
% Configuration: BCCH carrier, timeslot 0, downlink, non-combined, that is
% the SDCCH is carried on a separate timeslot. Frame numbers below are
% counted from 0 and may be checked directly against 3GPP TS 45.002.
cmap = cell(1,51);
for k = 1:51, cmap{k} = 'C'; end
cmap{1}  = 'F'; cmap{2}  = 'S';
cmap{11} = 'F'; cmap{12} = 'S';
cmap{21} = 'F'; cmap{22} = 'S';
cmap{31} = 'F'; cmap{32} = 'S';
cmap{41} = 'F'; cmap{42} = 'S';
cmap{51} = 'I';
for k = 3:6, cmap{k} = 'B'; end
fprintf('51-frame control multiframe, downlink timeslot 0, non-combined\n');
fprintf('  F FCCH   S SCH   B BCCH   C CCCH (PCH and AGCH)   I idle\n  ');
fprintf('%s ',cmap{:}); fprintf('\n');
lab = 'FSBCI';
for q = 1:numel(lab)
    idx = find(cellfun(@(z) z==lab(q), cmap)) - 1;
    fprintf('  %c at frames :',lab(q));
    fprintf(' %d',idx); fprintf('\n');
end
fprintf('  CCCH occupies %d of the 51 frames\n\n',sum(cellfun(@(z) z=='C',cmap)));

%% Logical channel summary
fprintf('%-8s %-12s %-10s %s\n','Channel','Direction','Type','Function');
fprintf('%-8s %-12s %-10s %s\n','FCCH','Downlink','Broadcast','Frequency correction')
    ↪ ;
fprintf('%-8s %-12s %-10s %s\n','SCH','Downlink','Broadcast','Frame synchronisation,
    ↪ BSIC');
fprintf('%-8s %-12s %-10s %s\n','BCCH','Downlink','Broadcast','Cell and network
    ↪ parameters');
fprintf('%-8s %-12s %-10s %s\n','PCH','Downlink','Common','Paging of mobile stations'
    ↪ );
fprintf('%-8s %-12s %-10s %s\n','AGCH','Downlink','Common','Assignment of a dedicated
    ↪  channel');
fprintf('%-8s %-12s %-10s %s\n','RACH','Uplink','Common','Random access request');
fprintf('%-8s %-12s %-10s %s\n','SDCCH','Both','Dedicated','Call setup signalling');
fprintf('%-8s %-12s %-10s %s\n','SACCH','Both','Dedicated','Power control, timing
    ↪ advance');
fprintf('%-8s %-12s %-10s %s\n','TCH','Both','Traffic','Encoded speech or data');

%% Figure 1 : burst structure
figure;
seg = [tail1 data1 steal1 tsc steal2 data2 tail2 Guard];
lab = {'T','Data','S','Training','S','Data','T','Guard'};
col = [0.35 0.35 0.35;0.33 0.55 0.78;0.85 0.33 0.31;0.47 0.72 0.42;...
       0.85 0.33 0.31;0.33 0.55 0.78;0.35 0.35 0.35;0.85 0.85 0.85];
```

```
x = 0;
hold on;
for k = 1:numel(seg)
    rectangle('Position',[x 0 seg(k) 1],'FaceColor',col(k,:));
    if seg(k) > 3
        text(x+seg(k)/2,0.5,lab{k},'HorizontalAlignment','center','Color','w');
    end
    x = x + seg(k);
end
xlim([0 x]); ylim([0 1]); set(gca,'YTick',[]);
xlabel('Bit periods'); title('GSM Normal Burst, 156.25 Bit Periods');

%% Figure 2 : slot occupancy over one traffic multiframe
occ = zeros(Nslot,26);
for f = 1:26
    for s = 1:Nslot
        if strcmp(tmap{f},'T'), occ(s,f) = 1;
        elseif strcmp(tmap{f},'S'), occ(s,f) = 2;
        else, occ(s,f) = 0; end
    end
end
figure;
imagesc(1:26,1:Nslot,occ); set(gca,'YDir','normal');
colormap([0.9 0.9 0.9;0.33 0.55 0.78;0.95 0.75 0.30]);
xlabel('Frame number in the 26-frame multiframe'); ylabel('Timeslot');
title('Traffic Multiframe Occupancy (grey idle, blue TCH, orange SACCH)');

%% Figure 3 : timing hierarchy on a logarithmic scale
figure;
names = {'Bit','Timeslot','Frame','Multiframe 26','Multiframe 51','Superframe','
    ↪ Hyperframe'};
vals  = [Tbit Tburst Tframe Tmulti26 Tmulti51 Tsuper Thyper];
barh(log10(vals)); set(gca,'YTickLabel',names);
grid on; xlabel('log_{10} ( duration in seconds )');
title('GSM Timing Hierarchy');
```

Output

```
GSM normal burst composition
  Tail bits                : 3 + 3
  Encrypted data bits      : 57 + 57
  Stealing flags           : 1 + 1
  Training sequence        : 26
  Total                    : 148 bits
  Guard period             : 8.25 bit periods

GSM timing hierarchy
  Bit period               : 3.6923 us
  Timeslot (burst) duration : 576.9231 us
  TDMA frame (8 slots)     : 4.6154 ms
  Traffic multiframe (26)  : 120.00 ms
  Control multiframe (51)  : 235.38 ms
  Superframe (51x26)       : 6.120 s
  Hyperframe (2048 super)  : 12533.76 s

Traffic channel gross rate  : 24.70 kbit/s
Traffic channel useful rate : 22.80 kbit/s

26-frame traffic multiframe (T traffic, S SACCH, I idle)
  T T T T T T T T T T T S T T T T T T T T T T T I

51-frame control multiframe, downlink timeslot 0, non-combined
  F FCCH   S SCH   B BCCH   C CCCH (PCH and AGCH)   I idle
  F S B B B B C C C C F S C C C C C C C C F S C C C C C C C C F S C C C C C C C C F S C C C C C C C C I
  F at frames : 0 10 20 30 40
  S at frames : 1 11 21 31 41
  B at frames : 2 3 4 5
  C at frames : 6 7 8 9 12 13 14 15 16 17 18 19 22 23 24 25 26 27 28 29 32 33 34 35 36 37 38 39 42 43 44 45 46 47 48 49
  I at frames : 50
  CCCH occupies 36 of the 51 frames

Channel  Direction    Type       Function
FCCH     Downlink     Broadcast  Frequency correction
SCH      Downlink     Broadcast  Frame synchronisation, BSIC
BCCH     Downlink     Broadcast  Cell and network parameters
PCH      Downlink     Common     Paging of mobile stations
AGCH     Downlink     Common     Assignment of a dedicated channel
RACH     Uplink       Common     Random access request
SDCCH    Both         Dedicated  Call setup signalling
SACCH    Both         Dedicated  Power control, timing advance
TCH      Both         Traffic    Encoded speech or data
```

Command Window Output

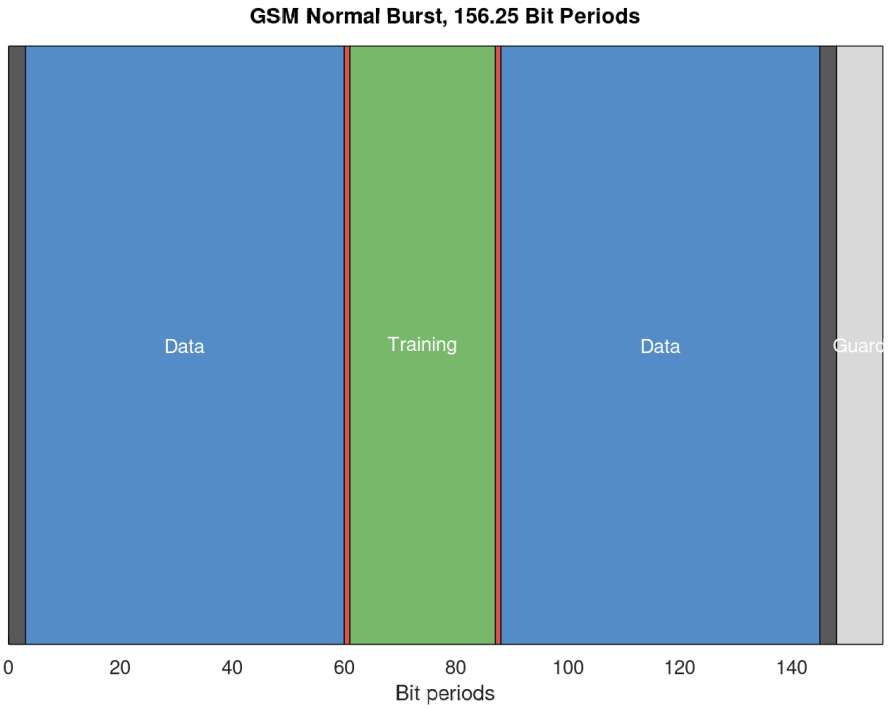


Figure 7.1

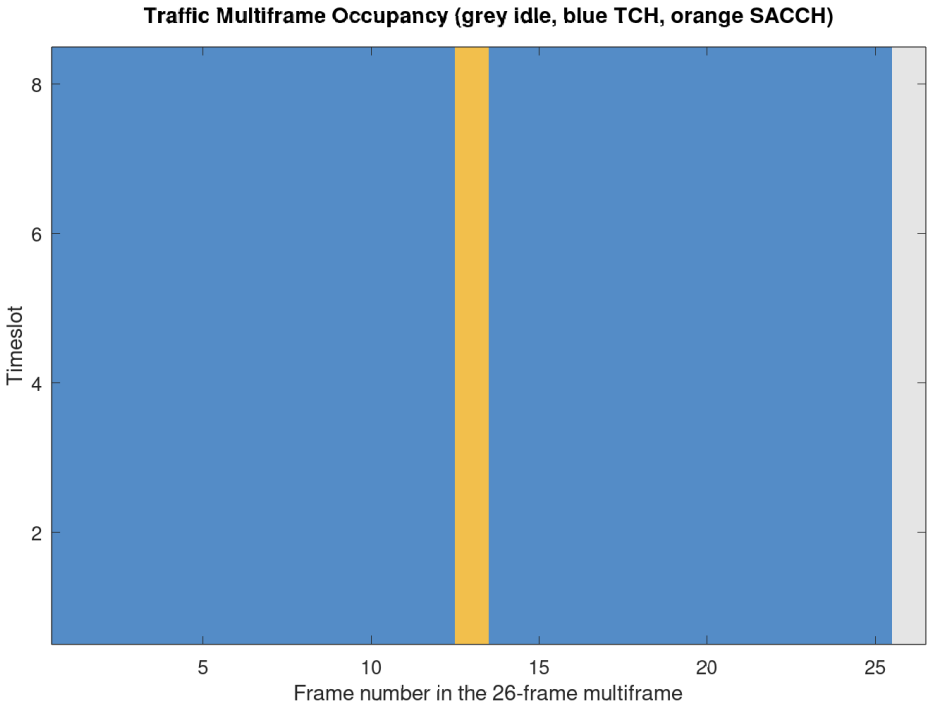


Figure 7.2

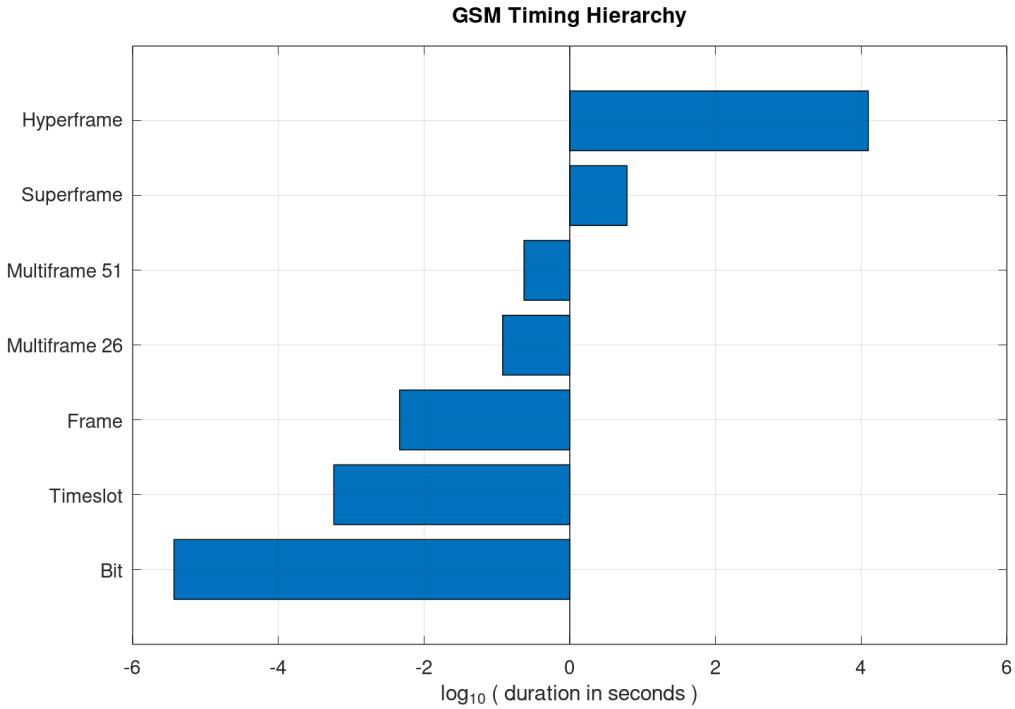


Figure 7.3

Experiment 8

GMSK Modulation and Spectral Analysis

Aim

To implement Gaussian minimum shift keying, examine the effect of the bandwidth time product on the transmitted spectrum, and verify the constant envelope property.

Check value (lab manual): *At $BT = 0.3$ the bandwidth containing 99 per cent of the power is approximately 250 kHz, and the envelope variation is of the order of* 10^{-16}.

Code

```
% Experiment 8 : GMSK Modulation and Spectral Analysis
clc; clear; close all;
rand('state',8);

%% Parameters
Rb   = 270833.33;     % GSM gross bit rate (bit/s)
sps  = 16;            % samples per bit
BT   = 0.3;           % Gaussian bandwidth-time product
span = 4;             % Gaussian pulse span (bit periods)
fs   = sps*Rb;
Tb   = 1/Rb;

fprintf('Bit rate          = %.2f kbit/s\n',Rb/1000);
fprintf('Bit period        = %.4f us\n',Tb*1e6);
fprintf('Sampling rate     = %.4f MHz\n',fs/1e6);
fprintf('BT product        = %.2f\n\n',BT);

%% Gaussian frequency pulse
t     = (-span:1/sps:span)';
sigma = sqrt(log(2))/(2*pi*BT);
gauss = exp(-t.^2/(2*sigma^2))/(sqrt(2*pi)*sigma);
rect  = ones(sps,1)/sps;
gp    = conv(gauss,rect,'same');
gp    = gp/sum(gp);                    % unit area, each bit adds pi/2

fprintf('Gaussian pulse: sigma = %.4f bit periods, area = %.6f\n\n',sigma,sum(gp));

%% GSM normal burst
tsc   = [0 0 1 0 0 1 0 1 1 1 0 0 0 0 1 0 0 0 1 0 0 1 0 1 1 1];
burst = [0 0 0 randi([0 1],1,57) 0 tsc 0 randi([0 1],1,57) 0 0 0];
fprintf('Normal burst length = %d bits\n',numel(burst));
fprintf('Burst duration      = %.4f ms\n',(numel(burst)+8.25)*Tb*1e3);
fprintf('TDMA frame duration = %.4f ms\n\n',8*(numel(burst)+8.25)*Tb*1e3);

%% Modulate the burst
a  = 2*burst - 1;
up = zeros(1,numel(a)*sps); up(1:sps:end) = a;
fq = conv(up,gp','same');
ph = (pi/2)*cumsum(fq);
s  = exp(1j*ph);

fprintf('Envelope variation max|s| - min|s| = %.3e\n',max(abs(s))-min(abs(s)));
fprintf('GMSK is a constant envelope modulation.\n\n');
```

```

%% Spectra for several BT products
Nb   = 4000;
bits = randi([0 1],1,Nb);
BTs  = [0.2 0.3 0.5 1e3];
lab  = {'BT = 0.2','BT = 0.3 (GSM)','BT = 0.5','MSK, no filter'};
nfft = 2048;
P    = cell(1,4);
fprintf('%-18s %14s %12s\n','Filter','99 % BW (kHz)','BW / Rb');
for k = 1:4
    sg = sqrt(log(2))/(2*pi*BTs(k));
    gg = exp(-t.^2/(2*sg^2))/(sqrt(2*pi)*sg);
    gk = conv(gg,rect,'same'); gk = gk/sum(gk);
    ak = 2*bits - 1;
    uk = zeros(1,numel(ak)*sps); uk(1:sps:end) = ak;
    sk = exp(1j*(pi/2)*cumsum(conv(uk,gk','same')));

    w = 0.5 - 0.5*cos(2*pi*(0:nfft-1)'/(nfft-1));
    nseg = floor(numel(sk)/nfft); Pk = zeros(nfft,1);
    for m = 1:nseg
        Pk = Pk + abs(fft(sk((m-1)*nfft+1:m*nfft)'.*w)).^2;
    end
    P{k} = fftshift(Pk/nseg);
    fax  = (-nfft/2:nfft/2-1)'*fs/nfft;

    tot = sum(P{k}); ctr = nfft/2+1; acc = P{k}(ctr); j = 0;
    while acc < 0.99*tot && ctr-j-1 >= 1 && ctr+j+1 <= nfft
        j = j + 1; acc = acc + P{k}(ctr-j) + P{k}(ctr+j);
    end
    bw = 2*j*(fax(2)-fax(1));
    fprintf('%-18s %14.1f %12.2f\n',lab{k},bw/1000,bw/Rb);
end
fprintf('\nThe GSM channel spacing is 200 kHz. The occupied bandwidth exceeds\n');
fprintf('this because channel spacing is set by the tolerable adjacent\n');
fprintf('channel power, not by the occupied bandwidth.\n');

%% Figure 1 : frequency pulse
figure;
plot((0:numel(gp)-1)/sps - span,gp*sps,'LineWidth',2); hold on;
plot([-0.5 -0.5 0.5 0.5],[0 1 1 0],'k--','LineWidth',1.5);
grid on; xlabel('Time (bit periods)'); ylabel('Normalised Amplitude');
legend('Gaussian pulse, BT = 0.3','Rectangular pulse, MSK');
title('Frequency Pulse Shapes');

%% Figure 2 : complex envelope
tt = (0:numel(s)-1)/fs;
figure;
plot(tt*1e6,real(s),'LineWidth',1.2); hold on;
plot(tt*1e6,imag(s),'LineWidth',1.2);
plot(tt*1e6,abs(s),'k','LineWidth',1.8);
grid on; axis([0 60 -1.3 1.3]);
xlabel('Time (\mus)'); ylabel('Amplitude');
legend('In-phase','Quadrature','Envelope','Location','southeast');
title('GMSK Complex Envelope');

%% Figure 3 : spectra
figure; hold on;
for k = 1:4
```

```
    plot(fax/1000,10*log10(P{k}/max(P{k})),'LineWidth',1.6);
end
plot([100 100],[-90 5],'k--',[-100 -100],[-90 5],'k--','LineWidth',1.4);
grid on; axis([-400 400 -90 5]);
xlabel('Frequency Offset (kHz)'); ylabel('Normalised PSD (dB)');
legend([lab,{'200 kHz channel edge'}]);
title('Effect of the BT Product on the Transmitted Spectrum');

%% Figure 4 : phase trajectory
figure;
plot((0:numel(s)-1)/sps,unwrap(angle(s))/pi,'LineWidth',1.6);
grid on; xlabel('Time (bit periods)'); ylabel('Phase / \pi');
title('Phase Trajectory : Each Bit Advances the Phase by \pm\pi/2');
```

Output

```
Bit rate          = 270.83 kbit/s
Bit period        = 3.6923 us
Sampling rate     = 4.3333 MHz
BT product        = 0.30

Gaussian pulse: sigma = 0.4417 bit periods, area = 1.000000

Normal burst length = 148 bits
Burst duration      = 0.5769 ms
TDMA frame duration = 4.6154 ms

Envelope variation max|s| - min|s| = 1.110e-16
GMSK is a constant envelope modulation.

Filter              99 % BW (kHz)      BW / Rb
BT = 0.2                   211.6         0.78
BT = 0.3 (GSM)             249.7         0.92
BT = 0.5                   279.3         1.03
MSK, no filter             317.4         1.17

The GSM channel spacing is 200 kHz. The occupied bandwidth exceeds
this because channel spacing is set by the tolerable adjacent
channel power, not by the occupied bandwidth.
```

Command Window Output

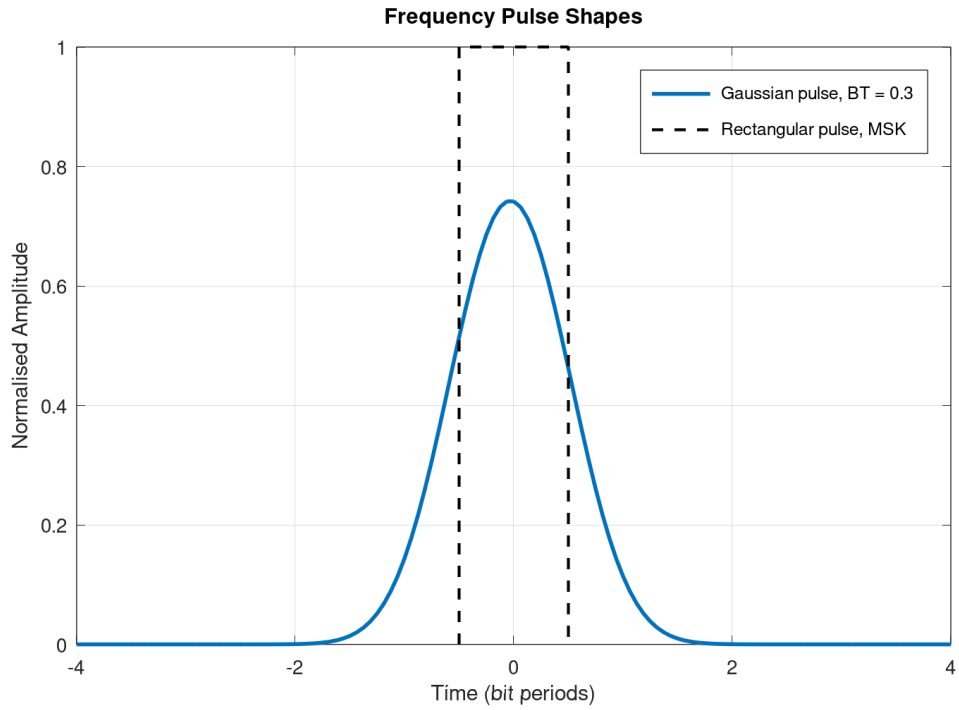


Figure 8.1

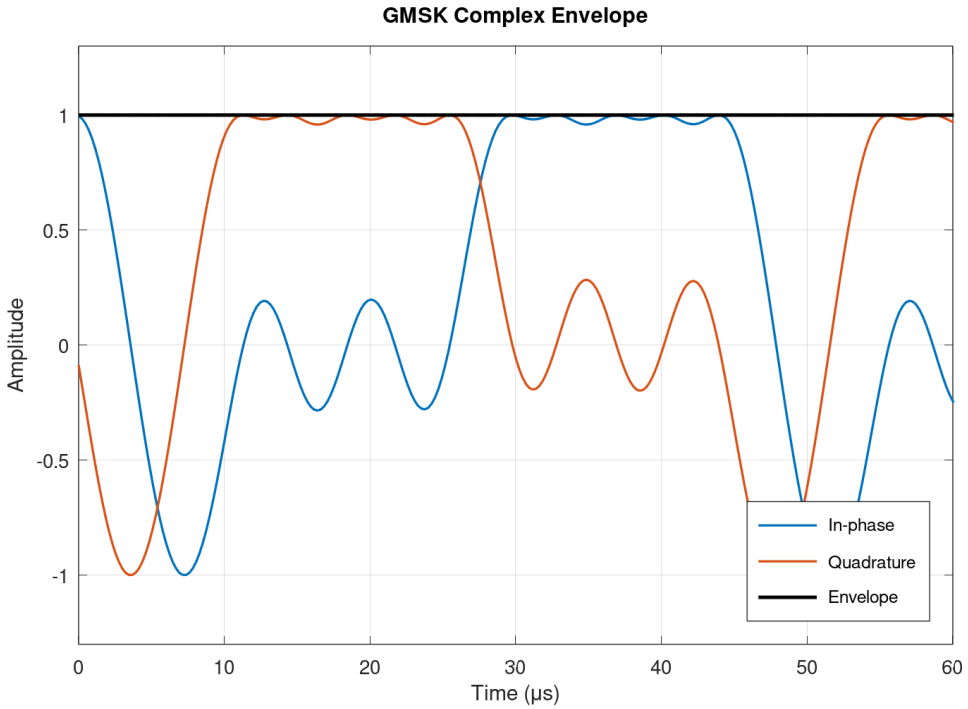


Figure 8.2

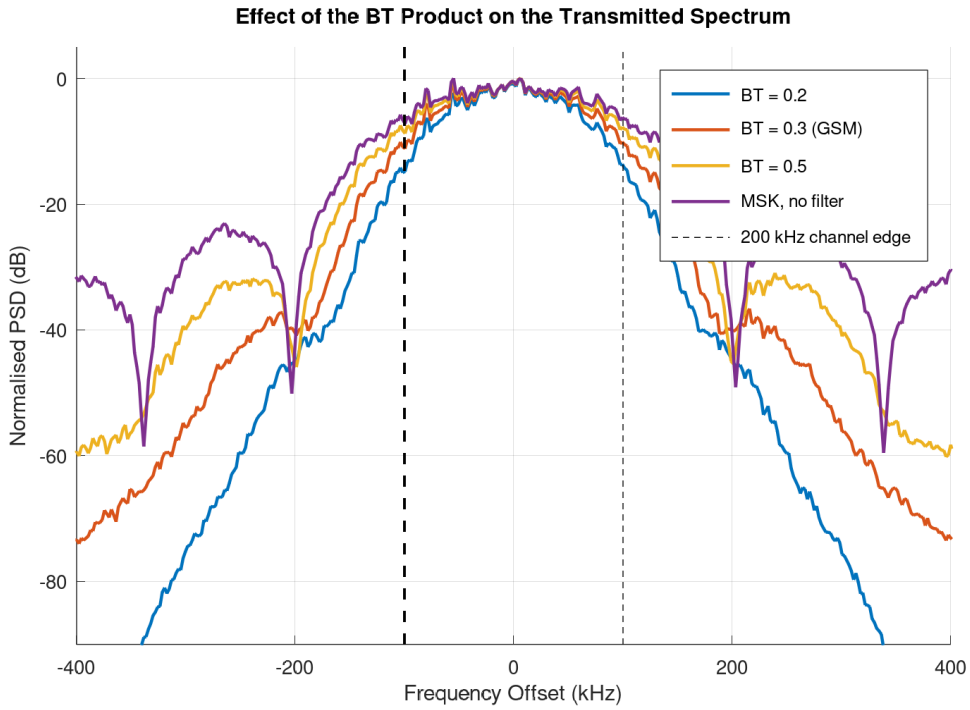


Figure 8.3

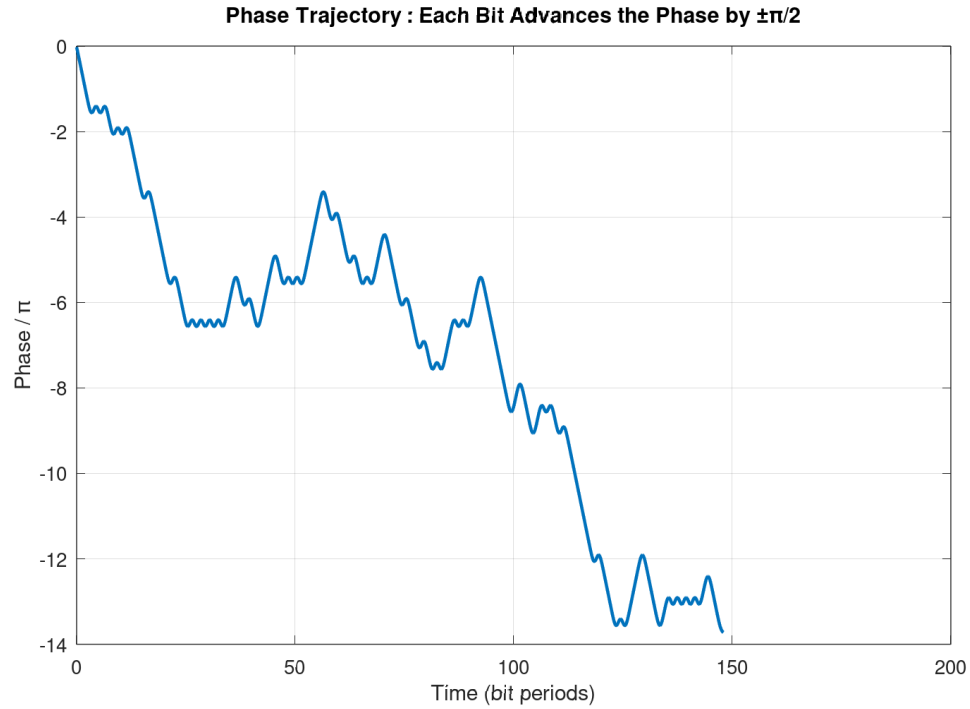


Figure 8.4

Experiment 9

OFDM Transceiver — Orthogonality, Cyclic Prefix and Peak to Average Power Ratio

Aim

To implement an OFDM transmitter and receiver, verify subcarrier orthogonality, determine the cyclic prefix length required by a given channel, and measure the peak to average power ratio.

Check value (lab manual): *For a channel of five taps the delay spread is four samples, and the error vector magnitude stops improving once the cyclic prefix reaches four samples.*

Code

```
% Experiment 9 : OFDM Transceiver - Orthogonality, Cyclic Prefix and PAPR
clc; clear; close all;
rand('state',9); randn('state',9);

%% Parameters
Nfft  = 64;             % FFT size
Ncp   = 16;             % cyclic prefix length
Nused = 52;             % used subcarriers
Nsym  = 2000;           % OFDM symbols
BW    = 20e6;           % signal bandwidth (Hz)
Mmod  = 4;              % QPSK

df  = BW/Nfft;
Tu  = 1/df;
Tcp = Ncp/Nfft*Tu;
Ts  = Tu + Tcp;

fprintf('FFT size                   = %d\n',Nfft);
fprintf('Used subcarriers           = %d\n',Nused);
fprintf('Subcarrier spacing         = %.4f kHz\n',df/1000);
fprintf('Useful symbol duration     = %.4f us\n',Tu*1e6);
fprintf('Cyclic prefix duration     = %.4f us\n',Tcp*1e6);
fprintf('Total symbol duration      = %.4f us\n',Ts*1e6);
fprintf('Cyclic prefix overhead     = %.2f %%\n',100*Ncp/(Nfft+Ncp));
fprintf('Symbol efficiency          = %.2f %%\n\n',100*Nfft/(Nfft+Ncp));

used = [Nfft/2-Nused/2+1 : Nfft/2, Nfft/2+2 : Nfft/2+1+Nused/2];

%% QPSK symbol generation
bits = randi([0 1],2,Nused*Nsym);
X    = ((1-2*bits(1,:)) + 1j*(1-2*bits(2,:)))/sqrt(2);
X    = reshape(X,Nused,Nsym);

%% OFDM modulation
grid_tx = zeros(Nfft,Nsym);
grid_tx(used,:) = X;
x  = ifft(ifftshift(grid_tx,1),Nfft)*sqrt(Nfft);
xc = [x(end-Ncp+1:end,:); x];
s  = xc(:);

%% Peak to average power ratio
papr = zeros(1,Nsym);
```

```
for k = 1:Nsym
    b = xc(:,k);
    papr(k) = max(abs(b).^2)/mean(abs(b).^2);
end
papr_dB = 10*log10(papr);
fprintf('PAPR mean                        = %.2f dB\n',mean(papr_dB));
fprintf('PAPR exceeded by 1 %% of symbols = %.2f dB\n',prctile_local(papr_dB,99));
fprintf('Theoretical limit 10log10(N) = %.2f dB\n\n',10*log10(Nfft));

%% Effect of cyclic prefix length on the error vector magnitude
% The metric is EVM rather than bit error rate. At the signal to noise
% ratio used here QPSK is robust enough that intersymbol interference from
% a short cyclic prefix degrades the constellation without yet producing
% bit errors, so EVM resolves the effect and BER does not.
h   = [1 0 0.6 0 0.3];                    % 5-tap multipath channel
Lch = numel(h);
cpv = [0 1 2 3 4 8 16];
snr = 30;
H   = fftshift(fft(h,Nfft)).';
evm = zeros(size(cpv));
fprintf('%10s %14s %14s %12s\n','CP length','CP >= L-1 ?','EVM','EVM (dB)');
for m = 1:numel(cpv)
    cp = cpv(m);
    if cp == 0, xcp = x; else, xcp = [x(end-cp+1:end,:); x]; end
    ss = xcp(:);
    rr = filter(h,1,ss);
    n  = sqrt(10^(-snr/10)*mean(abs(rr).^2)/2)*(randn(size(rr))+1j*randn(size(rr)));
    rr = rr + n;
    Lt = Nfft + cp;
    rm = reshape(rr(1:Nsym*Lt),Lt,Nsym);
    rm = rm(cp+1:end,:);
    Y  = fftshift(fft(rm,Nfft),1)/sqrt(Nfft);
    Ye = Y(used,:)./repmat(H(used),1,Nsym);
    evm(m) = sqrt(mean(abs(Ye(:)-X(:)).^2)/mean(abs(X(:)).^2));
    fprintf('%10d %14s %14.4f %12.2f\n',cp,...
            ternary_local(cp>=Lch-1,'yes','no'),evm(m),20*log10(evm(m)));
end
fprintf('\nThe channel delay spread is %d samples. Once the cyclic prefix\n',Lch-1);
fprintf('reaches that length the EVM stops improving, because orthogonality\n');
fprintf('has been restored and only thermal noise remains.\n\n');

%% Orthogonality check
fprintf('Subcarrier orthogonality check\n');
k1 = 10; k2 = 17;
v1 = exp(1j*2*pi*k1*(0:Nfft-1)/Nfft);
v2 = exp(1j*2*pi*k2*(0:Nfft-1)/Nfft);
fprintf('  Inner product of subcarriers %d and %d = %.3e\n',k1,k2,abs(v1*v2')/Nfft);
fprintf('  Inner product of subcarrier %d with itself = %.4f\n\n',k1,abs(v1*v1')/Nfft
    ↪ );

%% Figure 1 : PAPR complementary cumulative distribution
figure;
lv = 0:0.1:12;
cc = zeros(size(lv));
for k = 1:numel(lv), cc(k) = mean(papr_dB > lv(k)); end
semilogy(lv,max(cc,1e-4),'LineWidth',2); grid on;
axis([0 12 1e-4 1]);
xlabel('PAPR_0 (dB)'); ylabel('P ( PAPR > PAPR_0 )');
```

```
title(sprintf('PAPR Distribution, N = %d Subcarriers',Nfft));

%% Figure 2 : transmitted spectrum
nfft2 = 1024;
w = 0.5 - 0.5*cos(2*pi*(0:nfft2-1)'/(nfft2-1));
nseg = floor(numel(s)/nfft2); Ps = zeros(nfft2,1);
for m = 1:nseg
    Ps = Ps + abs(fft(s((m-1)*nfft2+1:m*nfft2).*w)).^2;
end
Ps = fftshift(Ps/nseg);
fa = (-nfft2/2:nfft2/2-1)'*BW/nfft2;
figure;
plot(fa/1e6,10*log10(Ps/max(Ps)),'LineWidth',1.2); grid on;
xlabel('Frequency (MHz)'); ylabel('Normalised PSD (dB)');
title('OFDM Transmitted Spectrum');

%% Figure 3 : effect of the cyclic prefix on the constellation
figure;
for idx = 1:2
    cpsel = [0 16]; cp = cpsel(idx);
    xcp = [x(end-max(cp,1)+1:end,:); x];
    if cp == 0, xcp = x; end
    ss = xcp(:); rr = filter(h,1,ss);
    n  = sqrt(10^(-snr/10)*mean(abs(rr).^2)/2)*(randn(size(rr))+1j*randn(size(rr)));
    rr = rr + n;
    Lt = Nfft + cp;
    rm = reshape(rr(1:Nsym*Lt),Lt,Nsym);
    rm = rm(cp+1:end,:);
    Y  = fftshift(fft(rm,Nfft),1)/sqrt(Nfft);
    Ye = Y(used,:)./repmat(H(used),1,Nsym);
    subplot(1,2,idx);
    plot(real(Ye(:)),imag(Ye(:)),'.','MarkerSize',1);
    axis square; axis([-2 2 -2 2]); grid on;
    xlabel('In-phase'); ylabel('Quadrature');
    title(sprintf('Cyclic prefix = %d samples',cp));
end

%% Figure 4 : EVM against cyclic prefix length
figure;
plot(cpv,20*log10(evm),'o-','LineWidth',2); hold on;
plot([Lch-1 Lch-1],[min(20*log10(evm))-2 max(20*log10(evm))+2],'k--','LineWidth',1.5)
    ↪ ;
grid on; xlabel('Cyclic prefix length (samples)'); ylabel('EVM (dB)');
legend('Measured EVM','Channel delay spread L-1','Location','northeast');
title('Cyclic Prefix Length against Error Vector Magnitude');

%% Figure 5 : time domain OFDM symbol with cyclic prefix
figure;
plot(abs(xc(:,1)),'LineWidth',1.4); hold on;
plot(1:Ncp,abs(xc(1:Ncp,1)),'r','LineWidth',2.2);
plot(Nfft+1:Nfft+Ncp,abs(xc(Nfft+1:Nfft+Ncp,1)),'r--','LineWidth',2.2);
grid on; xlabel('Sample'); ylabel('Magnitude');
legend('OFDM symbol','Cyclic prefix','Copied from the tail');
title('Cyclic Prefix Insertion');

function y = prctile_local(x,p)
x = sort(x(:));
n = numel(x);
```

```
y = interp1(100*((1:n)'-0.5)/n, x, min(max(p,100*0.5/n),100*(n-0.5)/n));
end

function out = ternary_local(cond,a,b)
if cond, out = a; else, out = b; end
end
```

Output

```
FFT size                    = 64
Used subcarriers            = 52
Subcarrier spacing          = 312.5000 kHz
Useful symbol duration      = 3.2000 us
Cyclic prefix duration      = 0.8000 us
Total symbol duration       = 4.0000 us
Cyclic prefix overhead      = 20.00 %
Symbol efficiency           = 80.00 %

PAPR mean                       = 6.68 dB
PAPR exceeded by 1 % of symbols = 9.33 dB
Theoretical limit 10log10(N) = 18.06 dB

 CP length    CP >= L-1 ?            EVM     EVM (dB)
         0             no         0.1660       -15.60
         1             no         0.1337       -17.48
         2             no         0.0951       -20.44
         3             no         0.0721       -22.85
         4            yes         0.0383       -28.34
         8            yes         0.0381       -28.38
        16            yes         0.0383       -28.34

The channel delay spread is 4 samples. Once the cyclic prefix
reaches that length the EVM stops improving, because orthogonality
has been restored and only thermal noise remains.

Subcarrier orthogonality check
  Inner product of subcarriers 10 and 17 = 1.638e-16
  Inner product of subcarrier 10 with itself = 1.0000
```

Command Window Output

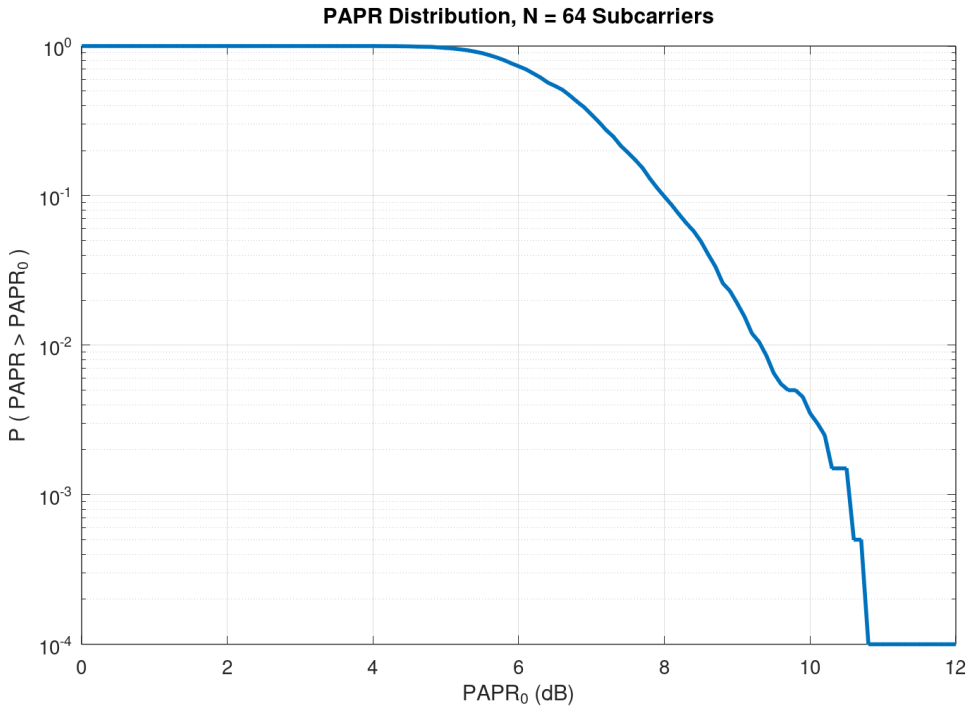


Figure 9.1

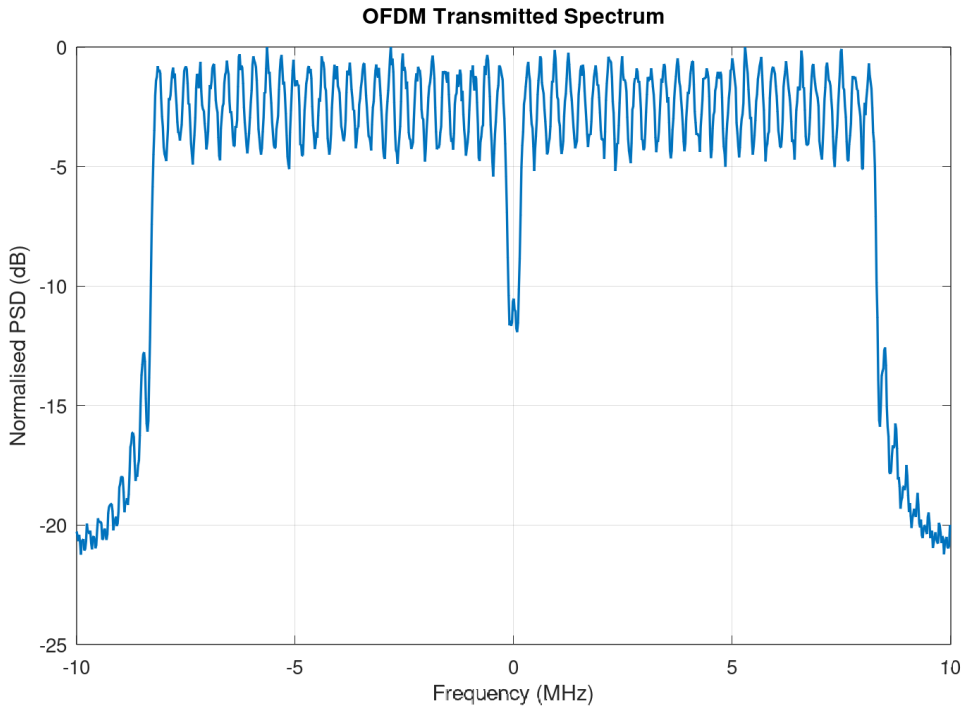


Figure 9.2

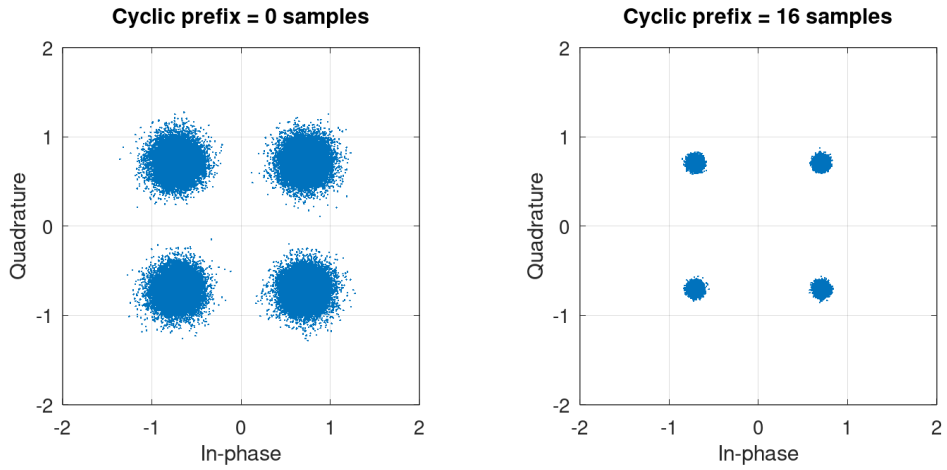


Figure 9.3

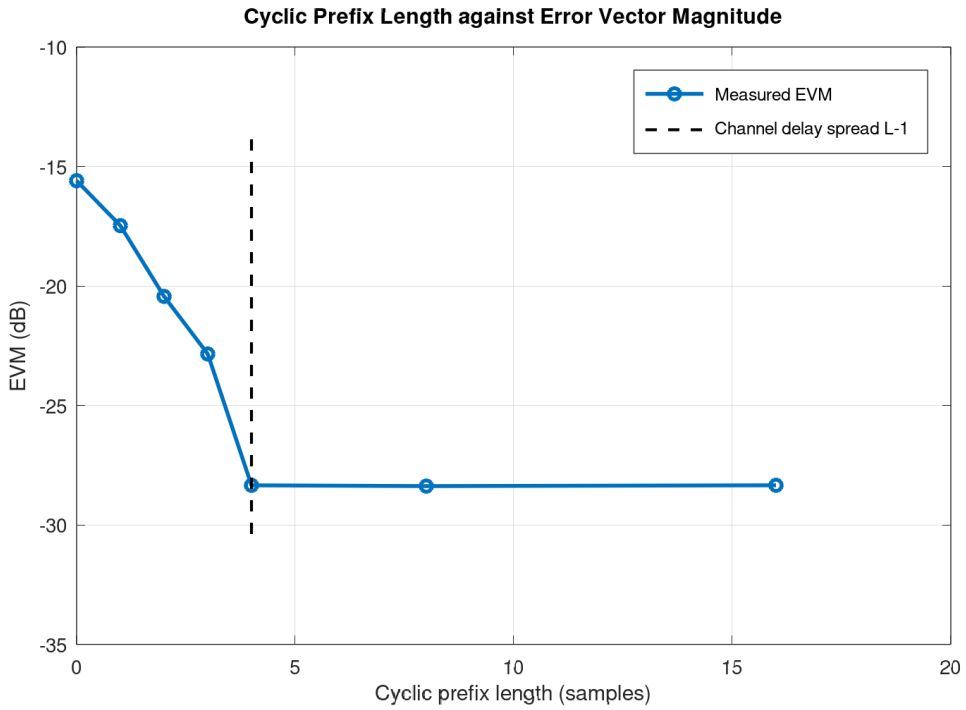


Figure 9.4

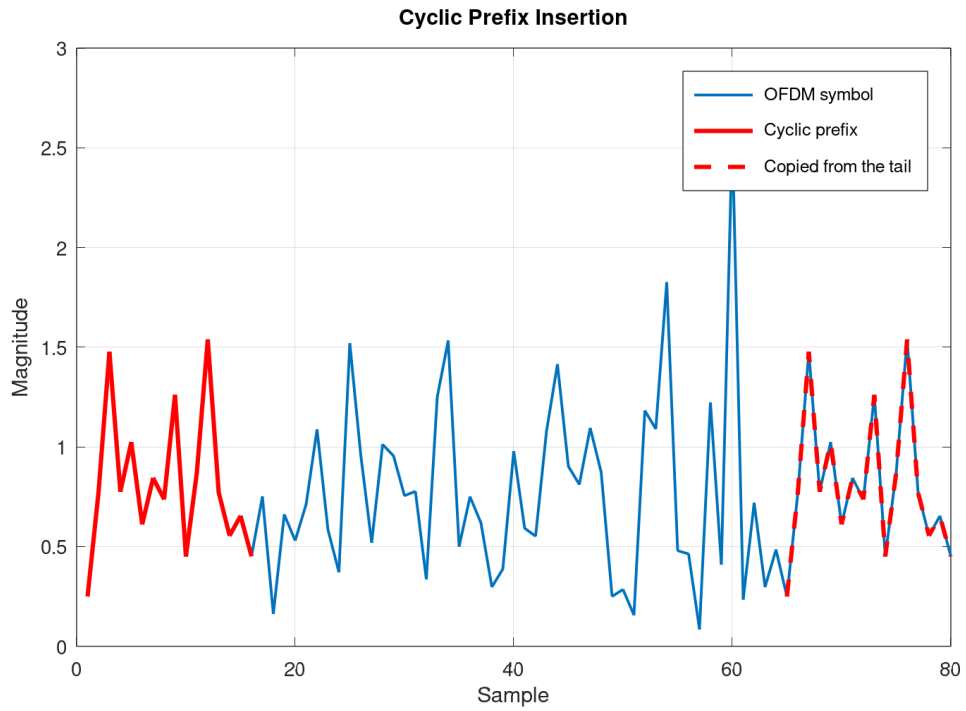


Figure 9.5

Experiment 10

MIMO — SISO, Alamouti Transmit Diversity and Spatial Multiplexing Capacity

Aim

To compare a single antenna link with Alamouti transmit diversity and with receive combining, and to compute the ergodic capacity of one by one, one by two, two by one and two by two Rayleigh channels.

Check value (lab manual): *The measured diversity order is approximately 1 for the single antenna link and approximately 2 for both Alamouti and two branch receive combining. The two by two capacity slope is close to twice the single antenna slope.*

Code

```
% Experiment 10 : MIMO - SISO, Alamouti Transmit Diversity and Spatial Multiplexing
clc; clear; close all;
randn('state',10);

%% Parameters
snr_dB = 0:2:24;
Nsym   = 1e6;
Ncap   = 2e4;

fprintf('Rayleigh flat fading, perfect channel knowledge at the receiver\n');
fprintf('Modulation : BPSK for the error rate study\n');
fprintf('Symbols per point = %g, Monte Carlo resolution = %.1e\n\n',Nsym,1/Nsym);

ber_siso = zeros(size(snr_dB));
ber_ala  = zeros(size(snr_dB));
ber_mrc  = zeros(size(snr_dB));

for k = 1:numel(snr_dB)
    snr = 10^(snr_dB(k)/10);
    b   = randi([0 1],1,Nsym);
    s   = 1 - 2*b;

    % 1 x 1 SISO
    h = (randn(1,Nsym) + 1j*randn(1,Nsym))/sqrt(2);
    n = (randn(1,Nsym) + 1j*randn(1,Nsym))/sqrt(2*snr);
    y = h.*s + n;
    ber_siso(k) = mean(b ~= (real(conj(h).*y) < 0));

    % 2 x 1 Alamouti, transmit power split between the two antennas
    Np = Nsym/2;
    s1 = s(1:2:end); s2 = s(2:2:end);
    h1 = (randn(1,Np) + 1j*randn(1,Np))/sqrt(2);
    h2 = (randn(1,Np) + 1j*randn(1,Np))/sqrt(2);
    n1 = (randn(1,Np) + 1j*randn(1,Np))/sqrt(2*snr);
    n2 = (randn(1,Np) + 1j*randn(1,Np))/sqrt(2*snr);
    y1 = (h1.*s1 + h2.*s2)/sqrt(2) + n1;
    y2 = (-h1.*conj(s2) + h2.*conj(s1))/sqrt(2) + n2;
    z1 = conj(h1).*y1 + h2.*conj(y2);
    z2 = conj(h2).*y1 - h1.*conj(y2);
```

```
    bh = zeros(1,Nsym);
    bh(1:2:end) = real(z1) < 0;
    bh(2:2:end) = real(z2) < 0;
    ber_ala(k) = mean(b ~= bh);

    % 1 x 2 receive diversity with maximal ratio combining
    ha = (randn(1,Nsym) + 1j*randn(1,Nsym))/sqrt(2);
    hb = (randn(1,Nsym) + 1j*randn(1,Nsym))/sqrt(2);
    na = (randn(1,Nsym) + 1j*randn(1,Nsym))/sqrt(2*snr);
    nb = (randn(1,Nsym) + 1j*randn(1,Nsym))/sqrt(2*snr);
    z  = conj(ha).*(ha.*s + na) + conj(hb).*(hb.*s + nb);
    ber_mrc(k) = mean(b ~= (real(z) < 0));
end

fprintf('%10s %14s %14s %14s\n','SNR (dB)','SISO','Alamouti 2x1','MRC 1x2');
for k = 1:numel(snr_dB)
    fprintf('%10d %14.5f %14.5f %14.5f\n',snr_dB(k),ber_siso(k),ber_ala(k),ber_mrc(k)
    ↪ );
end

%% Diversity order from the slope at high signal to noise ratio
% The slope is measured between 10 and 16 dB, where all three curves lie
% well above the Monte Carlo resolution of 1/Nsym.
i1 = find(snr_dB == 10); i2 = find(snr_dB == 16);
d_siso = (log10(ber_siso(i1))-log10(ber_siso(i2)))/((snr_dB(i2)-snr_dB(i1))/10);
d_ala  = (log10(ber_ala(i1)) -log10(ber_ala(i2)) )/((snr_dB(i2)-snr_dB(i1))/10);
d_mrc  = (log10(ber_mrc(i1)) -log10(ber_mrc(i2)) )/((snr_dB(i2)-snr_dB(i1))/10);
fprintf('\nDiversity order from the slope between 10 and 16 dB\n');
fprintf('  SISO          : %.2f  (expected 1)\n',d_siso);
fprintf('  Alamouti 2x1  : %.2f  (expected 2)\n',d_ala);
fprintf('  MRC 1x2       : %.2f  (expected 2)\n',d_mrc);
fprintf('\nAlamouti achieves the same diversity order as two-branch receive\n');
fprintf('combining but is 3 dB worse, because the transmit power is divided\n');
fprintf('between the two transmit antennas.\n\n');

%% Ergodic capacity
fprintf('%10s %12s %12s %12s %12s\n','SNR (dB)','1x1','1x2','2x1','2x2');
cap = zeros(4,numel(snr_dB));
for k = 1:numel(snr_dB)
    snr = 10^(snr_dB(k)/10);
    c11 = 0; c12 = 0; c21 = 0; c22 = 0;
    for m = 1:Ncap
        H11 = (randn + 1j*randn)/sqrt(2);
        c11 = c11 + log2(1 + snr*abs(H11)^2);
        H12 = (randn(2,1) + 1j*randn(2,1))/sqrt(2);
        c12 = c12 + log2(1 + snr*(H12'*H12));
        H21 = (randn(1,2) + 1j*randn(1,2))/sqrt(2);
        c21 = c21 + log2(1 + snr/2*(H21*H21'));
        H22 = (randn(2,2) + 1j*randn(2,2))/sqrt(2);
        c22 = c22 + real(log2(det(eye(2) + snr/2*(H22*H22'))));
    end
    cap(:,k) = [c11 c12 c21 c22]'/Ncap;
    fprintf('%10d %12.3f %12.3f %12.3f %12.3f\n',snr_dB(k),cap(1,k),cap(2,k),cap(3,k),
    ↪ cap(4,k));
end
fprintf('\nAt high signal to noise ratio the 2x2 capacity grows at twice the\n');
fprintf('rate of the 1x1 capacity, which is the spatial multiplexing gain.\n');
sl = (cap(:,end)-cap(:,end-1))/((snr_dB(end)-snr_dB(end-1))/3.0103);
```

```
fprintf('%10s %12s %12s %12s %12s\n','Slope','1x1','1x2','2x1','2x2');
fprintf('%10s %12.2f %12.2f %12.2f %12.2f\n','bit/s/Hz per 3 dB',sl(1),sl(2),sl(3),sl(4));

%% Figure 1 : bit error rate
figure;
semilogy(snr_dB,ber_siso,'o-','LineWidth',1.8); hold on;
semilogy(snr_dB,ber_ala,'s-','LineWidth',1.8);
semilogy(snr_dB,ber_mrc,'d-','LineWidth',1.8);
gr = 10.^(snr_dB/10);
semilogy(snr_dB,0.5*(1-sqrt(gr./(1+gr))),'k:','LineWidth',1.6);
semilogy(snr_dB,0.5*erfc(sqrt(gr)),'k--','LineWidth',1.4);
grid on; axis([0 24 1e-6 1]);
xlabel('SNR (dB)'); ylabel('Bit Error Rate');
legend('SISO 1x1','Alamouti 2x1','MRC 1x2','SISO theory','AWGN, no fading',...
       'Location','southwest');
title('Effect of Transmit and Receive Diversity');

%% Figure 2 : ergodic capacity
figure;
plot(snr_dB,cap(1,:),'o-','LineWidth',1.8); hold on;
plot(snr_dB,cap(2,:),'s-','LineWidth',1.8);
plot(snr_dB,cap(3,:),'d-','LineWidth',1.8);
plot(snr_dB,cap(4,:),'^-','LineWidth',1.8);
grid on; xlabel('SNR (dB)'); ylabel('Ergodic Capacity (bit/s/Hz)');
legend('1 x 1','1 x 2','2 x 1','2 x 2','Location','northwest');
title('Ergodic Capacity of Rayleigh MIMO Channels');
```

Output

```
Rayleigh flat fading, perfect channel knowledge at the receiver
Modulation : BPSK for the error rate study
Symbols per point = 1e+06, Monte Carlo resolution = 1.0e-06

  SNR (dB)           SISO   Alamouti 2x1        MRC 1x2
         0        0.14622        0.11560        0.05767
         2        0.10828        0.07452        0.03275
         4        0.07679        0.04414        0.01681
         6        0.05296        0.02411        0.00808
         8        0.03559        0.01179        0.00361
        10        0.02339        0.00567        0.00161
        12        0.01510        0.00247        0.00073
        14        0.00963        0.00101        0.00028
        16        0.00614        0.00040        0.00011
        18        0.00393        0.00016        0.00005
        20        0.00250        0.00007        0.00002
        22        0.00163        0.00003        0.00001
        24        0.00099        0.00001        0.00000

Diversity order from the slope between 10 and 16 dB
  SISO          : 0.97  (expected 1)
  Alamouti 2x1  : 1.92  (expected 2)
  MRC 1x2       : 1.92  (expected 2)

Alamouti achieves the same diversity order as two-branch receive
combining but is 3 dB worse, because the transmit power is divided
between the two transmit antennas.

  SNR (dB)          1x1          1x2          2x1          2x2
         0        0.865        1.440        0.921        1.682
         2        1.158        1.871        1.253        2.258
         4        1.516        2.355        1.649        2.916
         6        1.930        2.903        2.107        3.698
         8        2.396        3.444        2.607        4.591
        10        2.905        4.047        3.164        5.524
        12        3.453        4.681        3.758        6.578
        14        4.048        5.316        4.364        7.691
        16        4.632        5.970        5.001        8.874
        18        5.259        6.606        5.626       10.045
        20        5.875        7.270        6.282       11.274
        22        6.522        7.939        6.928       12.560
        24        7.178        8.590        7.601       13.848

At high signal to noise ratio the 2x2 capacity grows at twice the
rate of the 1x1 capacity, which is the spatial multiplexing gain.
     Slope          1x1          1x2          2x1          2x2
bit/s/Hz per 3 dB         0.99         0.98         1.01         1.94
```

Command Window Output

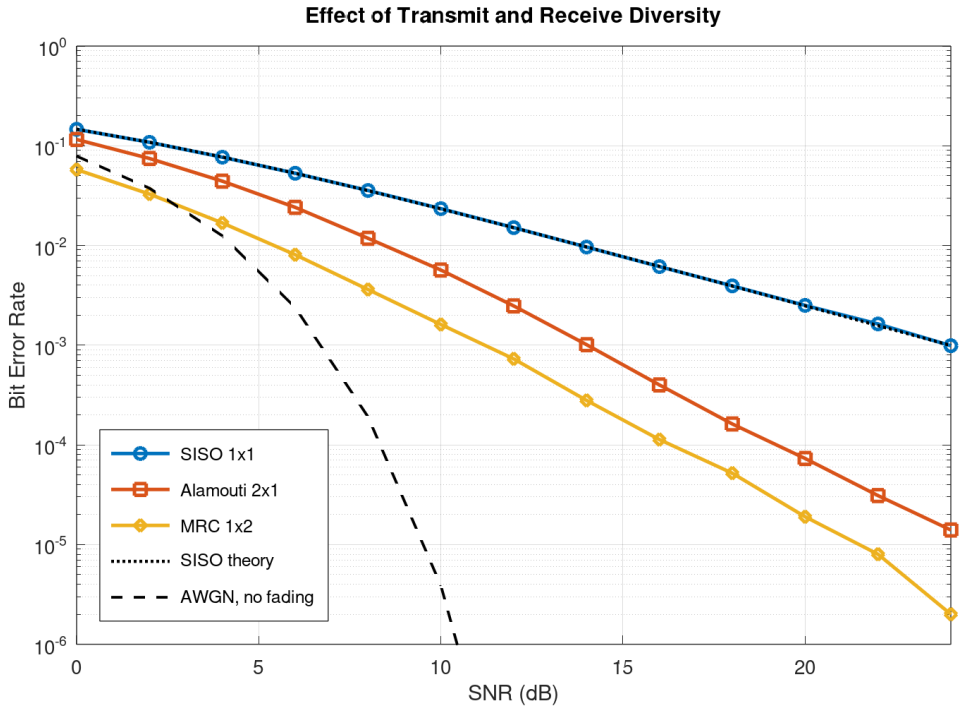


Figure 10.1

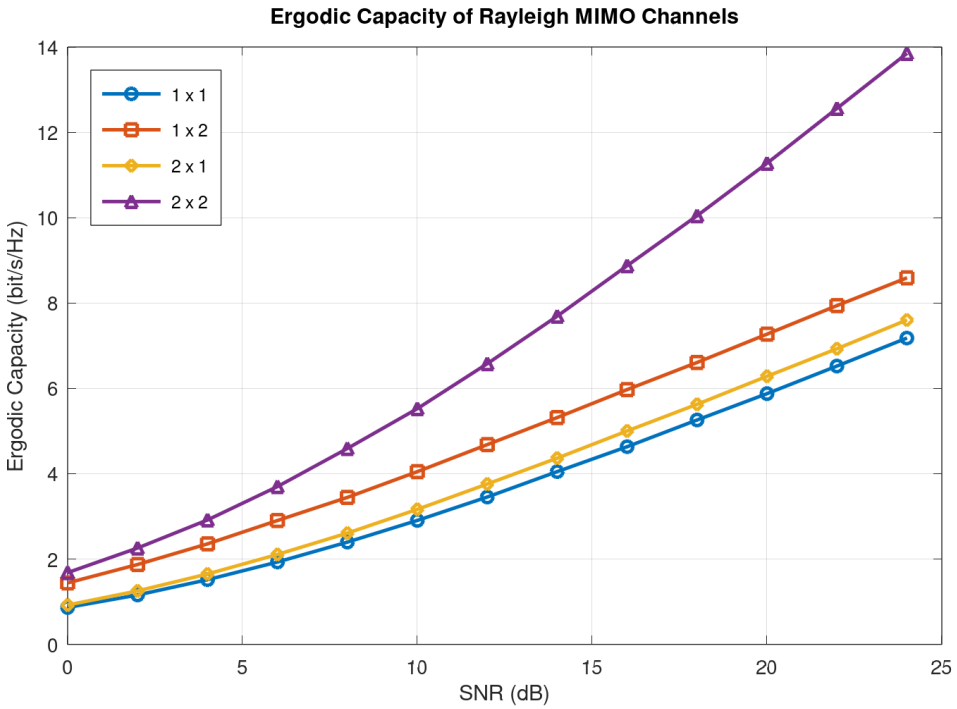

Ergodic Capacity of Rayleigh MIMO Channels
1 x 1
1 x 2
2 x 1
2 x 2
Ergodic Capacity (bit/s/Hz)
SNR (dB)
0
2
4
6
8
10
12
14
0
5
10
15
20
25

Figure 10.2

Experiment 11

LTE Resource Grid, Pilot Insertion and Channel Estimation

Aim

To construct the LTE downlink resource grid, insert reference signals, estimate the channel by least squares with interpolation, and measure the estimation error and the resulting error vector magnitude.

Check value (lab manual): *With a pilot spacing of six subcarriers at 15 kHz the pilots are 90 kHz apart, which is well inside the 400 kHz coherence bandwidth of a channel with 0.5 μs delay spread.*

Code

```
% Experiment 11 : LTE Resource Grid, Pilot Insertion and Channel Estimation
clc; clear; close all;
randn('state',11); rand('state',11);

%% Parameters
Nsc    = 12;           % subcarriers per resource block
Nsym   = 7;            % OFDM symbols per slot, normal cyclic prefix
Nrb    = 25;           % resource blocks (5 MHz LTE)
df     = 15e3;         % subcarrier spacing (Hz)
Nfft   = 512;
pilotK = 6;            % pilot spacing in frequency (subcarriers)
pilotL = [1 5];        % pilot positions in time (symbol index)
snr_dB = 0:3:30;
Nslot  = 200;

Ntot  = Nrb*Nsc;
BW    = Ntot*df;
Tslot = 0.5e-3;

fprintf('Subcarrier spacing          = %.1f kHz\n',df/1000);
fprintf('Resource blocks             = %d\n',Nrb);
fprintf('Occupied subcarriers        = %d\n',Ntot);
fprintf('Occupied bandwidth          = %.3f MHz\n',BW/1e6);
fprintf('Symbols per slot            = %d\n',Nsym);
fprintf('Slot duration               = %.2f ms\n',Tslot*1e3);
fprintf('Resource elements per slot = %d\n',Ntot*Nsym);
fprintf('Pilot spacing in frequency = %d subcarriers\n',pilotK);
fprintf('Pilot symbols per slot      = %d of %d\n',numel(pilotL),Nsym);
pilotFrac = (numel(pilotL)*ceil(Ntot/pilotK))/(Ntot*Nsym);
fprintf('Pilot overhead              = %.2f %%\n\n',100*pilotFrac);

%% Coherence bandwidth check
tauRMS = 0.5e-6;                        % rms delay spread (s)
Bc     = 1/(5*tauRMS);
fprintf('Assumed rms delay spread   = %.2f us\n',tauRMS*1e6);
fprintf('Coherence bandwidth        = %.1f kHz\n',Bc/1000);
fprintf('Pilot spacing in frequency = %.1f kHz\n',pilotK*df/1000);
if pilotK*df < Bc
    fprintf('Pilot spacing is inside the coherence bandwidth, so linear\n');
    fprintf('interpolation between pilots is valid.\n\n');
else
    fprintf('Pilot spacing exceeds the coherence bandwidth. Interpolation\n');
```

```
    fprintf('will fail and the estimate will be biased.\n\n');
end

%% Channel estimation performance
pidx = 1:pilotK:Ntot;
mse  = zeros(size(snr_dB));
evm  = zeros(size(snr_dB));
for k = 1:numel(snr_dB)
    snr = 10^(snr_dB(k)/10);
    e   = 0; ev = 0;
    for m = 1:Nslot
        hd = (randn(1,6)+1j*randn(1,6)).*exp(-(0:5)/3)/sqrt(2);
        H  = fft(hd,Nfft); H = H(1:Ntot);
        Hg = repmat(H.',1,Nsym);

        b  = randi([0 1],2,Ntot*Nsym);
        X  = reshape(((1-2*b(1,:))+1j*(1-2*b(2,:)))/sqrt(2),Ntot,Nsym);
        X(pidx,pilotL) = (1+1j)/sqrt(2);

        N  = (randn(Ntot,Nsym)+1j*randn(Ntot,Nsym))/sqrt(2*snr);
        Y  = Hg.*X + N;

        Hp   = Y(pidx,pilotL)./X(pidx,pilotL);
        Hp   = mean(Hp,2);
        He   = interp1(pidx,Hp,1:Ntot,'linear','extrap').';
        Hest = repmat(He,1,Nsym);

        e  = e + mean(abs(Hest(:)-Hg(:)).^2)/mean(abs(Hg(:)).^2);
        % MMSE equalisation. Zero forcing divides by the channel and so
        % amplifies noise without limit on subcarriers in a deep fade.
        Xe = Y.*conj(Hest)./(abs(Hest).^2 + 1/snr);
        dm = setdiff(1:Nsym,pilotL);
        ev = ev + mean(abs(Xe(:,dm)-X(:,dm)).^2,'all')/mean(abs(X(:,dm)).^2,'all');
    end
    mse(k) = e/Nslot; evm(k) = sqrt(ev/Nslot);
end

fprintf('%10s %16s %14s %12s\n','SNR (dB)','Estimation MSE','EVM','EVM (dB)');
for k = 1:numel(snr_dB)
    fprintf('%10d %16.5f %14.4f %12.2f\n',snr_dB(k),mse(k),evm(k),20*log10(evm(k)));
end
fprintf('\nThe estimation error falls with signal to noise ratio until it\n');
fprintf('reaches the floor set by interpolation between pilots.\n');
fprintf('MMSE equalisation is used, since zero forcing amplifies noise\n');
fprintf('without limit on subcarriers lying in a deep fade.\n\n');

%% Figure 1 : resource grid with pilots
figure;
grid_map = zeros(Nsc*2,Nsym);
grid_map(:) = 1;
for c = pilotL
    grid_map(1:pilotK:end,c) = 2;
end
imagesc(1:Nsym,1:Nsc*2,grid_map); set(gca,'YDir','normal');
colormap([0.80 0.86 0.92;0.90 0.55 0.20]);
set(gca,'XTick',1:Nsym,'YTick',1:2:Nsc*2);
xlabel('OFDM symbol index'); ylabel('Subcarrier index');
title('LTE Resource Grid : Two Resource Blocks with Reference Signals');
```

```
grid on;

%% Figure 2 : channel estimation error
figure;
semilogy(snr_dB,mse,'o-','LineWidth',1.8); hold on;
semilogy(snr_dB,10.^(-snr_dB/10),'k--','LineWidth',1.5);
grid on; xlabel('SNR (dB)'); ylabel('Normalised Mean Square Error');
legend('Least squares estimate with interpolation','1 / SNR reference');
title('Channel Estimation Error');

%% Figure 3 : error vector magnitude
figure;
plot(snr_dB,20*log10(evm),'s-','LineWidth',1.8); grid on;
xlabel('SNR (dB)'); ylabel('EVM (dB)');
title('Error Vector Magnitude after Equalisation');

%% Figure 4 : one channel realisation against its estimate
hd = (randn(1,6)+1j*randn(1,6)).*exp(-(0:5)/3)/sqrt(2);
H  = fft(hd,Nfft); H = H(1:Ntot).';
snr = 10^(20/10);
Xp = (1+1j)/sqrt(2)*ones(numel(pidx),1);
Yp = H(pidx).*Xp + (randn(numel(pidx),1)+1j*randn(numel(pidx),1))/sqrt(2*snr);
Hp = Yp./Xp;
He = interp1(pidx,Hp,1:Ntot,'linear','extrap').';
figure;
plot(20*log10(abs(H)),'LineWidth',1.6); hold on;
plot(20*log10(abs(He)),'--','LineWidth',1.6);
plot(pidx,20*log10(abs(Hp)),'o','MarkerSize',5);
grid on; xlabel('Subcarrier index'); ylabel('|H| (dB)');
legend('True channel','Interpolated estimate','Pilot measurements');
title('Frequency Selective Channel and its Estimate at 20 dB');
```

Output

```
Subcarrier spacing          = 15.0 kHz
Resource blocks             = 25
Occupied subcarriers        = 300
Occupied bandwidth          = 4.500 MHz
Symbols per slot            = 7
Slot duration               = 0.50 ms
Resource elements per slot = 2100
Pilot spacing in frequency = 6 subcarriers
Pilot symbols per slot      = 2 of 7
Pilot overhead              = 4.76 %
Assumed rms delay spread   = 0.50 us
Coherence bandwidth        = 400.0 kHz
Pilot spacing in frequency = 90.0 kHz
Pilot spacing is inside the coherence bandwidth, so linear
interpolation between pilots is valid.
  SNR (dB)   Estimation MSE            EVM     EVM (dB)
         0          0.26500         0.7391        -2.63
         3          0.13684         0.6254        -4.08
         6          0.07461         0.5388        -5.37
         9          0.03610         0.4347        -7.24
        12          0.01703         0.3391        -9.39
        15          0.00821         0.2598       -11.71
        18          0.00476         0.2093       -13.58
        21          0.00232         0.1606       -15.89
        24          0.00109         0.1205       -18.38
        27          0.00071         0.1029       -19.75
        30          0.00029         0.0649       -23.75
The estimation error falls with signal to noise ratio until it
reaches the floor set by interpolation between pilots.
MMSE equalisation is used, since zero forcing amplifies noise
without limit on subcarriers lying in a deep fade.
```

Command Window Output

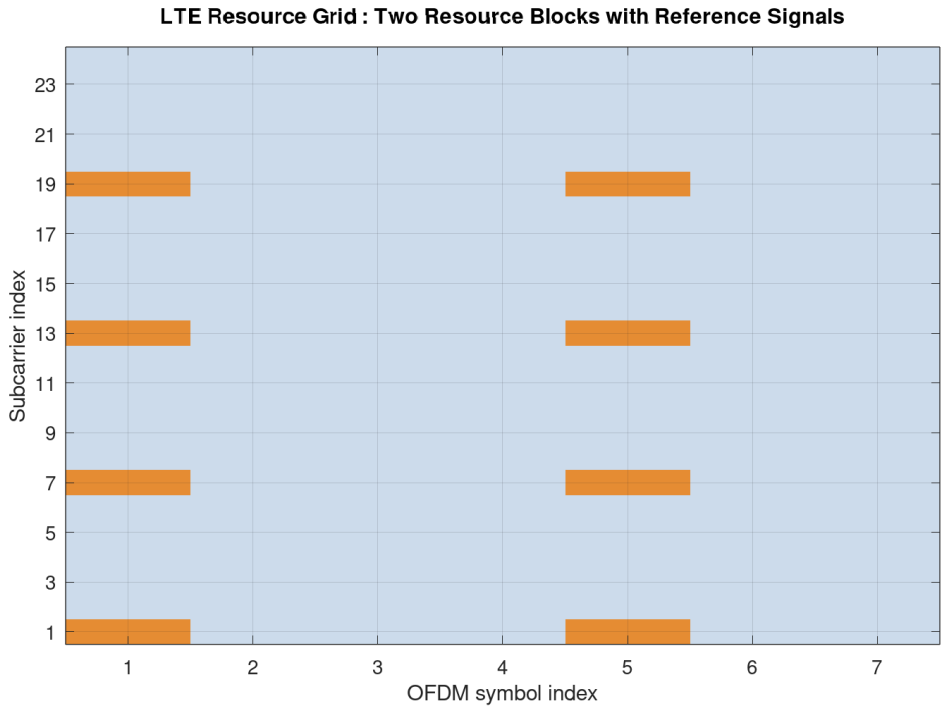


Figure 11.1

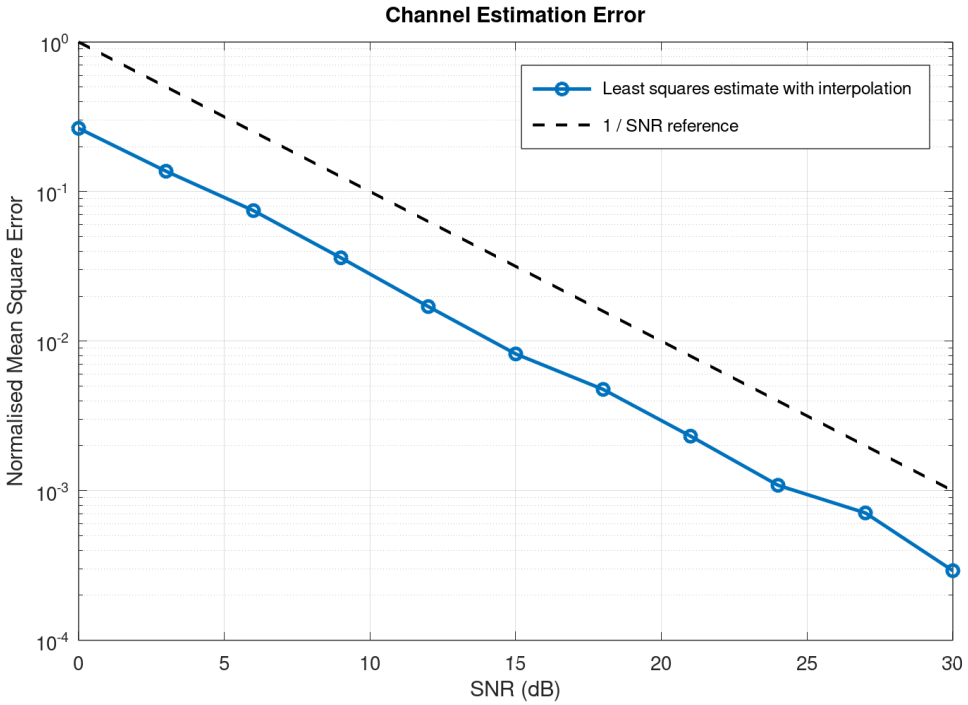


Figure 11.2

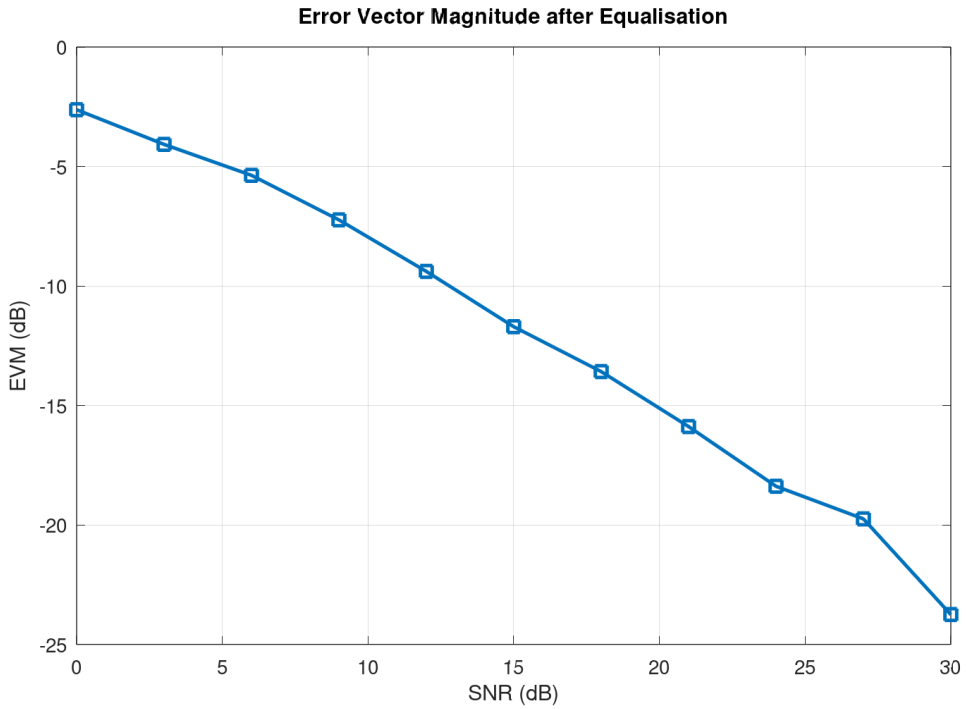


Figure 11.3

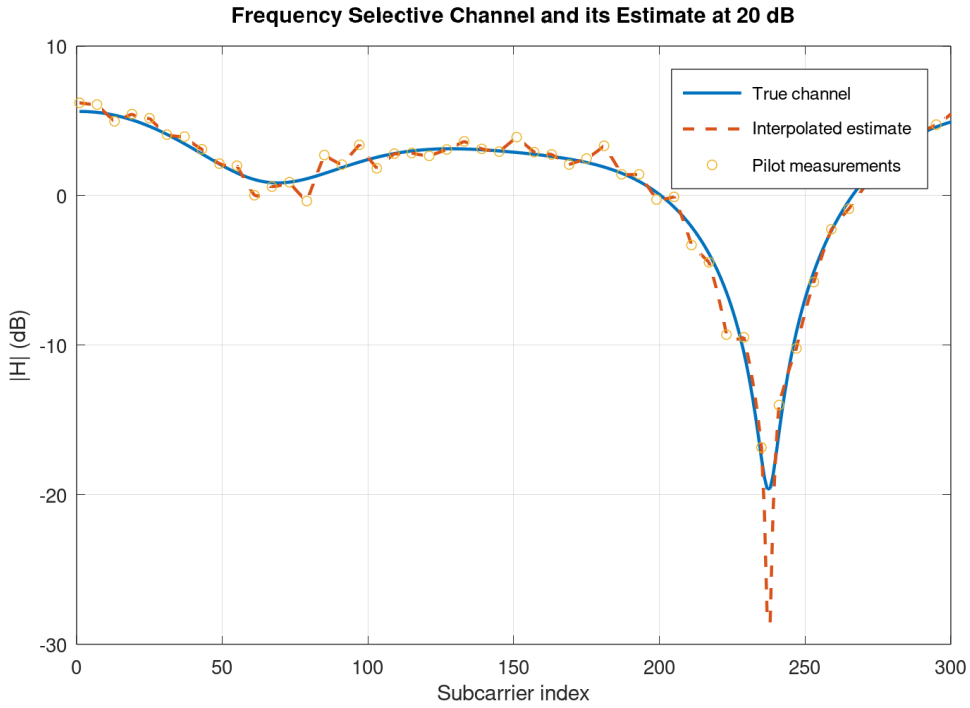


Figure 11.4

Experiment 12

5G NR Numerology and Waveform Comparison

Aim

To compute the 5G New Radio numerologies, compare their symbol and slot durations, cyclic prefix lengths and Doppler tolerance, and relate each to its intended deployment.

Check value (lab manual): *At $\mu = 0$ the useful symbol duration is 66.67 μs and the slot duration is approximately 1 ms; at $\mu = 1$ the slot duration is approximately 0.5 ms.*

Code

```
% Experiment 12 : 5G NR Numerology and Waveform Comparison
clc; clear; close all;
rand('state',12); randn('state',12);

%% Numerology definition
mu   = 0:4;
scs  = 15e3*2.^mu;                    % subcarrier spacing (Hz)
Nfft = 4096;

fprintf('5G NR numerology, subcarrier spacing = 15 x 2^mu kHz\n\n');
fprintf('%5s %10s %12s %12s %12s %12s %10s\n',...
        'mu','SCS (kHz)','Tu (us)','TCP (us)','Tsym (us)','Slot (ms)','Slots/ms');
Tu = zeros(size(mu)); Tcp = Tu; Tsym = Tu; Tslot = Tu;
for k = 1:numel(mu)
    Tu(k)    = 1/scs(k);
    Tcp(k)   = 144/2048*Tu(k);                % normal cyclic prefix
    Tsym(k)  = Tu(k) + Tcp(k);
    Tslot(k) = 14*Tsym(k);
    fprintf('%5d %10.0f %12.3f %12.3f %12.3f %12.4f %10.0f\n',...
            mu(k),scs(k)/1000,Tu(k)*1e6,Tcp(k)*1e6,Tsym(k)*1e6,Tslot(k)*1e3,1e-3/
    ↪ Tslot(k));
end

%% Delay spread and Doppler limits
fprintf('\n%5s %10s %18s %18s %16s\n',...
        'mu','SCS (kHz)','Max delay spread','Max Doppler','Max speed at 3.5 GHz');
fc = 3.5e9; c = 3e8;
for k = 1:numel(mu)
    tmax = Tcp(k);
    % Design criterion adopted for this experiment: the Doppler shift is
    % held below one per cent of the subcarrier spacing so that the
    % intercarrier interference remains small. This is a working heuristic
    % for the exercise, not a value specified by 3GPP.
    fdmax = 0.01*scs(k);
    vmax  = fdmax*c/fc*3.6;
    fprintf('%5d %10.0f %15.3f us %15.0f Hz %13.0f km/h\n',...
            mu(k),scs(k)/1000,tmax*1e6,fdmax,vmax);
end
fprintf('\nA wider subcarrier spacing shortens the symbol and the cyclic\n');
fprintf('prefix, which reduces latency and raises the tolerable Doppler\n');
fprintf('shift, but reduces the delay spread that can be absorbed.\n\n');

%% Deployment mapping
```

```
% A numerology is not tied to a frequency range by the standard. The
% association below is a typical deployment choice, arrived at from the
% delay spread and mobility of each scenario, not a rule.
fprintf('%5s %10s %-30s %s\n','mu','SCS (kHz)','Typical deployment','Governing
    ↪ constraint');
dep = {'Wide area coverage, large cells','Large delay spread';
       'Urban macro and micro cells','Balanced';
       'Small cells, low latency services','Latency and Doppler';
       'Dense small cell and millimetre wave','Phase noise and Doppler';
       'Wideband millimetre wave','Phase noise and Doppler'};
for k = 1:numel(mu)
    fprintf('%5d %10.0f %-30s %s\n',mu(k),scs(k)/1000,dep{k,1},dep{k,2});
end
fprintf('\nA given numerology may be used in more than one frequency band.\n');
fprintf('The band and the numerology are linked through the delay spread\n');
fprintf('and mobility of the deployment, not by any direct rule.\n');

%% Waveform generation and spectral comparison
Nrb = 51; Nsc = 12;
fprintf('\n%5s %10s %14s %16s %16s\n',...
        'mu','SCS (kHz)','Occupied BW','Samples per slot','PAPR 99 % (dB)');
for k = 1:3
    Nu   = Nrb*Nsc;
    Nf   = 2^ceil(log2(Nu/0.85));
    used = [Nf/2-Nu/2+1:Nf/2, Nf/2+2:Nf/2+1+Nu/2];
    Ns   = 14;
    b    = randi([0 1],2,Nu*Ns);
    X    = reshape(((1-2*b(1,:))+1j*(1-2*b(2,:)))/sqrt(2),Nu,Ns);
    G    = zeros(Nf,Ns); G(used,:) = X;
    x    = ifft(ifftshift(G,1),Nf)*sqrt(Nf);
    ncp  = round(144/2048*Nf);
    xc   = [x(end-ncp+1:end,:); x];
    pp   = 10*log10(max(abs(xc).^2,[],1)./mean(abs(xc).^2,1));
    fprintf('%5d %10.0f %11.2f MHz %16d %16.2f\n',...
            mu(k),scs(k)/1000,Nu*scs(k)/1e6,numel(xc),prctile_local(pp,99));
end

%% Figure 1 : symbol duration and slot duration
figure;
subplot(1,2,1);
semilogy(mu,Tsym*1e6,'o-','LineWidth',2); grid on;
xlabel('\mu'); ylabel('Symbol duration (\mus)');
set(gca,'XTick',mu); title('OFDM Symbol Duration');
subplot(1,2,2);
semilogy(mu,Tslot*1e3,'s-','LineWidth',2); grid on;
xlabel('\mu'); ylabel('Slot duration (ms)');
set(gca,'XTick',mu); title('Slot Duration');

%% Figure 2 : cyclic prefix against delay spread
figure;
bar(mu,Tcp*1e6); hold on;
plot([-0.5 4.5],[0.5 0.5],'r--','LineWidth',2);
plot([-0.5 4.5],[0.1 0.1],'k--','LineWidth',2);
grid on; set(gca,'XTick',mu);
xlabel('\mu'); ylabel('Cyclic prefix duration (\mus)');
legend('Normal cyclic prefix','Typical urban delay spread',...
       'Typical indoor delay spread');
title('Cyclic Prefix against Channel Delay Spread');
```

```
100
101 %% Figure 3 : subcarrier spectra for three numerologies
102 figure; hold on;
103 for k = 1:3
104     f = linspace(-4*scs(k),4*scs(k),2000);
105     plot(f/1e3,20*log10(abs(sinc(f/scs(k)))+1e-6),'LineWidth',1.6);
106 end
107 grid on; axis([-500 500 -60 5]);
108 xlabel('Frequency (kHz)'); ylabel('Normalised amplitude (dB)');
109 legend('15 kHz','30 kHz','60 kHz');
110 title('Subcarrier Spectra for Three Numerologies');
111
112 %% Figure 4 : latency against Doppler tolerance
113 figure;
114 plotyy_x = mu;
115 [ax,h1,h2] = plotyy(plotyy_x,Tslot*1e3,plotyy_x,0.01*scs/1000);
116 set(h1,'Marker','o','LineWidth',2); set(h2,'Marker','s','LineWidth',2);
117 set(ax,'XTick',mu);
118 xlabel('\mu'); ylabel(ax(1),'Slot duration (ms)'); ylabel(ax(2),'Tolerable Doppler (
        ↪ kHz)');
119 legend('Slot duration','Tolerable Doppler');
120 title('Latency against Doppler Tolerance');
121 grid on;
122
123 function y = prctile_local(x,p)
124 x = sort(x(:));
125 n = numel(x);
126 y = interp1(100*((1:n)'-0.5)/n, x, min(max(p,100*0.5/n),100*(n-0.5)/n));
127 end
```

Output

```
5G NR numerology, subcarrier spacing = 15 x 2^mu kHz
   mu  SCS (kHz)      Tu (us)     TCP (us)    Tsym (us)    Slot (ms)   Slots/ms
    0         15       66.667        4.688       71.354       0.9990          1
    1         30       33.333        2.344       35.677       0.4995          2
    2         60       16.667        1.172       17.839       0.2497          4
    3        120        8.333        0.586        8.919       0.1249          8
    4        240        4.167        0.293        4.460       0.0624         16
   mu  SCS (kHz)   Max delay spread        Max Doppler Max speed at 3.5 GHz
    0         15           4.688 us             150 Hz            46 km/h
    1         30           2.344 us             300 Hz            93 km/h
    2         60           1.172 us             600 Hz           185 km/h
    3        120           0.586 us            1200 Hz           370 km/h
    4        240           0.293 us            2400 Hz           741 km/h
A wider subcarrier spacing shortens the symbol and the cyclic
prefix, which reduces latency and raises the tolerable Doppler
shift, but reduces the delay spread that can be absorbed.
   mu  SCS (kHz) Typical deployment             Governing constraint
    0         15 Wide area coverage, large cells Large delay spread
    1         30 Urban macro and micro cells    Balanced
    2         60 Small cells, low latency services Latency and Doppler
    3        120 Dense small cell and millimetre wave Phase noise and Doppler
    4        240 Wideband millimetre wave       Phase noise and Doppler
A given numerology may be used in more than one frequency band.
The band and the numerology are linked through the delay spread
and mobility of the deployment, not by any direct rule.
   mu  SCS (kHz)    Occupied BW Samples per slot   PAPR 99 % (dB)
    0         15        9.18 MHz           15344            9.20
    1         30       18.36 MHz           15344           10.18
    2         60       36.72 MHz           15344           10.42
```

Command Window Output

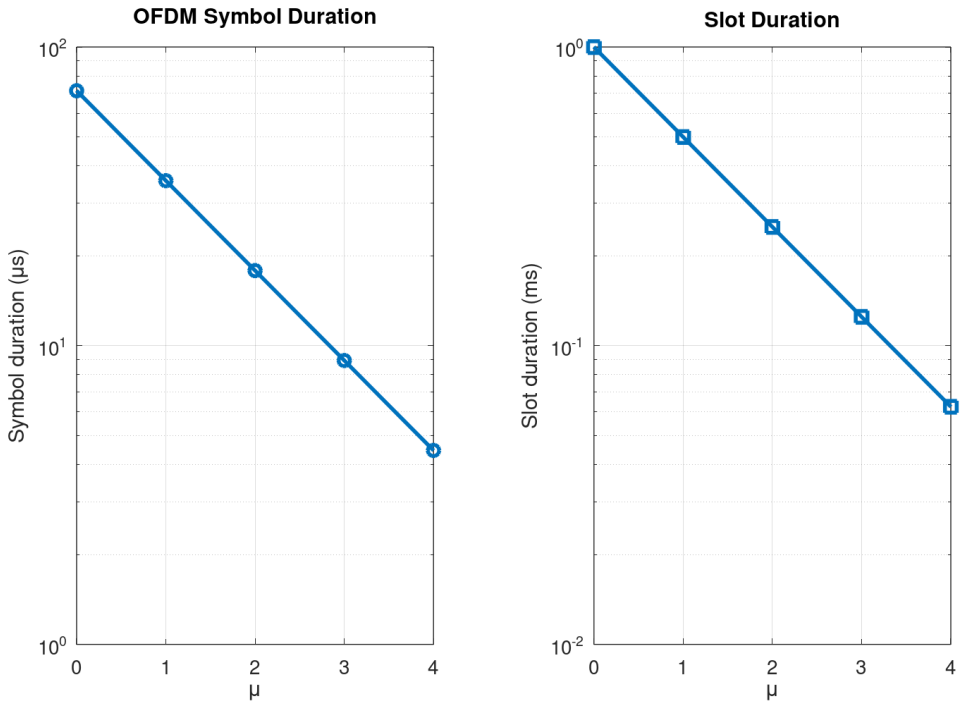


Figure 12.1

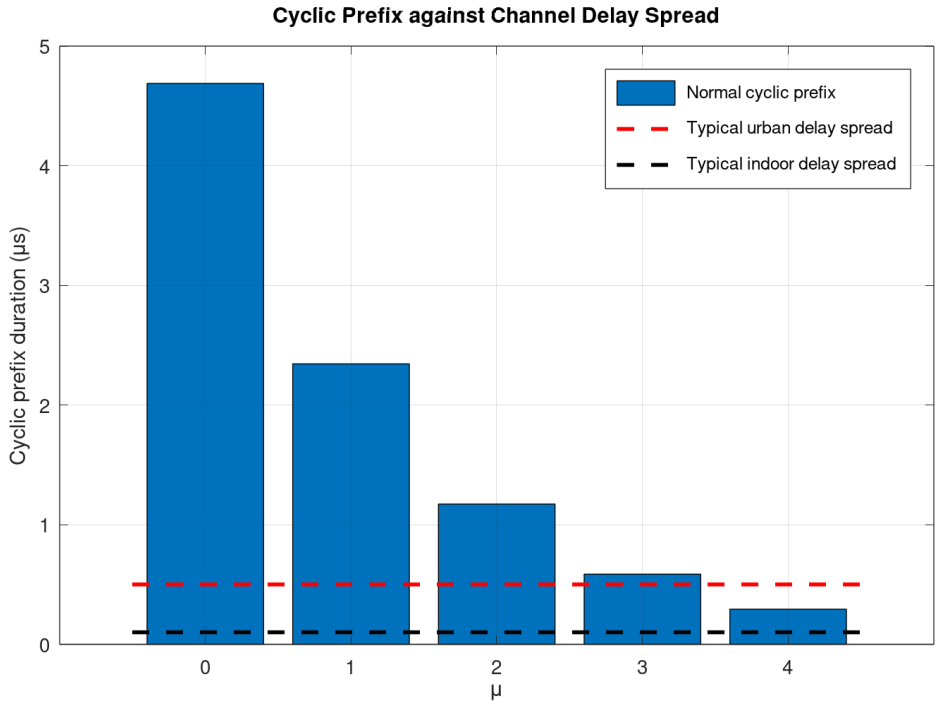


Figure 12.2



Figure 12.3

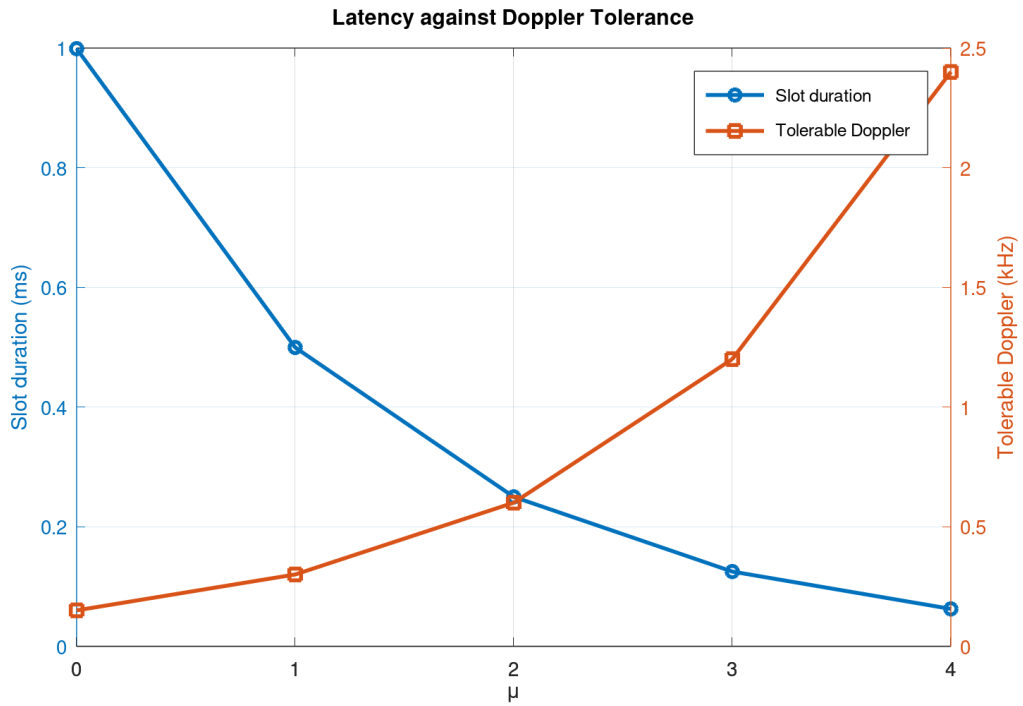


Figure 12.4

Experiment 13

Wireless LAN Basic Service Set — Contention and Scheduled Access

Aim

To simulate a wireless LAN basic service set using the distributed coordination function, determine the packet transmission performance as the number of stations increases, and compare it with the scheduled access used by WiMAX.

Check value (lab manual): *At ten stations the normalised saturation throughput of the distributed coordination function is approximately 0.76, and the analytical and simulated values agree to within one per cent.*

Code

```
% Experiment 13 : Wireless LAN Basic Service Set - CSMA/CA and Scheduled Access
% Syllabus experiments 1 and 2 : packet transmission performance of a BSS,
% and comparison of Wi-Fi with WiMAX.
clc; clear; close all;
rand('state',13);

%% IEEE 802.11 DCF parameters
Rdata   = 1e6;            % channel bit rate (bit/s)
Payload = 8184;           % payload (bits)
MAChdr  = 272;
PHYhdr  = 128;
ACK     = 112 + PHYhdr;
slot    = 20e-6;
SIFS    = 28e-6;
DIFS    = 128e-6;
delta   = 1e-6;           % propagation delay
W       = 32;             % minimum contention window
m       = 3;              % maximum backoff stage

H  = MAChdr + PHYhdr;
Ts = (H + Payload)/Rdata + SIFS + delta + ACK/Rdata + DIFS + delta;
Tc = (H + Payload)/Rdata + DIFS + delta;

fprintf('IEEE 802.11 DCF parameters\n');
fprintf('  Channel bit rate              = %.1f Mbit/s\n',Rdata/1e6);
fprintf('  Payload                       = %d bits\n',Payload);
fprintf('  Successful transmission Ts= %.2f ms\n',Ts*1e3);
fprintf('  Collision duration Tc         = %.2f ms\n',Tc*1e3);
fprintf('  Minimum contention window = %d\n',W);
fprintf('  Idle slot                     = %.0f us\n\n',slot*1e6);

%% Saturation throughput, Bianchi analysis
nvec = [2 5 10 15 20 30 40 50];
S_an = zeros(size(nvec));
tau_v = zeros(size(nvec));
for k = 1:numel(nvec)
    n = nvec(k);
    p = 0.1;
    for it = 1:500
        tau = 2*(1-2*p)/((1-2*p)*(W+1) + p*W*(1-(2*p)^m));
        pn  = 1 - (1-tau)^(n-1);
```

```
        p   = p + 0.5*(pn - p);
    end
    tau = 2*(1-2*p)/((1-2*p)*(W+1) + p*W*(1-(2*p)^m));
    Ptr = 1 - (1-tau)^n;
    Ps  = n*tau*(1-tau)^(n-1)/Ptr;
    S_an(k) = Ps*Ptr*Payload/Rdata / ...
              ((1-Ptr)*slot + Ptr*Ps*Ts + Ptr*(1-Ps)*Tc);
    tau_v(k) = tau;
end

%% Saturation throughput, slotted simulation
Nslots = 2e5;
S_sim  = zeros(size(nvec));
Pcoll  = zeros(size(nvec));
for k = 1:numel(nvec)
    n  = nvec(k);
    bo = floor(rand(1,n)*W);
    st = zeros(1,n);
    T  = 0; succ = 0; coll = 0;
    for it = 1:Nslots
        ready = (bo == 0);
        nr = sum(ready);
        if nr == 0
            T = T + slot; bo = bo - 1;
        elseif nr == 1
            T = T + Ts; succ = succ + 1;
            st(ready) = 0; bo(ready) = floor(rand(1,nr)*W);
            bo(~ready) = bo(~ready) - 1;
        else
            T = T + Tc; coll = coll + 1;
            st(ready) = min(st(ready)+1,m);
            bo(ready) = floor(rand(1,nr).*(W*2.^st(ready)));
            bo(~ready) = bo(~ready) - 1;
        end
        bo = max(bo,0);
    end
    S_sim(k) = succ*Payload/Rdata/T;
    Pcoll(k) = coll/(succ+coll);
end

%% WiMAX scheduled access, no contention
Wpay  = 8184;
Wover = 48 + 88;                        % MAC header and preamble overhead (bits)
Wrate = 1e6;
S_wmx = Wpay/(Wpay+Wover)*ones(size(nvec));

fprintf('%8s %14s %14s %14s %14s\n',...
        'Stations','DCF analysis','DCF simulation','Collision prob','WiMAX TDMA');
for k = 1:numel(nvec)
    fprintf('%8d %14.4f %14.4f %14.4f %14.4f\n',...
            nvec(k),S_an(k),S_sim(k),Pcoll(k),S_wmx(k));
end

fprintf('\nThe normalised throughput of the contention based DCF falls as\n');
fprintf('stations are added, because the collision probability rises. The\n');
fprintf('scheduled WiMAX access retains a constant efficiency because the\n');
fprintf('base station allocates the slots and no collisions occur. The\n');
fprintf('penalty is the signalling required to request and grant slots.\n\n');
```

```
100
101 %% Packet delivery ratio against offered load, non-saturated
102 lam = 0.05:0.05:1.2;
103 n   = 10;
104 pdr = zeros(size(lam));
105 for k = 1:numel(lam)
106     G   = lam(k);
107     Ps  = exp(-2*G);                          % slotted access approximation
108     pdr(k) = min(1,Ps + (1-Ps)*0.5);
109 end
110
111 %% Figure 1 : throughput against number of stations
112 figure;
113 plot(nvec,S_an,'o-','LineWidth',1.8); hold on;
114 plot(nvec,S_sim,'s--','LineWidth',1.8);
115 plot(nvec,S_wmx,'d-','LineWidth',1.8);
116 grid on; axis([0 50 0 1]);
117 xlabel('Number of stations'); ylabel('Normalised throughput');
118 legend('802.11 DCF, analysis','802.11 DCF, simulation','WiMAX scheduled access',...
119        'Location','east');
120 title('Basic Service Set Throughput against Load');
121
122 %% Figure 2 : collision probability
123 figure;
124 plot(nvec,Pcoll,'o-','LineWidth',1.8); hold on;
125 plot(nvec,tau_v,'s-','LineWidth',1.8);
126 grid on; xlabel('Number of stations'); ylabel('Probability');
127 legend('Collision probability','Transmission probability \tau','Location','east');
128 title('Contention Behaviour of the Distributed Coordination Function');
129
130 %% Figure 3 : time spent in each state
131 figure;
132 Tsucc = zeros(size(nvec)); Tcol = Tsucc; Tidl = Tsucc;
133 for k = 1:numel(nvec)
134     n = nvec(k); tau = tau_v(k);
135     Ptr = 1-(1-tau)^n; Ps = n*tau*(1-tau)^(n-1)/Ptr;
136     tot = (1-Ptr)*slot + Ptr*Ps*Ts + Ptr*(1-Ps)*Tc;
137     Tidl(k)  = (1-Ptr)*slot/tot;
138     Tsucc(k) = Ptr*Ps*Ts/tot;
139     Tcol(k)  = Ptr*(1-Ps)*Tc/tot;
140 end
141 bar(nvec,[Tsucc' Tcol' Tidl'],'stacked');
142 xlabel('Number of stations'); ylabel('Fraction of channel time');
143 legend('Successful transmission','Collision','Idle','Location','east');
144 title('Channel Time Utilisation');
```

Output

```
IEEE 802.11 DCF parameters
  Channel bit rate              = 1.0 Mbit/s
  Payload                       = 8184 bits
  Successful transmission Ts= 8.98 ms
  Collision duration Tc         = 8.71 ms
  Minimum contention window = 32
  Idle slot                     = 20 us
Stations   DCF analysis DCF simulation Collision prob     WiMAX TDMA
       2         0.8696         0.8666         0.0326         0.9837
       5         0.8193         0.8175         0.0982         0.9837
      10         0.7584         0.7573         0.1692         0.9837
      15         0.7153         0.7154         0.2172         0.9837
      20         0.6816         0.6820         0.2552         0.9837
      30         0.6293         0.6278         0.3160         0.9837
      40         0.5883         0.5890         0.3595         0.9837
      50         0.5540         0.5533         0.3992         0.9837
The normalised throughput of the contention based DCF falls as
stations are added, because the collision probability rises. The
scheduled WiMAX access retains a constant efficiency because the
base station allocates the slots and no collisions occur. The
penalty is the signalling required to request and grant slots.
```

Command Window Output

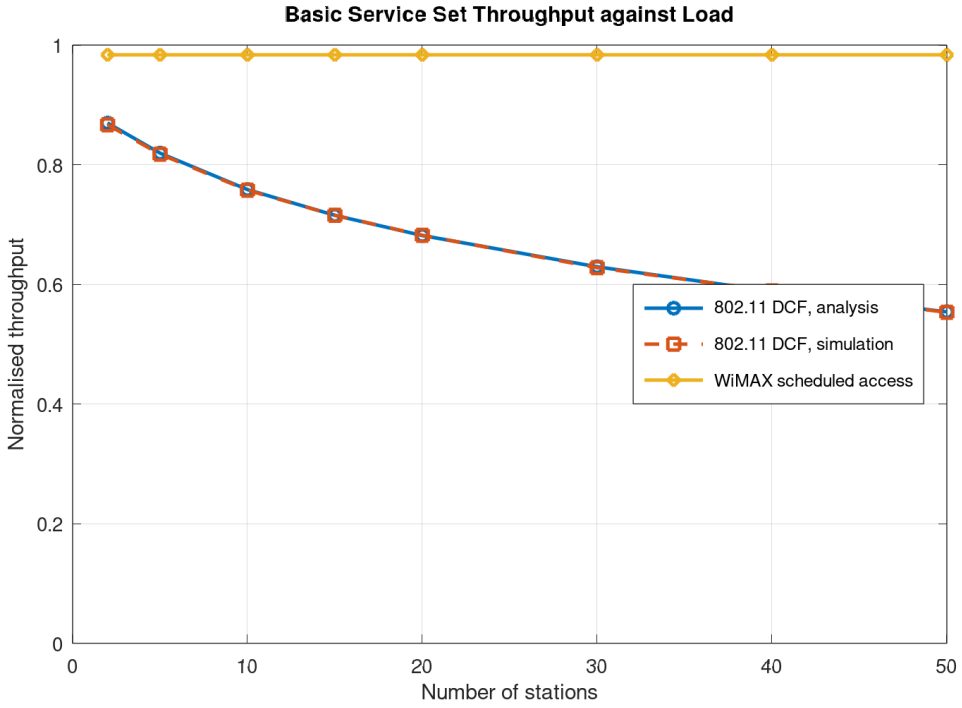


Figure 13.1

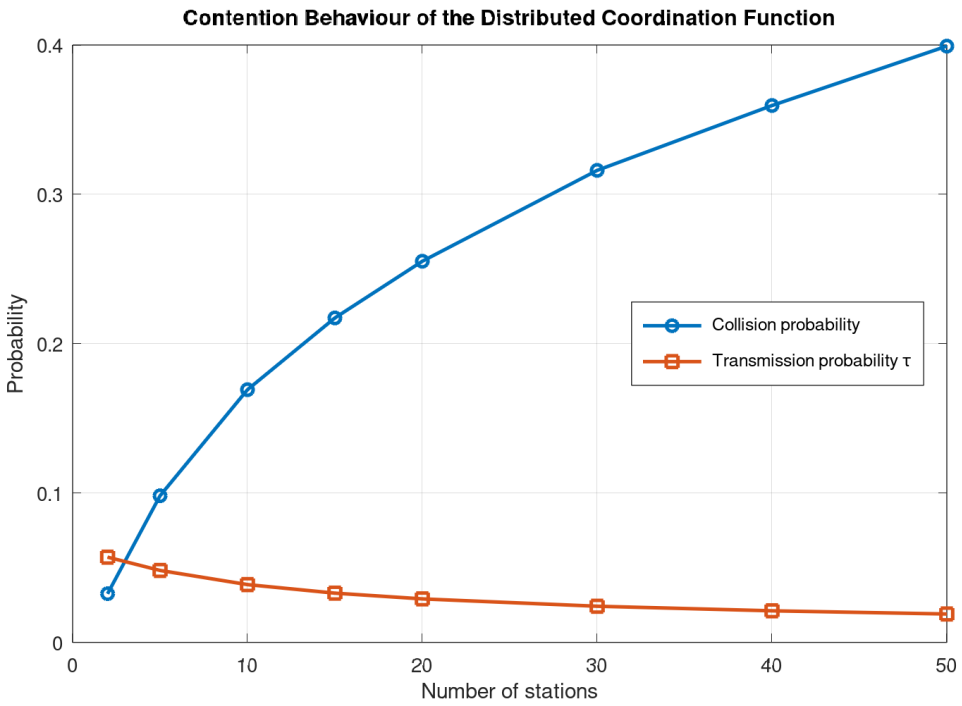


Figure 13.2

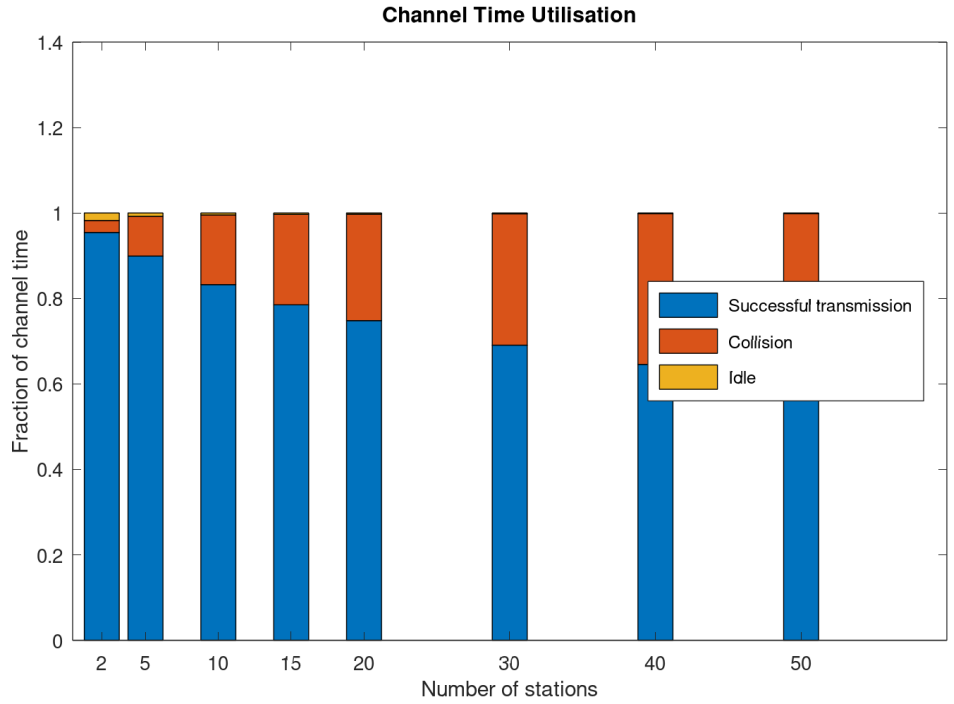


Figure 13.3

Experiment 14

Mobile Ad Hoc Network with an Omnidirectional Antenna Model

Aim

To simulate a mobile ad hoc network of nodes with omnidirectional antennas, determine the transmission range from a path loss model, and analyse connectivity, route length and the connectivity-based delivery ratio against node density and transmission range.

Check value (lab manual): *At 20 dBm with a path loss exponent of 3 and a sensitivity of* -85 *dBm the transmission range is approximately 146 m.*

Code

```
% Experiment 14 : Mobile Ad Hoc Network with an Omnidirectional Antenna Model
% Syllabus experiment 3.
clc; clear; close all;
rand('state',14); randn('state',14);

%% Radio parameters, omnidirectional antennas
Pt   = 20;                  % transmit power (dBm)
Gt   = 0; Gr = 0;           % omnidirectional antennas (dBi)
fc   = 2.4e9;
Sens = -85;                 % receiver sensitivity (dBm)
n_pl = 3.0;                 % path loss exponent
d0   = 1;                   % reference distance (m)
c    = 3e8;
lambda = c/fc;

PL0  = 20*log10(4*pi*d0/lambda);
Rmax = d0*10^((Pt+Gt+Gr-Sens-PL0)/(10*n_pl));
fprintf('Omnidirectional antenna model\n');
fprintf('  Transmit power        = %d dBm\n',Pt);
fprintf('  Receiver sensitivity  = %d dBm\n',Sens);
fprintf('  Path loss exponent    = %.1f\n',n_pl);
fprintf('  Reference loss at 1 m = %.2f dB\n',PL0);
fprintf('  Transmission range    = %.1f m\n\n',Rmax);

%% Network area and node counts
A      = 500;                    % square side (m)
nvec   = 10:5:60;
Ntrial = 200;

conn = zeros(size(nvec)); hops = conn; pdr = conn; deg = conn;

for k = 1:numel(nvec)
    N = nvec(k);
    cc = 0; hh = 0; pp = 0; dd = 0; cnt = 0;
    for t = 1:Ntrial
        x = A*rand(1,N); y = A*rand(1,N);
        D = sqrt((x-x').^2 + (y-y').^2);
        Adj = (D <= Rmax) & ~eye(N);
        dd = dd + mean(sum(Adj,2));

        % breadth first search from node 1 gives hop counts
        hopmat = bfs_hops(Adj);
```

```
        reach  = hopmat(1,:);
        reach(1) = 0;
        cc = cc + mean(isfinite(reach));
        fin = reach(isfinite(reach) & reach > 0);
        if ~isempty(fin), hh = hh + mean(fin); cnt = cnt + 1; end

        % connectivity based delivery ratio, 50 random source-destination
        % pairs. A pair counts as delivered when a multihop path exists; the
        % simulation does not model medium access or collisions.
        s = randi(N,1,50); d = randi(N,1,50);
        ok = 0;
        for q = 1:50
            if s(q) ~= d(q) && isfinite(hopmat(s(q),d(q))), ok = ok + 1; end
        end
        pp = pp + ok/50;
    end
    conn(k) = cc/Ntrial; hops(k) = hh/max(cnt,1);
    pdr(k)  = pp/Ntrial; deg(k)  = dd/Ntrial;
end

fprintf('%8s %14s %16s %14s %14s\n',...
        'Nodes','Mean degree','Reachable frac','Mean hops','Conn. delivery');
for k = 1:numel(nvec)
    fprintf('%8d %14.2f %16.4f %14.2f %14.4f\n',...
            nvec(k),deg(k),conn(k),hops(k),pdr(k));
end

kcrit = find(pdr > 0.95,1);
if isempty(kcrit), kcrit = numel(nvec); end
fprintf('\nThe delivery ratio first exceeds 0.95 at %d nodes, where the mean\n',nvec(
    ↪ kcrit));
fprintf('node degree is %.1f. Below this density the graph partitions\n',deg(kcrit));
fprintf('and packets addressed to unreachable nodes are lost, however good\n');
fprintf('the routing protocol may be.\n\n');

%% Effect of transmission range at fixed node count
N = 30;
rvec = 60:20:260;
pdr_r = zeros(size(rvec));
for k = 1:numel(rvec)
    pp = 0;
    for t = 1:Ntrial
        x = A*rand(1,N); y = A*rand(1,N);
        D = sqrt((x-x').^2 + (y-y').^2);
        Adj = (D <= rvec(k)) & ~eye(N);
        hm = bfs_hops(Adj);
        s = randi(N,1,50); d = randi(N,1,50);
        ok = 0;
        for q = 1:50
            if s(q) ~= d(q) && isfinite(hm(s(q),d(q))), ok = ok + 1; end
        end
        pp = pp + ok/50;
    end
    pdr_r(k) = pp/Ntrial;
end
fprintf('%16s %24s\n','Range (m)','Connectivity delivery ratio');
for k = 1:numel(rvec)
    fprintf('%16d %24.4f\n',rvec(k),pdr_r(k));
```

```
end
fprintf('\nRaising the transmission power extends the range and improves\n');
fprintf('connectivity, but increases interference and drains the battery,\n');
fprintf('so the useful operating point lies just above the connectivity\n');
fprintf('threshold rather than at maximum power.\n\n');

%% Figure 1 : one network realisation with routes
N = 30;
x = A*rand(1,N); y = A*rand(1,N);
D = sqrt((x-x').^2 + (y-y').^2);
Adj = (D <= Rmax) & ~eye(N);
figure; hold on;
for i = 1:N
    for j = i+1:N
        if Adj(i,j), plot([x(i) x(j)],[y(i) y(j)],'Color',[0.75 0.8 0.85]); end
    end
end
plot(x,y,'o','MarkerFaceColor',[0.20 0.40 0.65],'MarkerSize',7);
[pathN,~] = bfs_path(Adj,1,N);
if ~isempty(pathN)
    plot(x(pathN),y(pathN),'r-','LineWidth',2.5);
    plot(x(pathN),y(pathN),'rs','MarkerSize',9,'LineWidth',1.5);
end
axis equal; axis([0 A 0 A]); grid on;
xlabel('x (m)'); ylabel('y (m)');
title(sprintf('MANET, %d nodes, range %.0f m, route shown in red',N,Rmax));

%% Figure 2 : connectivity against node count
figure;
plot(nvec,pdr,'o-','LineWidth',1.8); hold on;
plot(nvec,conn,'s-','LineWidth',1.8);
grid on; axis([min(nvec) max(nvec) 0 1.05]);
xlabel('Number of nodes'); ylabel('Fraction');
legend('Connectivity delivery ratio','Reachable node fraction','Location','southeast'
    ↪ );
title('Connectivity against Node Density');

%% Figure 3 : mean degree and hop count
figure;
subplot(1,2,1);
plot(nvec,deg,'o-','LineWidth',1.8); grid on;
xlabel('Number of nodes'); ylabel('Mean node degree');
title('Node Degree');
subplot(1,2,2);
plot(nvec,hops,'s-','LineWidth',1.8); grid on;
xlabel('Number of nodes'); ylabel('Mean hop count');
title('Route Length');

%% Figure 4 : delivery ratio against transmission range
figure;
plot(rvec,pdr_r,'o-','LineWidth',1.8); hold on;
plot([Rmax Rmax],[0 1.05],'k--','LineWidth',1.5);
grid on; axis([min(rvec) max(rvec) 0 1.05]);
xlabel('Transmission range (m)'); ylabel('Connectivity delivery ratio');
legend('Connectivity delivery ratio','Range at 20 dBm','Location','southeast');
title('Effect of Transmission Range, 30 Nodes');

function Hm = bfs_hops(Adj)
```

```
% Hop count between every pair of nodes by breadth first search.
% Unreachable pairs are returned as Inf.
N  = size(Adj,1);
Hm = inf(N);
for s = 1:N
    d = inf(1,N); d(s) = 0; q = s;
    while ~isempty(q)
        v = q(1); q(1) = [];
        nb = find(Adj(v,:) & isinf(d));
        d(nb) = d(v) + 1;
        q = [q nb];
    end
    Hm(s,:) = d;
end
end

function [p,h] = bfs_path(Adj,s,t)
% Shortest hop path from s to t by breadth first search.
N = size(Adj,1);
prev = zeros(1,N); d = inf(1,N); d(s) = 0; q = s;
while ~isempty(q)
    v = q(1); q(1) = [];
    nb = find(Adj(v,:) & isinf(d));
    d(nb) = d(v) + 1; prev(nb) = v;
    q = [q nb];
end
if isinf(d(t)), p = []; h = Inf; return; end
p = t; while p(1) ~= s, p = [prev(p(1)) p]; end
h = d(t);
end
```

Output

```
Omnidirectional antenna model
  Transmit power        = 20 dBm
  Receiver sensitivity  = -85 dBm
  Path loss exponent    = 3.0
  Reference loss at 1 m = 40.05 dB
  Transmission range    = 146.3 m

   Nodes    Mean degree   Reachable frac      Mean hops Conn. delivery
      10           1.93           0.4240           1.46         0.3640
      15           2.88           0.5683           1.94         0.5086
      20           3.91           0.7228           2.43         0.6874
      25           4.96           0.8810           2.70         0.8295
      30           5.96           0.9607           2.86         0.9134
      35           6.92           0.9760           2.79         0.9435
      40           8.02           0.9846           2.73         0.9645
      45           9.10           0.9983           2.69         0.9742
      50          10.10           0.9992           2.64         0.9785
      55          11.12           0.9995           2.63         0.9829
      60          12.17           0.9990           2.59         0.9818

The delivery ratio first exceeds 0.95 at 40 nodes, where the mean
node degree is 8.0. Below this density the graph partitions
and packets addressed to unreachable nodes are lost, however good
the routing protocol may be.

       Range (m) Connectivity delivery ratio
              60                     0.0731
              80                     0.1749
             100                     0.4280
             120                     0.7058
             140                     0.8909
             160                     0.9383
             180                     0.9590
             200                     0.9640
             220                     0.9680
             240                     0.9683
             260                     0.9664

Raising the transmission power extends the range and improves
connectivity, but increases interference and drains the battery,
so the useful operating point lies just above the connectivity
threshold rather than at maximum power.
```

Command Window Output

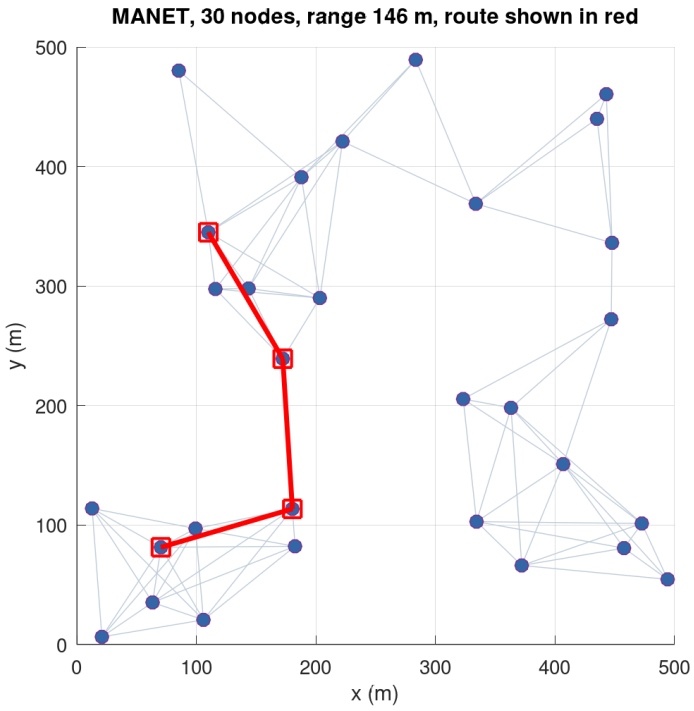


Figure 14.1

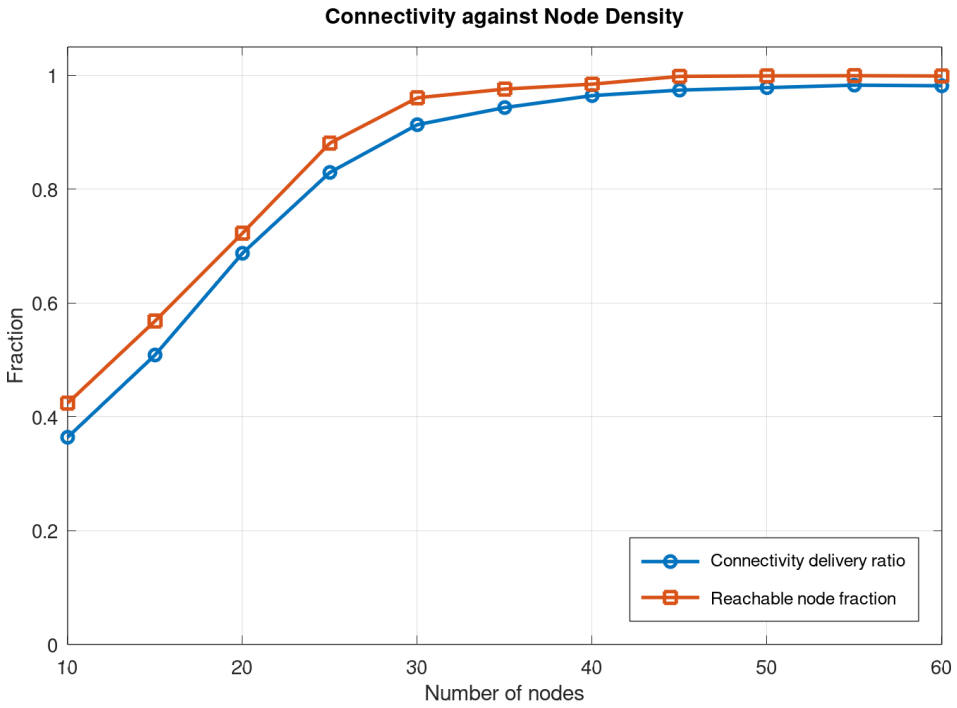


Figure 14.2

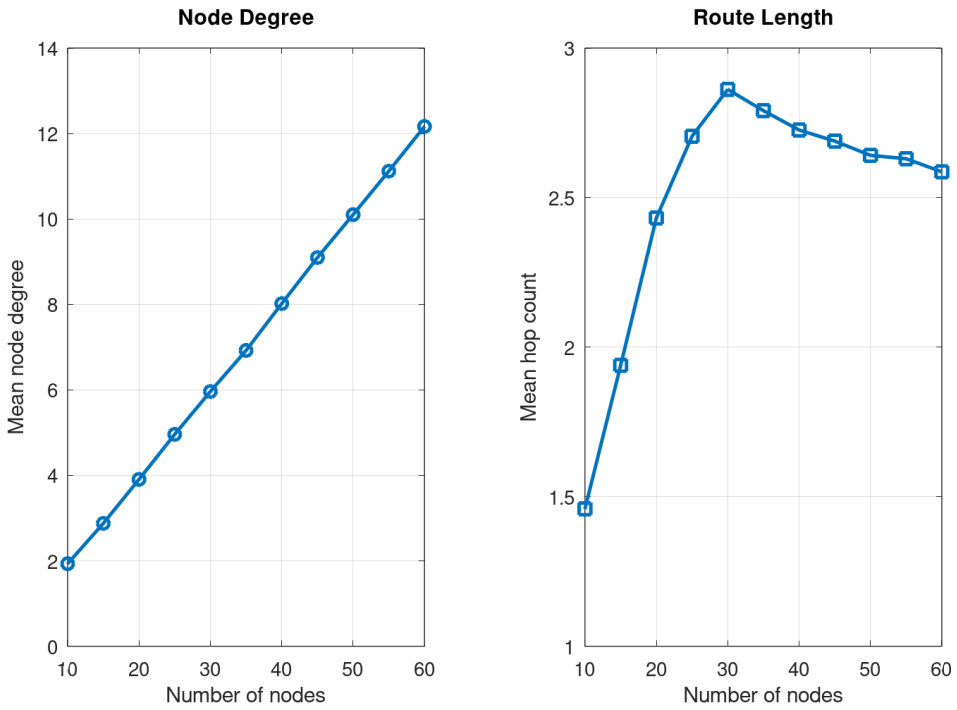


Figure 14.3

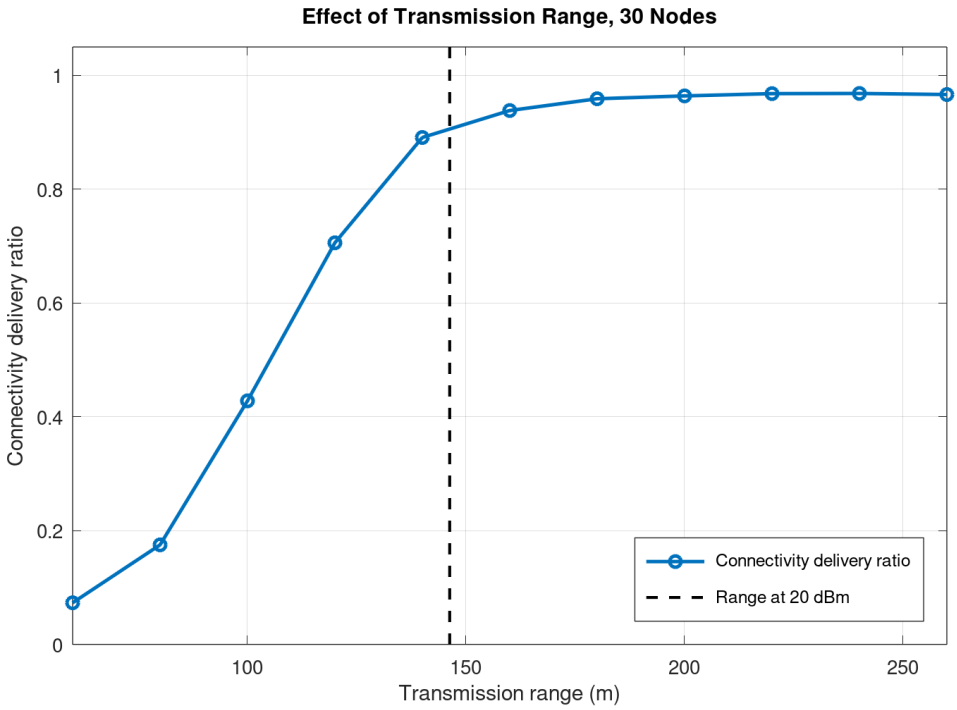

Effect of Transmission Range, 30 Nodes
Connectivity delivery ratio
Transmission range (m)
0
0.2
0.4
0.6
0.8
1
100
150
200
250
Connectivity delivery ratio
Range at 20 dBm

Figure 14.4

Experiment 15

Optical Fibre Link — Attenuation, Power Budget, Digital and Analog Performance

Aim

To compute the attenuation spectrum of an optical fibre, establish the power budget and maximum link length, and determine the performance of a digital link and of an analog link.

Check value (lab manual): *A bit error rate of* 10^{-9} *corresponds to a Q factor of 6. The scattering component decreases approximately with the inverse fourth power of wavelength, while the total attenuation also includes absorption losses.*

Code

```
% Experiment 15 : Optical Fibre Link - Analog Link, Digital Link and Fibre Loss
% Syllabus experiments 4, 5 and 6, implemented as MATLAB link computations.
clc; clear; close all;

%% PART A : Fibre attenuation against wavelength
lam  = 0.8:0.005:1.7;                          % wavelength (um)
Aray = 0.9./lam.^4;                            % Rayleigh scattering
Air  = 6.5e-3*exp((lam-1.55)/0.06);            % infrared absorption
Aoh  = 0.35*exp(-((lam-1.39)/0.025).^2);       % hydroxyl ion peak
Atot = Aray + Air + Aoh + 0.02;

fprintf('PART A : Fibre attenuation\n');
win = [0.85 1.31 1.55];
nam = {'First','Second','Third'};
fprintf('%10s %14s %18s\n','Window','Wavelength (um)','Attenuation (dB/km)');
alp = zeros(1,3);
for k = 1:3
    [~,i] = min(abs(lam-win(k)));
    alp(k) = Atot(i);
    fprintf('%10s %14.2f %18.4f\n',nam{k},win(k),alp(k));
end
[amin,imin] = min(Atot);
fprintf('\nMinimum attenuation %.4f dB/km at %.3f um\n',amin,lam(imin));
fprintf('Rayleigh scattering varies as 1/lambda^4 and dominates below\n');
fprintf('1.3 um; infrared absorption rises beyond 1.6 um. The minimum\n');
fprintf('between them is the reason long haul systems work at 1.55 um.\n\n');

%% PART B : Optical power budget and maximum link length
Ptx     = 0;                % launched power (dBm)
Prx     = -30;              % receiver sensitivity (dBm)
Lconn   = 0.5;              % connector loss (dB each)
Nconn   = 2;
Lsplice = 0.1;              % splice loss (dB each)
Lspan   = 2;                % splice spacing (km)
Margin  = 6;                % system margin (dB)

fprintf('PART B : Power budget at 1.55 um\n');
fprintf('  Launched power        = %.1f dBm\n',Ptx);
fprintf('  Receiver sensitivity  = %.1f dBm\n',Prx);
fprintf('  Total budget          = %.1f dB\n',Ptx-Prx);
fprintf('  Connector loss        = %.1f dB\n',Nconn*Lconn);
```

```
fprintf('  System margin         = %.1f dB\n',Margin);
avail = (Ptx-Prx) - Nconn*Lconn - Margin;
fprintf('  Available for fibre   = %.1f dB\n',avail);
Lmax = avail/(alp(3) + Lsplice/Lspan);
fprintf('  Fibre attenuation     = %.4f dB/km\n',alp(3));
fprintf('  Splice loss           = %.3f dB/km\n',Lsplice/Lspan);
fprintf('  Maximum link length   = %.2f km\n\n',Lmax);

%% PART C : Digital link, bit error rate against received power
Rb    = 2.5e9;
lamd  = 1.55e-6;
h     = 6.626e-34; cc = 3e8; q = 1.602e-19;
Rresp = 0.9;                                % responsivity (A/W)
Id    = 5e-9;                               % dark current (A)
RL    = 50; T = 300; kB = 1.381e-23;
Fn    = 3;                                  % amplifier noise figure (linear)
Be    = Rb/2;

Prx_dBm = -40:0.05:-15;
Prx_W   = 1e-3*10.^(Prx_dBm/10);
Ip      = Rresp*Prx_W;
sig_sh  = sqrt(2*q*(Ip+Id)*Be);
sig_th  = sqrt(4*kB*T*Fn*Be/RL);
% Shot and thermal noise are statistically independent, so their variances
% add. The total rms noise is the square root of the sum of the squares,
% not the sum of the two rms values.
sig_tot = sqrt(sig_sh.^2 + sig_th.^2);
Qf      = Ip./sig_tot;
BER     = 0.5*erfc(Qf/sqrt(2));

fprintf('PART C : Digital link at %.1f Gbit/s\n',Rb/1e9);
fprintf('%14s %12s %16s\n','Power (dBm)','Q factor','BER');
for p = [-35 -30 -28 -26 -24 -22 -20]
    [~,i] = min(abs(Prx_dBm-p));
    fprintf('%14.1f %12.2f %16.3e\n',p,Qf(i),BER(i));
end
[~,i9] = min(abs(BER-1e-9));
fprintf('\nReceiver sensitivity at BER = 1e-9 : %.2f dBm (Q = %.2f)\n',...
        Prx_dBm(i9),Qf(i9));
fprintf('A bit error rate of 1e-9 corresponds to Q = 6, which is the\n');
fprintf('standard design point for a digital optical receiver.\n\n');

%% PART D : Analog link, carrier to noise ratio
m      = 0.5;                               % optical modulation index
RIN_dB = -155;                              % relative intensity noise (dB/Hz)
Ba     = 4e6;                               % analog bandwidth (Hz)
RIN    = 10^(RIN_dB/10);
Ipa    = Rresp*Prx_W;
Csig   = 0.5*(m*Ipa).^2;
Nrin   = RIN*Ipa.^2*Ba;
Nsh    = 2*q*Ipa*Ba;
Nth    = 4*kB*T*Fn*Ba/RL;
CNR    = 10*log10(Csig./(Nrin+Nsh+Nth));

fprintf('PART D : Analog link, modulation index %.1f\n',m);
fprintf('%14s %12s %12s %12s %12s\n',...
        'Power (dBm)','CNR (dB)','RIN share','Shot share','Thermal share');
for p = [-30 -25 -20 -15]
```

```
    [~,i] = min(abs(Prx_dBm-p));
    tot = Nrin(i)+Nsh(i)+Nth;
    fprintf('%14.1f %12.2f %12.3f %12.3f %12.3f\n',...
            p,CNR(i),Nrin(i)/tot,Nsh(i)/tot,Nth/tot);
end
fprintf('\nAt low received power the thermal noise of the receiver dominates\n');
fprintf('and the carrier to noise ratio rises 2 dB for every 1 dB of\n');
fprintf('optical power. At high power the relative intensity noise of the\n');
fprintf('source dominates and the carrier to noise ratio saturates.\n');

%% Figure 1 : attenuation spectrum
figure;
plot(lam,Atot,'LineWidth',2); hold on;
plot(lam,Aray,'--','LineWidth',1.4);
plot(lam,Air,'-.','LineWidth',1.4);
for k = 1:3, plot(win(k),alp(k),'ro','MarkerFaceColor','r','MarkerSize',6); end
grid on; axis([0.8 1.7 0 2]);
xlabel('Wavelength (\mum)'); ylabel('Attenuation (dB/km)');
legend('Total','Rayleigh scattering','Infrared absorption','Operating windows');
title('Optical Fibre Attenuation Spectrum');

%% Figure 2 : power budget against link length
figure;
Lk = 0:1:120;
for k = 1:3
    plot(Lk,Ptx - Nconn*Lconn - (alp(k)+Lsplice/Lspan)*Lk,'LineWidth',1.8); hold on;
end
plot([0 120],[Prx Prx],'k--','LineWidth',1.6);
plot([0 120],[Prx+Margin Prx+Margin],'k:','LineWidth',1.6);
grid on; axis([0 120 -45 5]);
xlabel('Link length (km)'); ylabel('Received power (dBm)');
legend('0.85 \mum','1.31 \mum','1.55 \mum','Sensitivity','Sensitivity + margin');
title('Optical Power Budget');

%% Figure 3 : digital link performance
figure;
semilogy(Prx_dBm,BER,'LineWidth',2); hold on;
semilogy([min(Prx_dBm) max(Prx_dBm)],[1e-9 1e-9],'k--','LineWidth',1.5);
plot([Prx_dBm(i9) Prx_dBm(i9)],[1e-14 1],'r:','LineWidth',1.5);
grid on; axis([-40 -15 1e-14 1]);
xlabel('Received optical power (dBm)'); ylabel('Bit error rate');
legend('BER','1 \times 10^{-9} reference','Receiver sensitivity');
title(sprintf('Digital Optical Link at %.1f Gbit/s',Rb/1e9));

%% Figure 4 : analog link carrier to noise ratio
figure;
plot(Prx_dBm,CNR,'LineWidth',2); hold on;
plot(Prx_dBm,10*log10(Csig./Nth),'--','LineWidth',1.4);
plot(Prx_dBm,10*log10(Csig./Nrin),'-.','LineWidth',1.4);
grid on; axis([-40 -15 0 60]);
xlabel('Received optical power (dBm)'); ylabel('CNR (dB)');
legend('Total CNR','Thermal noise limit','RIN limit','Location','southeast');
title('Analog Optical Link Carrier to Noise Ratio');
```

Output

```
PART A : Fibre attenuation
    Window Wavelength (um) Attenuation (dB/km)
     First           0.85             1.7441
    Second           1.31             0.3257
     Third           1.55             0.1824
Minimum attenuation 0.1715 dB/km at 1.615 um
Rayleigh scattering varies as 1/lambda^4 and dominates below
1.3 um; infrared absorption rises beyond 1.6 um. The minimum
between them is the reason long haul systems work at 1.55 um.
PART B : Power budget at 1.55 um
  Launched power        = 0.0 dBm
  Receiver sensitivity  = -30.0 dBm
  Total budget          = 30.0 dB
  Connector loss        = 1.0 dB
  System margin         = 6.0 dB
  Available for fibre   = 23.0 dB
  Fibre attenuation     = 0.1824 dB/km
  Splice loss           = 0.050 dB/km
  Maximum link length   = 98.96 km
PART C : Digital link at 2.5 Gbit/s
   Power (dBm)     Q factor              BER
         -35.0         0.26        3.993e-01
         -30.0         0.81        2.098e-01
         -28.0         1.28        1.004e-01
         -26.0         2.03        2.133e-02
         -24.0         3.21        6.591e-04
         -22.0         5.09        1.800e-07
         -20.0         8.06        3.780e-16
Receiver sensitivity at BER = 1e-9 : -21.30 dBm (Q = 5.98)
A bit error rate of 1e-9 corresponds to Q = 6, which is the
standard design point for a digital optical receiver.
PART D : Analog link, modulation index 0.5
   Power (dBm)     CNR (dB)    RIN share   Shot share Thermal share
         -30.0        14.06        0.000        0.000        1.000
         -25.0        24.05        0.000        0.001        0.999
         -20.0        34.05        0.000        0.003        0.997
         -15.0        44.02        0.000        0.009        0.991
At low received power the thermal noise of the receiver dominates
and the carrier to noise ratio rises 2 dB for every 1 dB of
optical power. At high power the relative intensity noise of the
source dominates and the carrier to noise ratio saturates.
```

Command Window Output

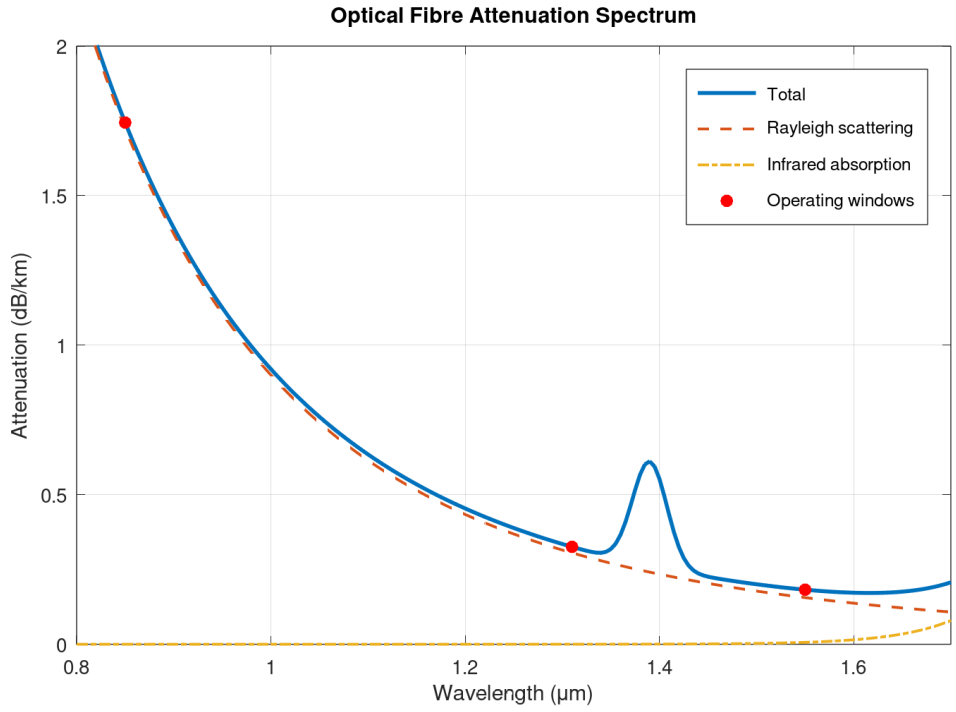


Figure 15.1

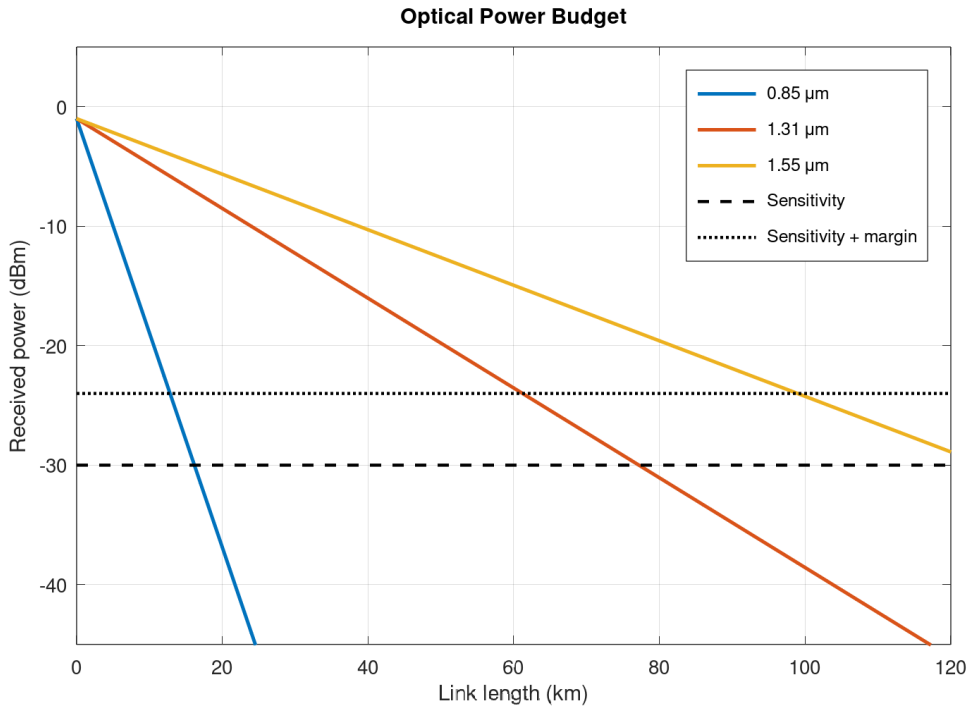


Figure 15.2

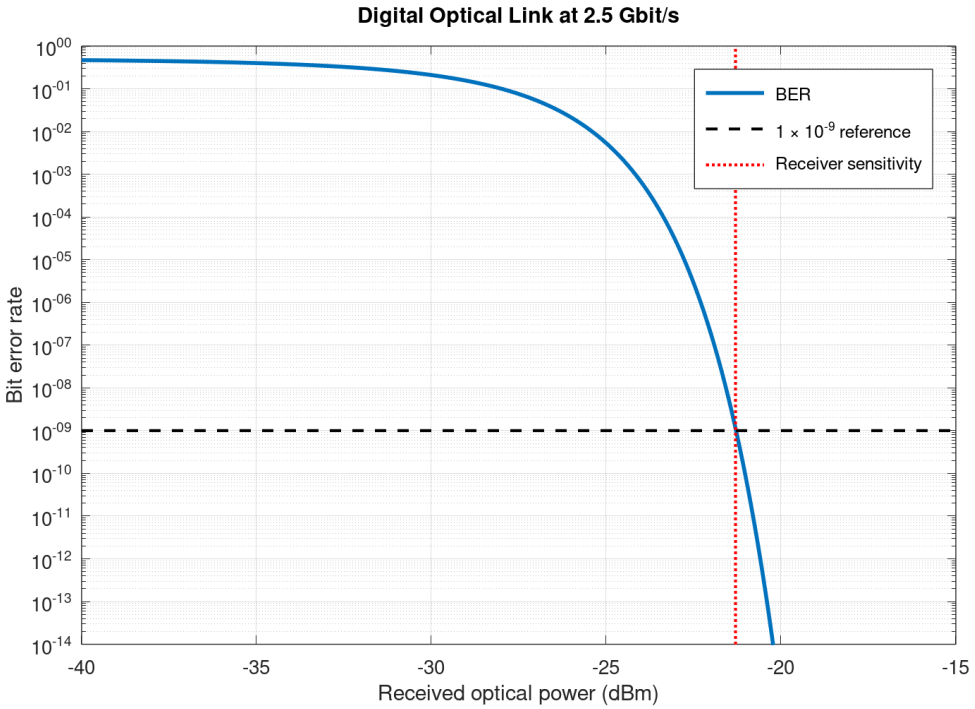


Figure 15.3

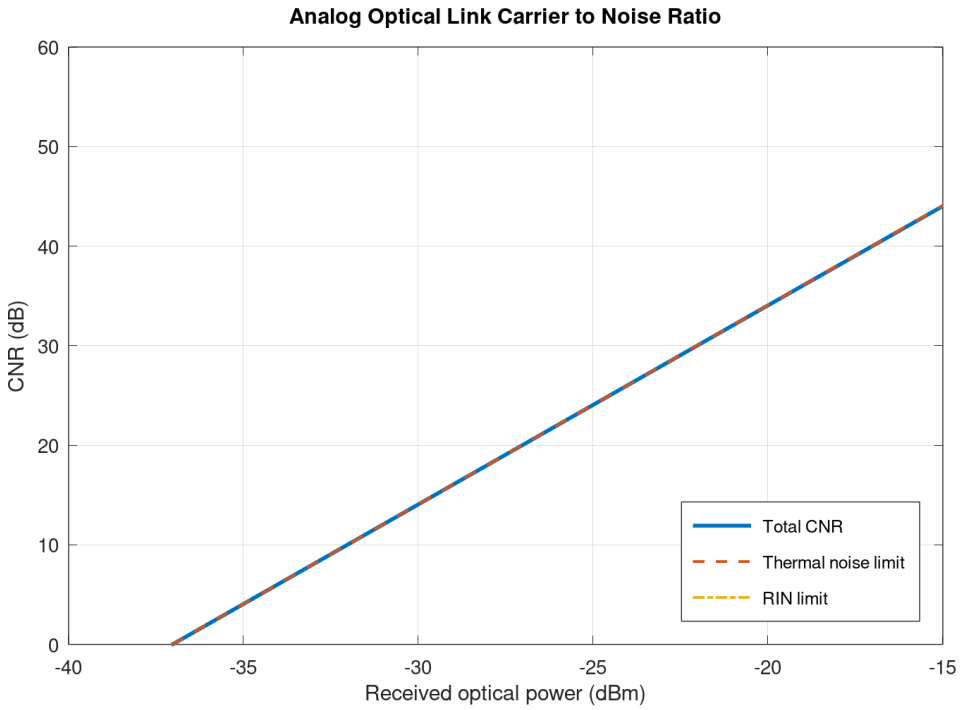

Analog Optical Link Carrier to Noise Ratio
CNR (dB)
Received optical power (dBm)
Total CNR
Thermal noise limit
RIN limit
60
50
40
30
20
10
0
-40
-35
-30
-25
-20
-15

Figure 15.4